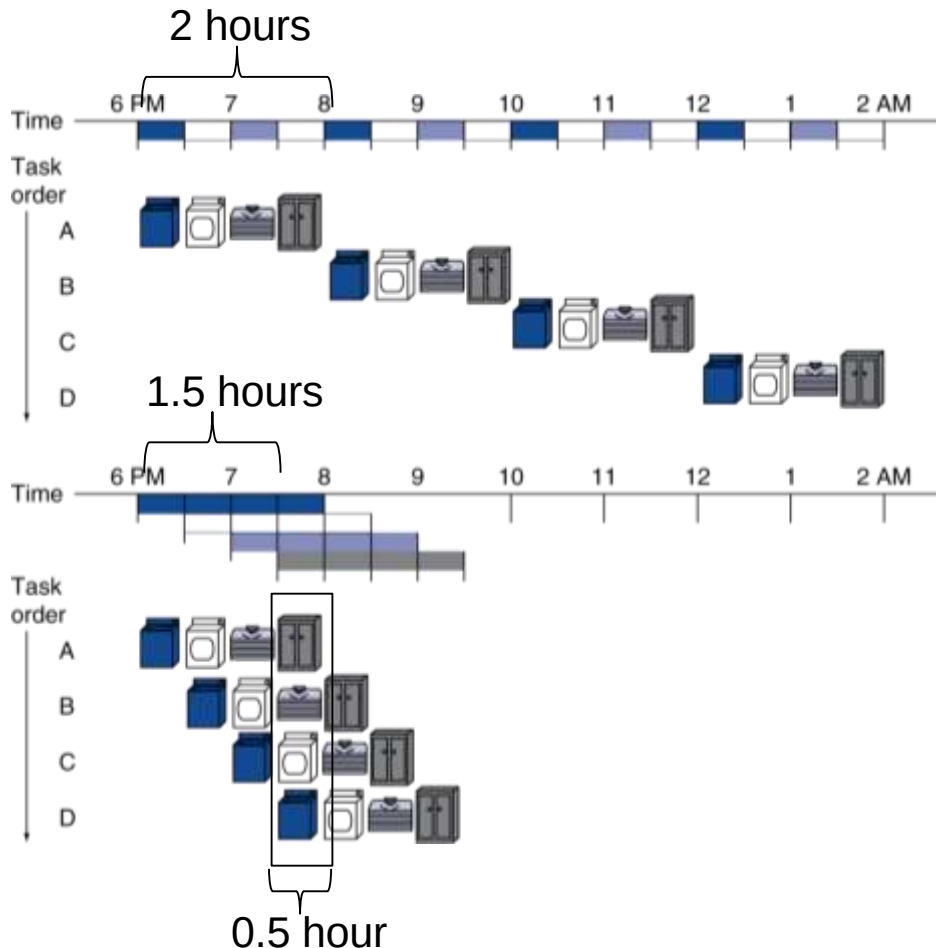


# **Chapter 4**

## **The Processor**

# Pipelining Analogy

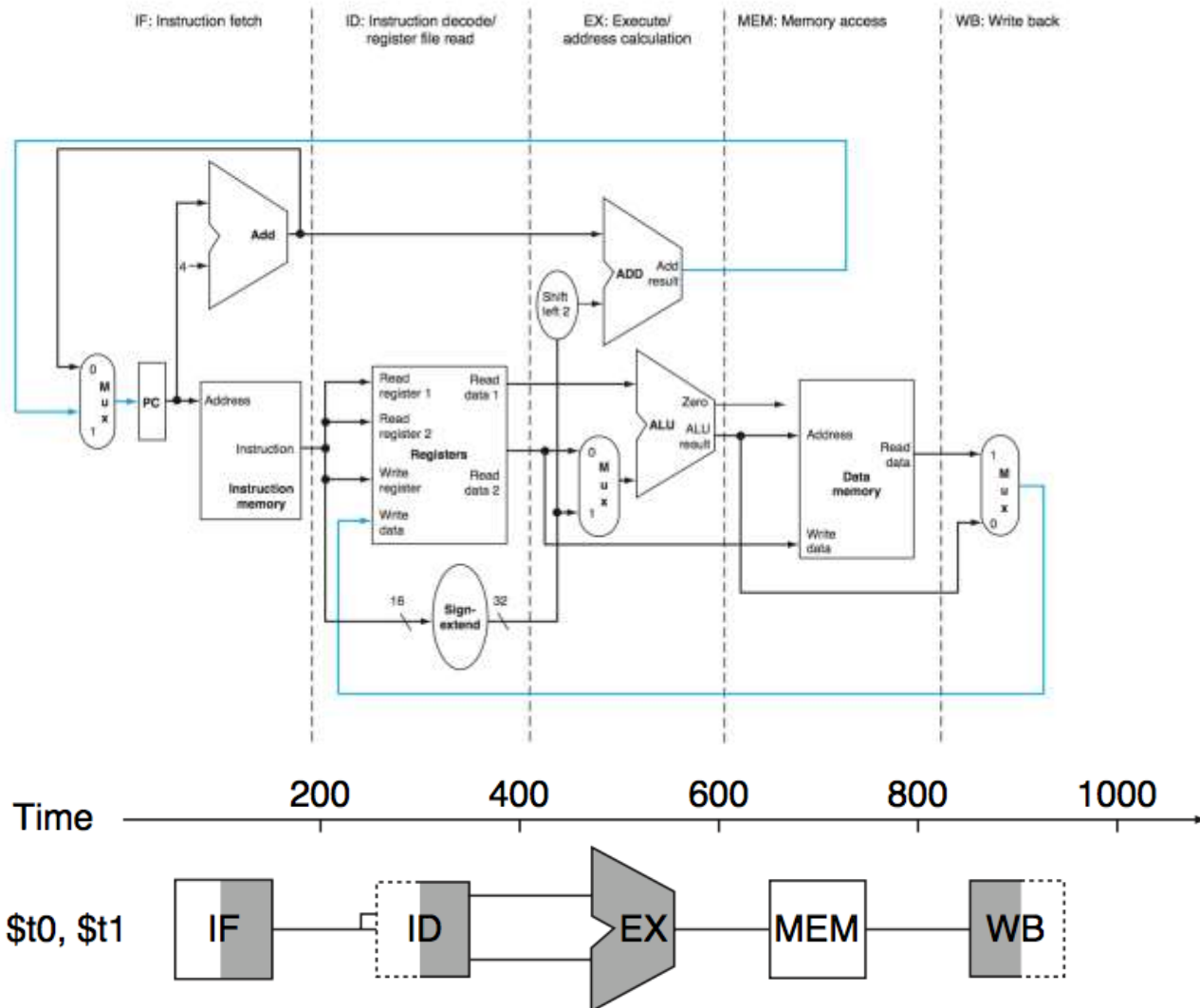
- Pipelined laundry: overlapping execution
  - Parallelism improves performance



# MIPS Pipeline

- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

# MIPS Stages



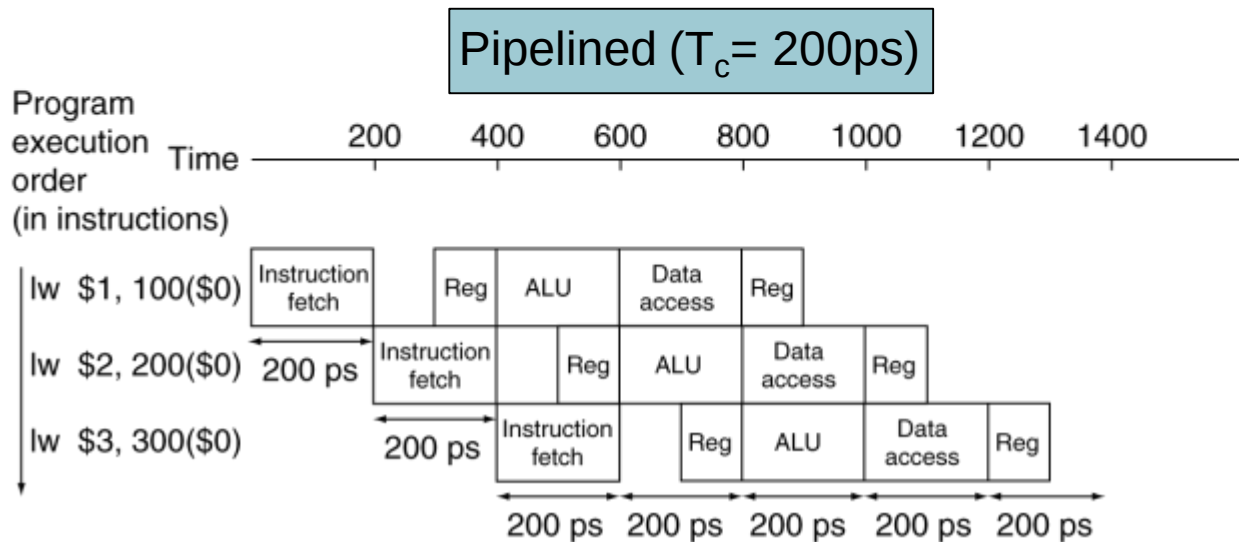
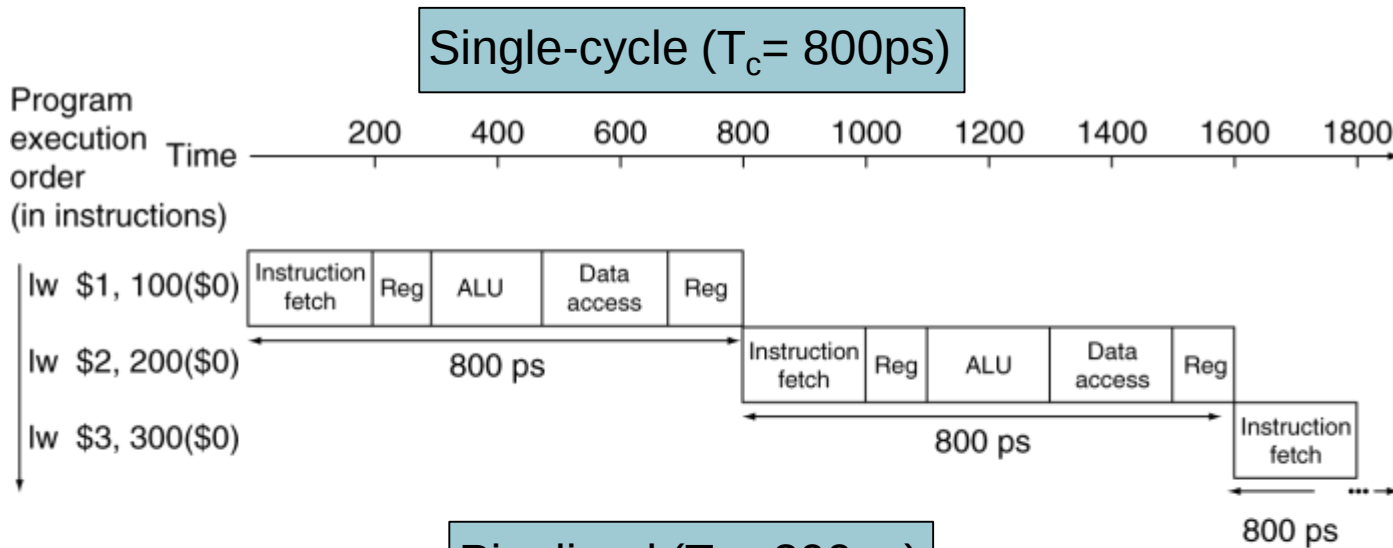
# Pipeline Performance

- Assume time for stages is
  - 100ps for register read or write
  - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

| Instr    | Instr fetch | Register read | ALU op | Memory access | Register write | Total time |
|----------|-------------|---------------|--------|---------------|----------------|------------|
| lw       | 200ps       | 100 ps        | 200ps  | 200ps         | 100 ps         | 800ps      |
| sw       | 200ps       | 100 ps        | 200ps  | 200ps         |                | 700ps      |
| R-format | 200ps       | 100 ps        | 200ps  |               | 100 ps         | 600ps      |
| beq      | 200ps       | 100 ps        | 200ps  |               |                | 500ps      |

# Pipeline Performance

$T_c$  = Time between instructions



# Pipeline Speedup

- If all stages are balanced
  - i.e., all take the same time
  - Time between instructions<sub>pipelined</sub>  
=  $\frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$ 
    - In our example,  $800/5 = 160\text{ps}$
- If not balanced, speedup is less
- Speedup due to increased throughput
  - Latency (time for each instruction) does not decrease

# Pipeline Speedup

- Our example with 3 instructions
  - $T_{\text{pipelined}} = 1400\text{ps}$
  - $T_{\text{nopipelined}} = 2400\text{ps}$
  - $\text{Speedup} = 2400/1400 = 1.714$
- Add 1,000,000 instructions
  - $T_{\text{pipelined}} = 1,000,000 * 200 + 1400$
  - $T_{\text{nopipelined}} = 1,000,000 * 800 + 2400$
  - $\text{Speedup} = 800,002,400/200,001,400 = 4$

# Pipelining and ISA Design

- MIPS ISA designed for pipelining
  - All instructions are 32-bits
    - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to 17-byte instructions
  - Few and regular instruction formats
    - Can decode and read registers in one step
  - Load/store addressing
    - Can calculate address in 3<sup>rd</sup> stage, access memory in 4<sup>th</sup> stage
  - Alignment of memory operands
    - Memory access takes only one cycle

# Hazards

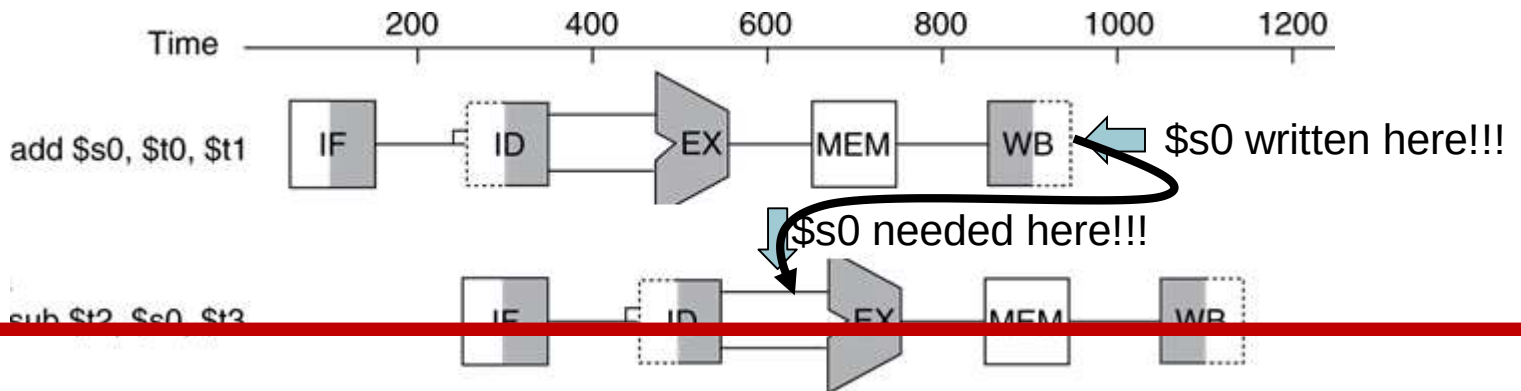
- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
  - A required resource is busy
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction

# Structure Hazards

- Conflict for use of a resource
- In MIPS pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to *stall* for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

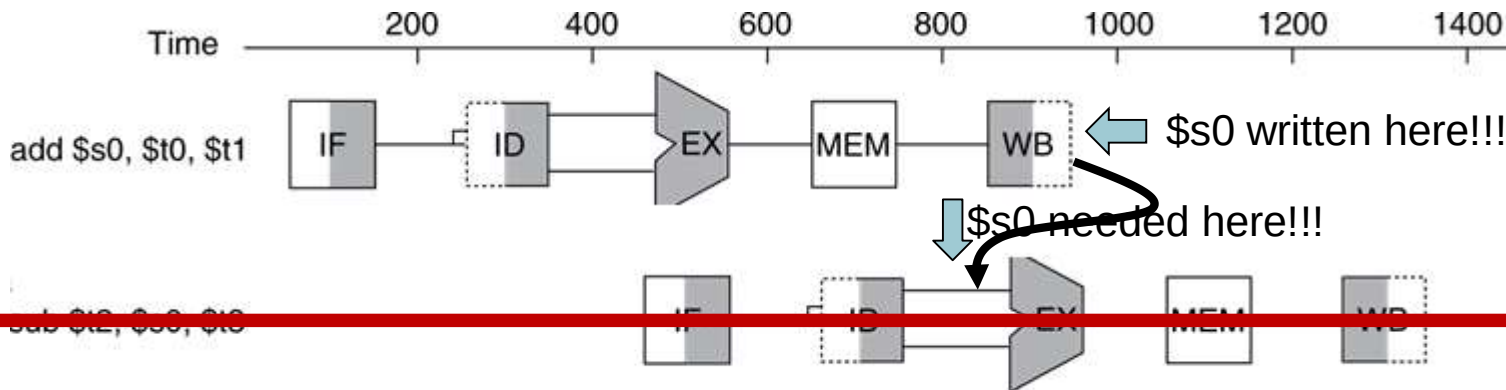
# Data Hazards

- An instruction depends on completion of data access by a previous instruction
  - add **\$s0**, \$t0, \$t1
  - sub \$t2, **\$s0**, \$t3



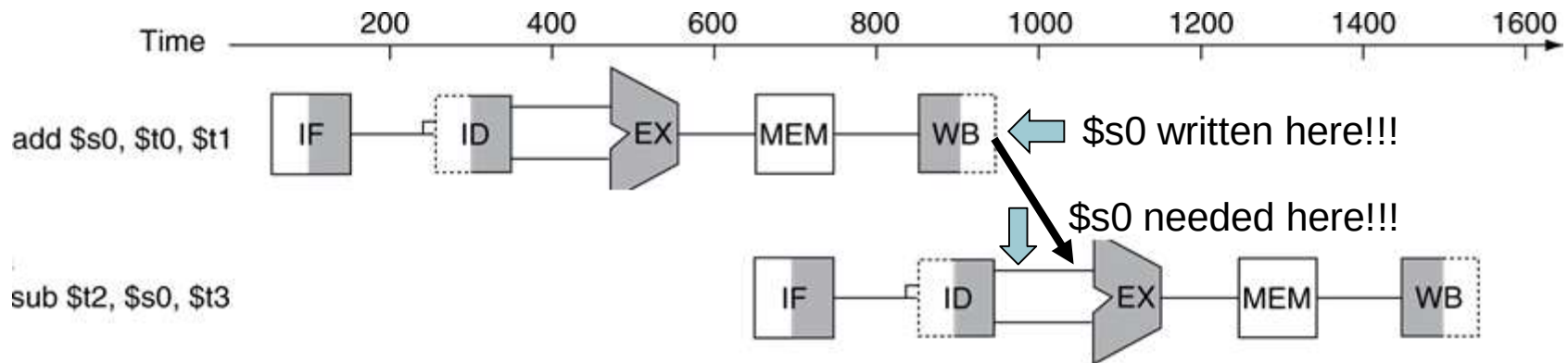
# Data Hazards

- An instruction depends on completion of data access by a previous instruction
  - add **\$s0**, \$t0, \$t1
  - sub \$t2, **\$s0**, \$t3



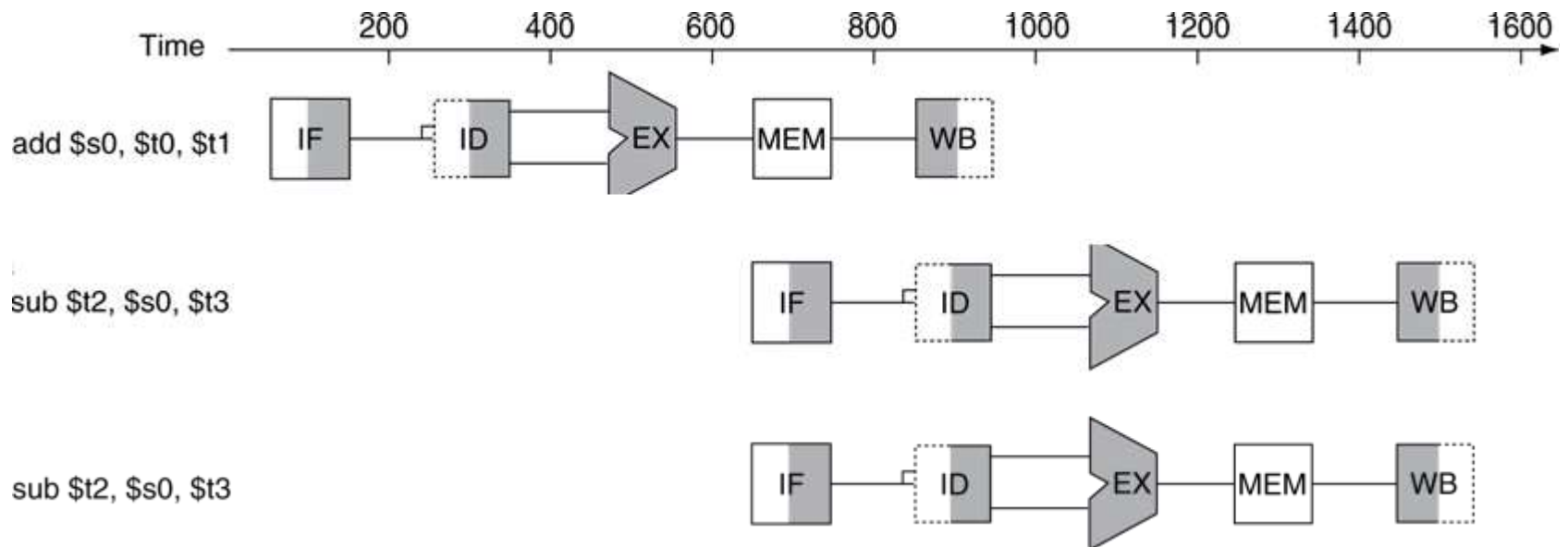
# Data Hazards

- An instruction depends on completion of data access by a previous instruction
  - add **\$s0**, \$t0, \$t1
  - sub \$t2, **\$s0**, \$t3

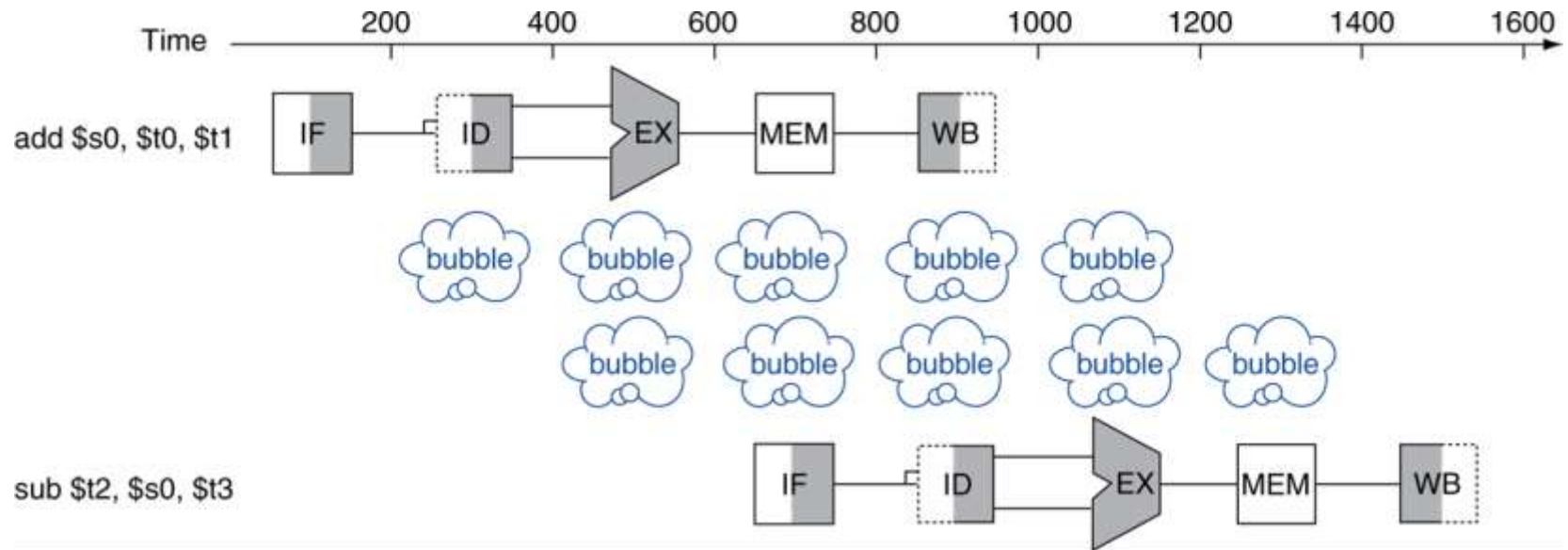


# Data Hazards

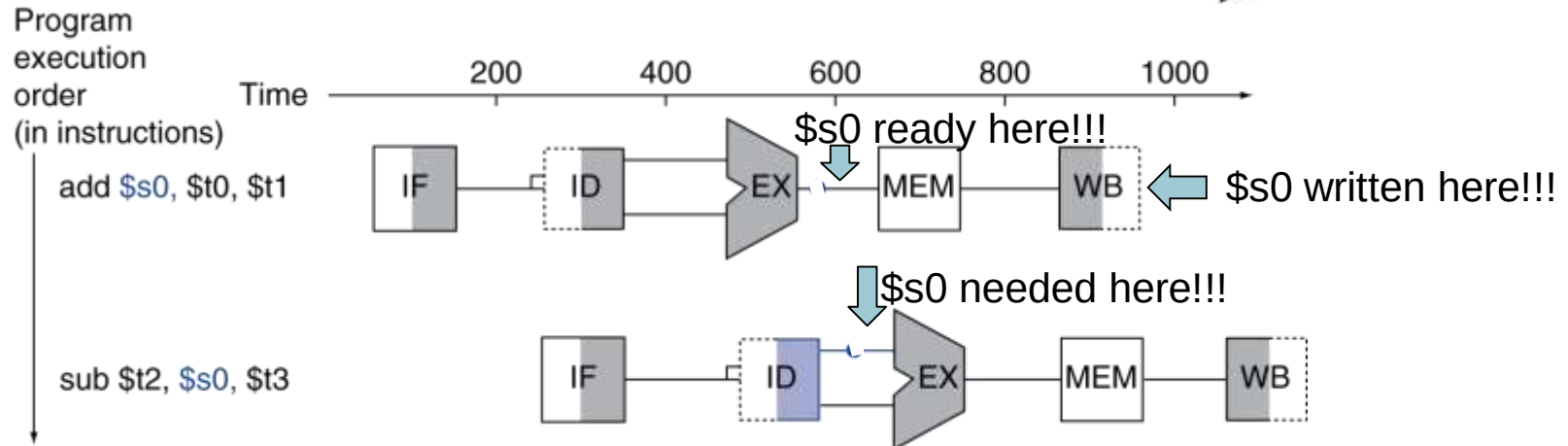
- An instruction depends on completion of data access by a previous instruction
  - add **\$s0**, \$t0, \$t1
  - sub \$t2, **\$s0**, \$t3



# Forwarding (aka Bypassing)

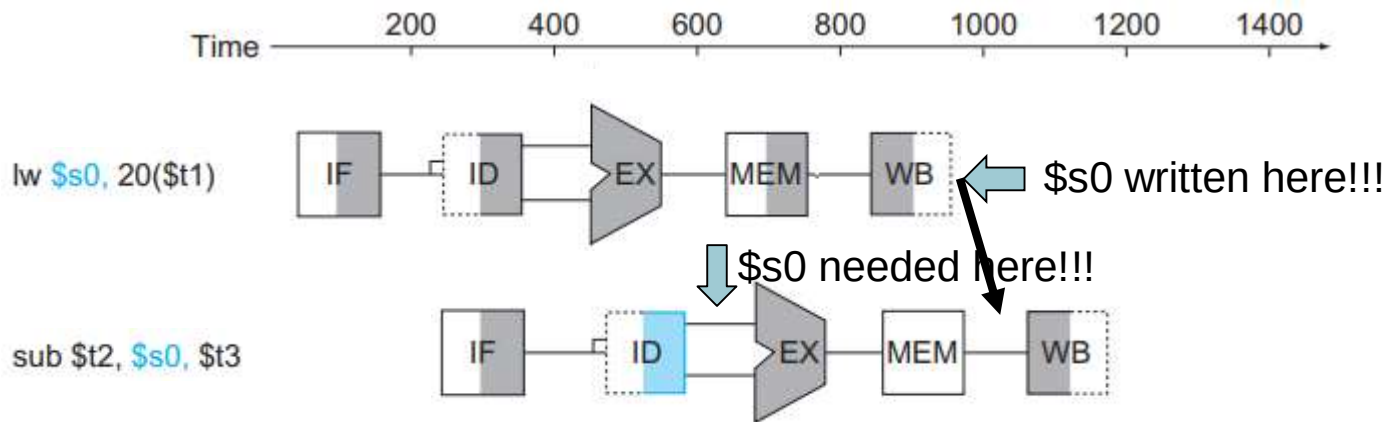


COMPARE



# Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!



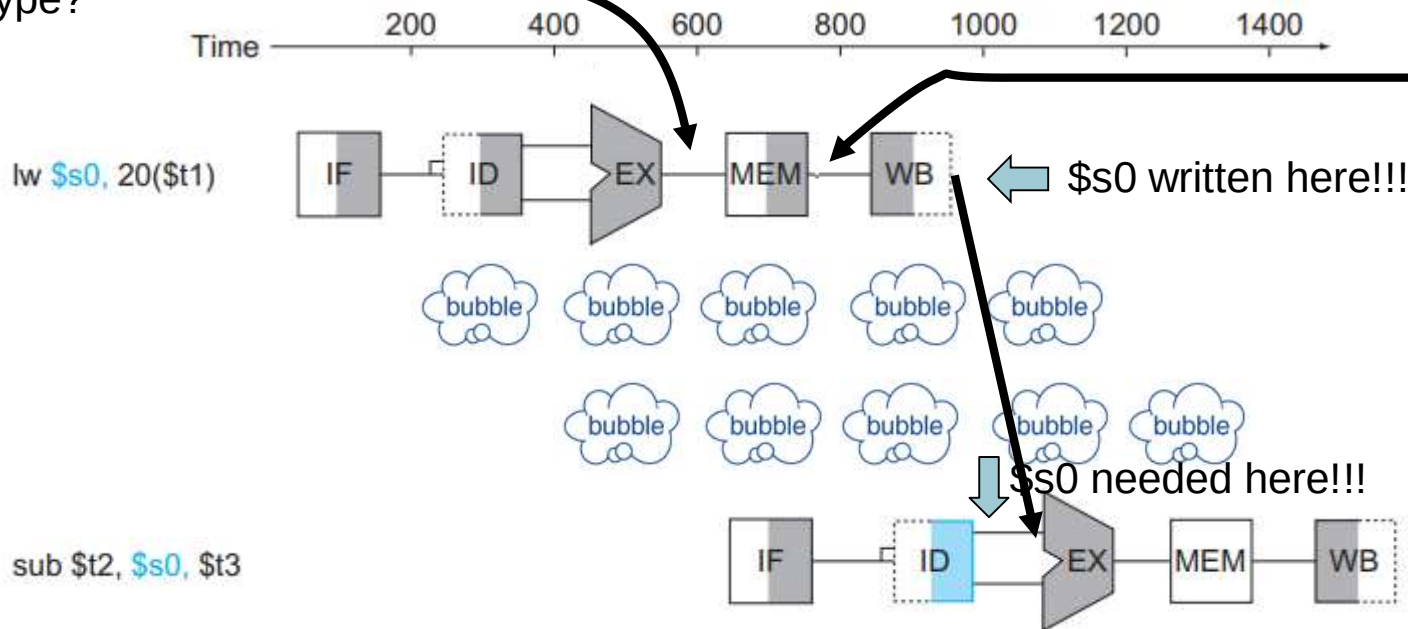
# Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!

When WAS the data Ready for R-type?

Can we solve by FORWARDING?

When is the data Ready?



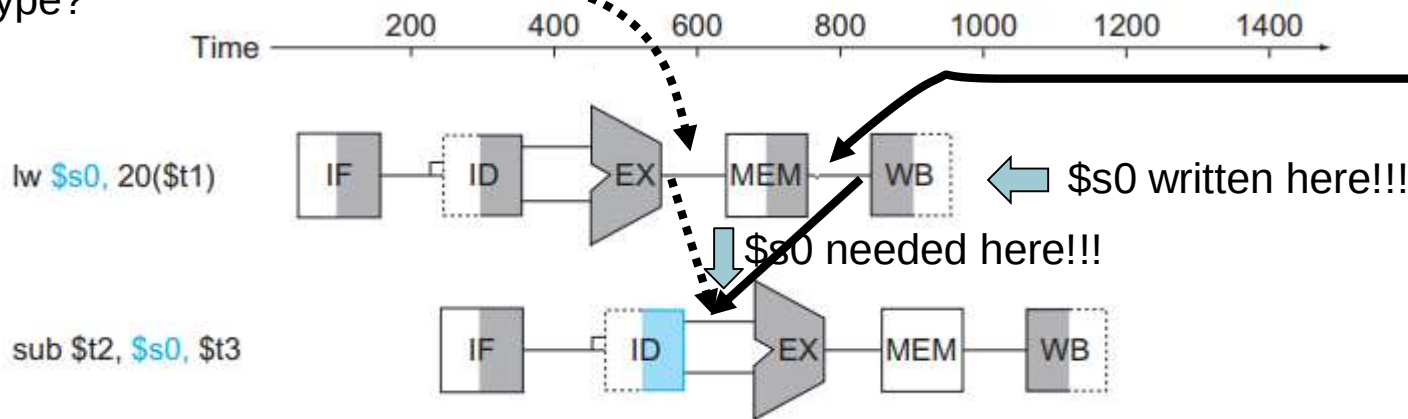
# Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!

Can we solve by FORWARDING?

When is the data Ready?

When WAS the data Ready for R-type?

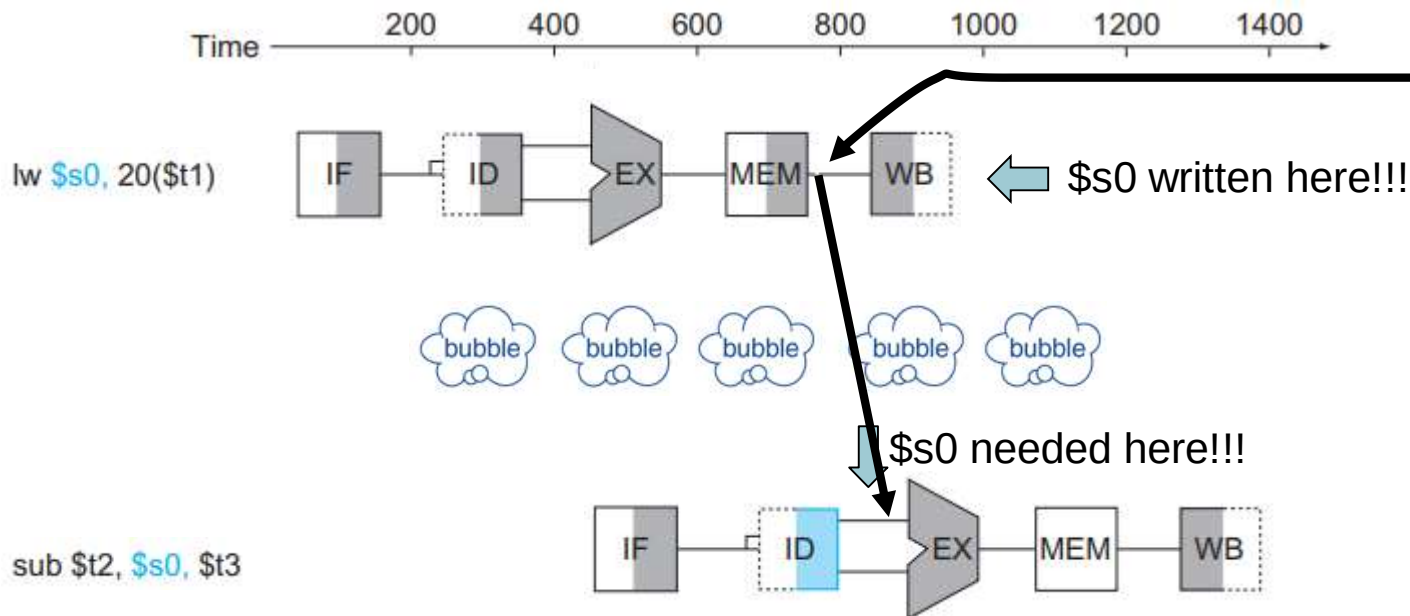


# Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!

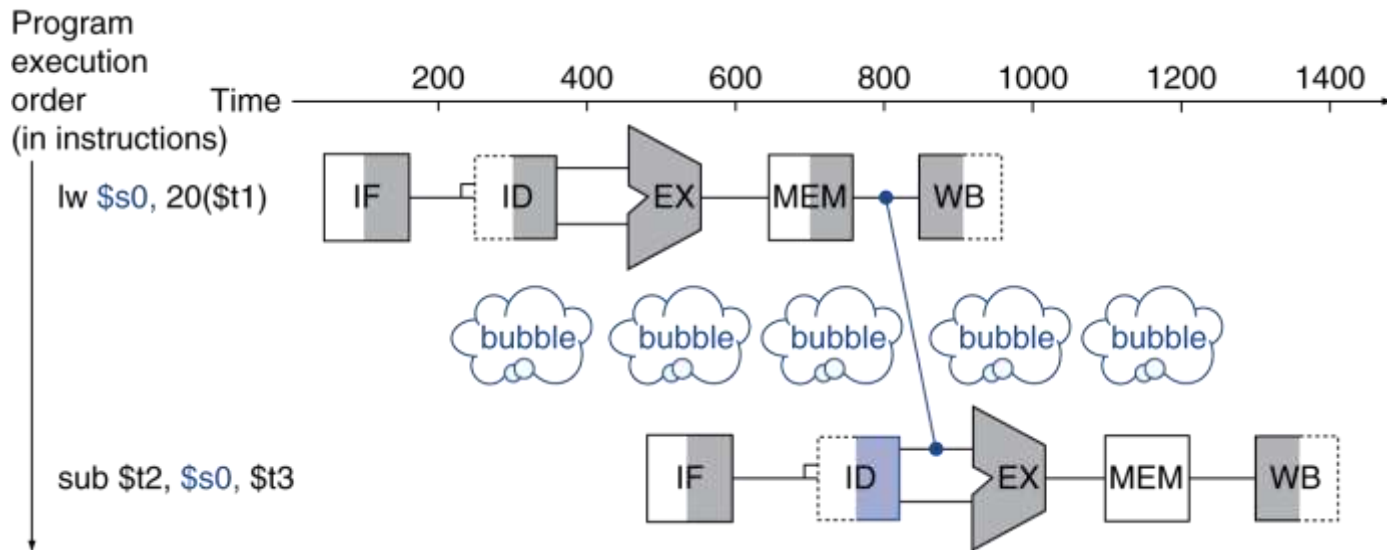
Can we solve by FORWARDING?

When is the data Ready?



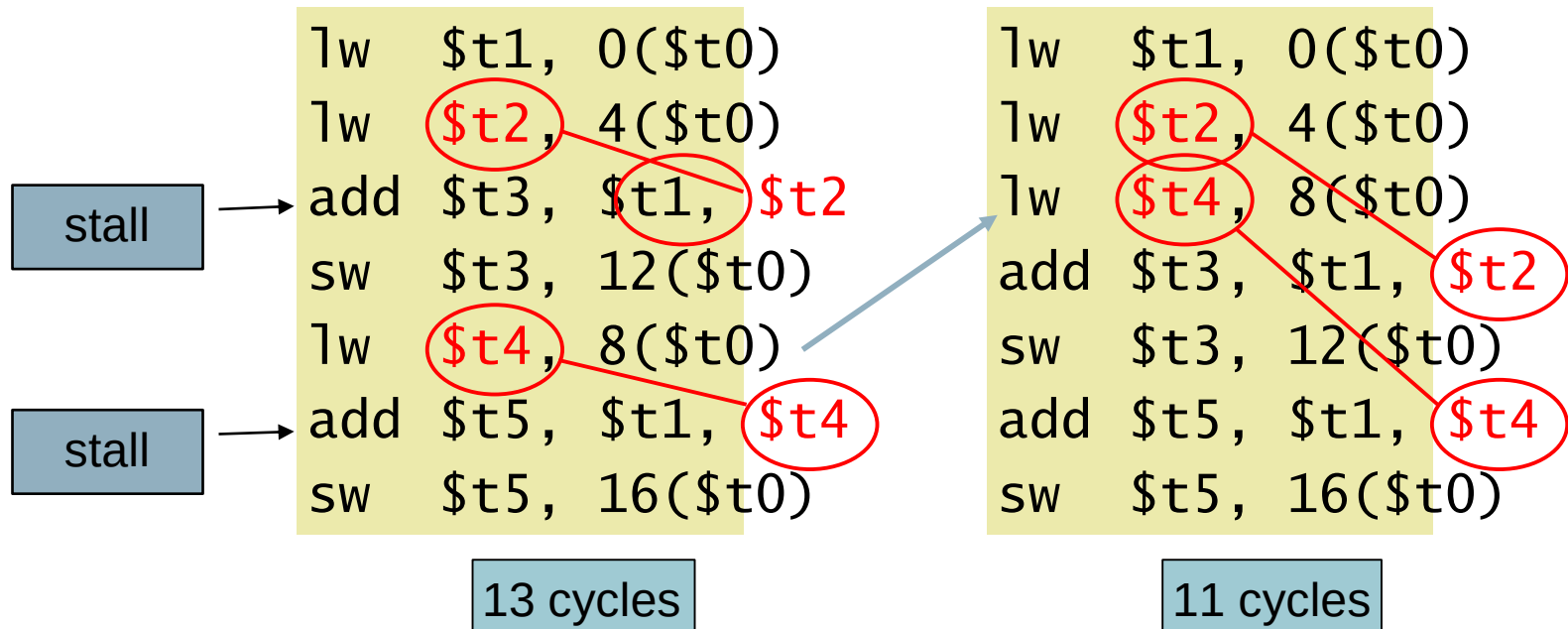
# Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!

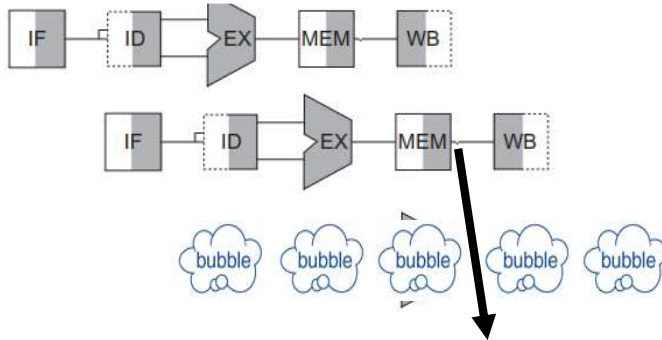


# Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for  $A = B + E$ ;  $C = B + F$ ;  
A(12), B(0), E(4), C(16), F(8)



| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|---|---|---|---|---|---|---|---|---|----|----|----|----|
|---|---|---|---|---|---|---|---|---|----|----|----|----|

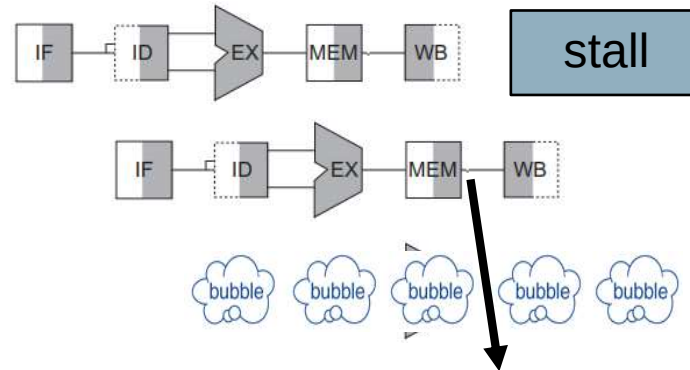


stall

```

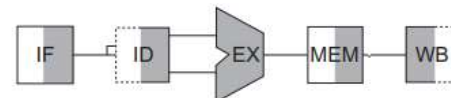
lw    $t1, 0($t0)
lw    $t2, 4($t0)
add   $t3, $t1, $t2
sw    $t3, 12($t0)
lw    $t4, 8($t0)
add   $t5, $t1, $t4
sw    $t5, 16($t0)

```

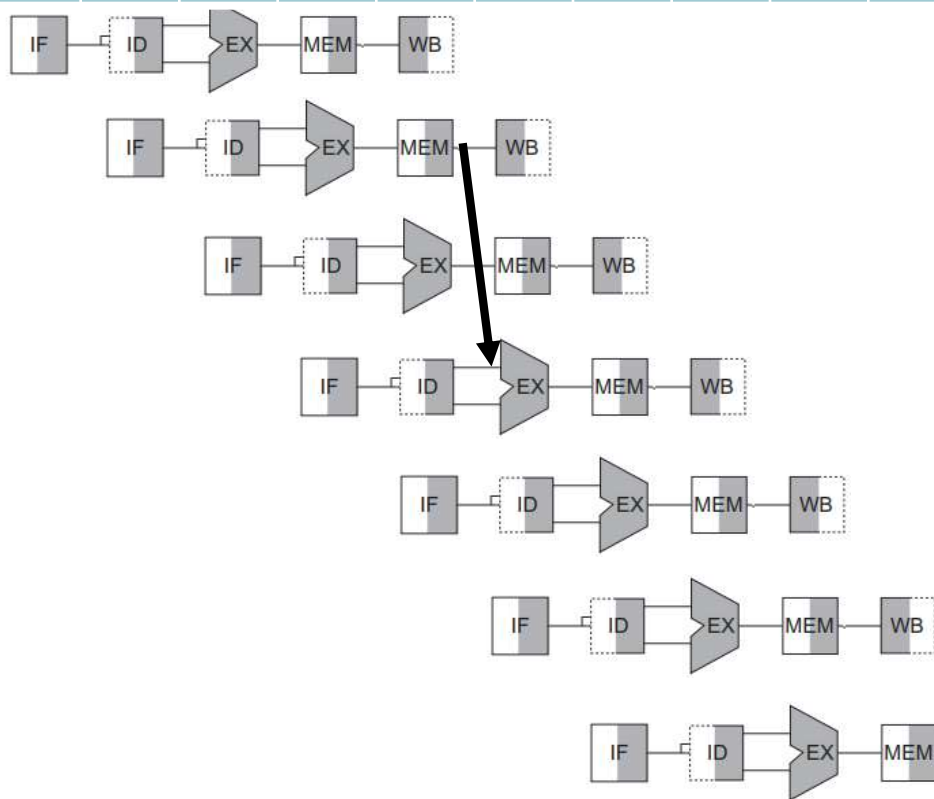


stall

13 cycles



1 2 3 4 5 6 7 8 9 10 11 12 13



```
lw $t1, 0($t0)
lw $t2, 4($t0)
lw $t4, 8($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

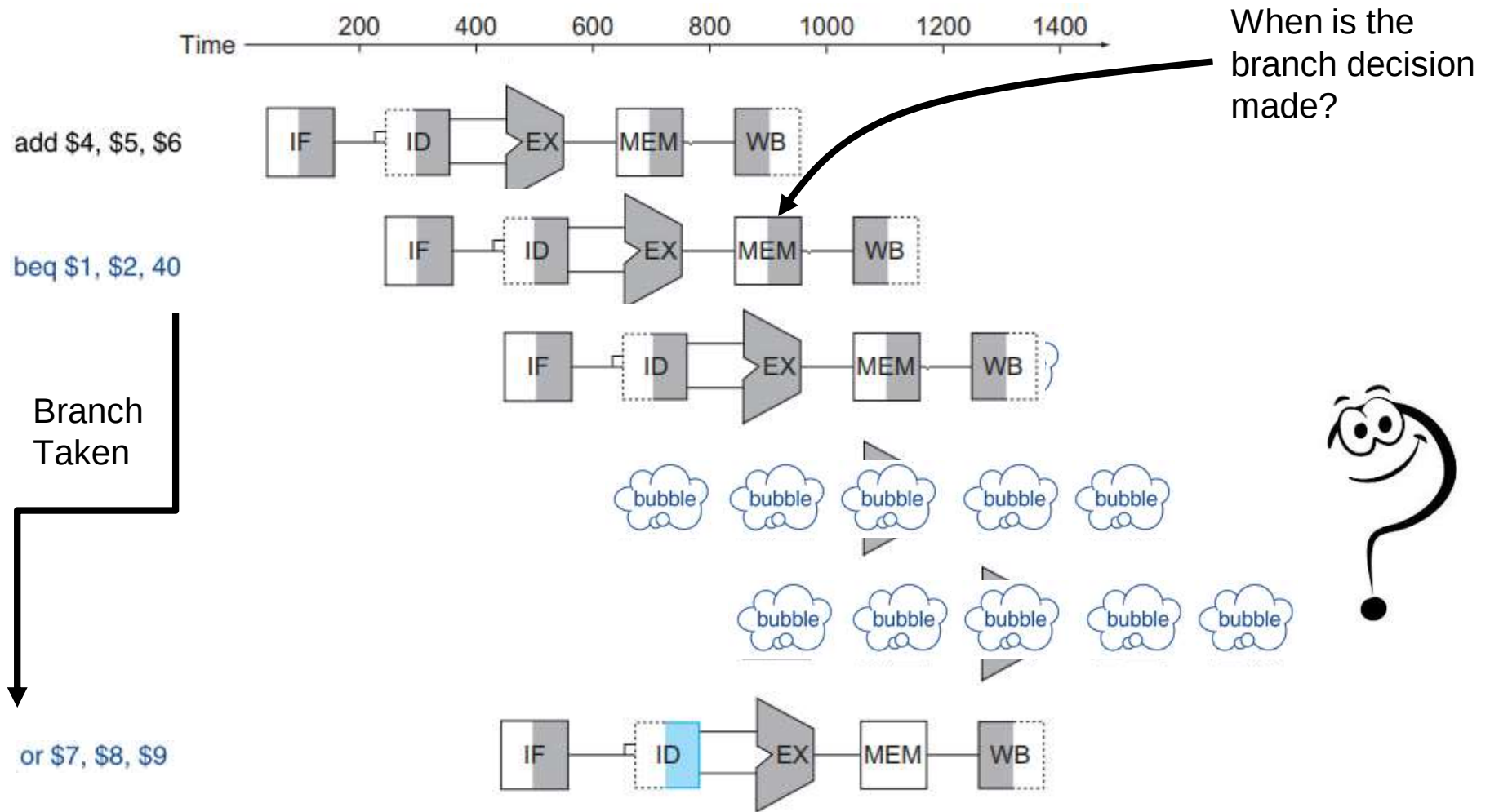
```
lw $t1, 0($t0)
lw $t2, 4($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
lw $t4, 8($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

11 cycles

# Control Hazards

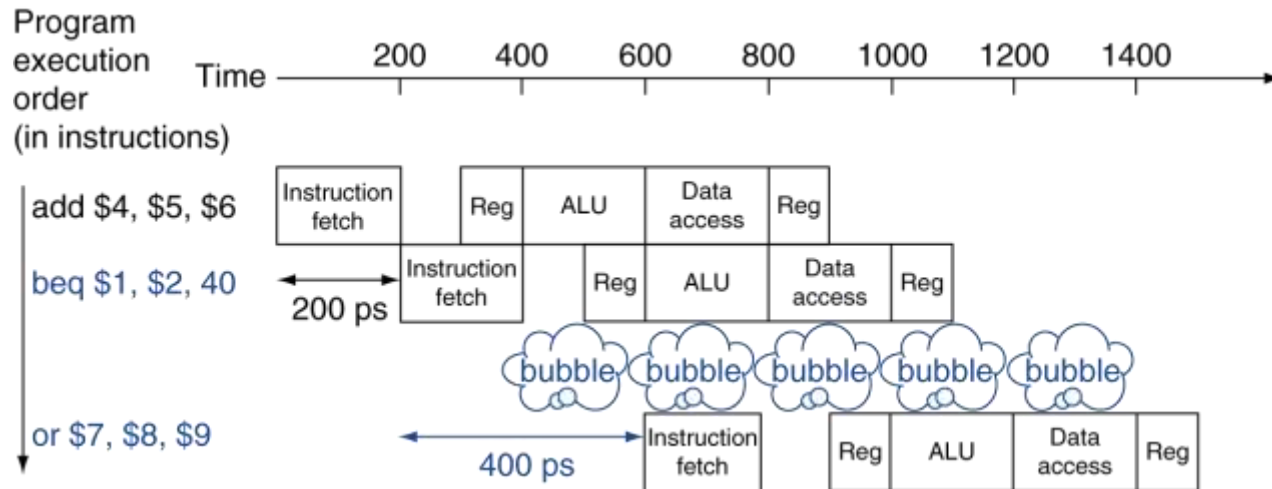
- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can't always fetch correct instruction
    - Still working on ID stage of branch
- In MIPS pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

# Branch Scenario



# Stall on Branch (Improved)

- Assume enough extra hardware to test registers, calculate the branch address, and update the PC during the second stage (ID stage)

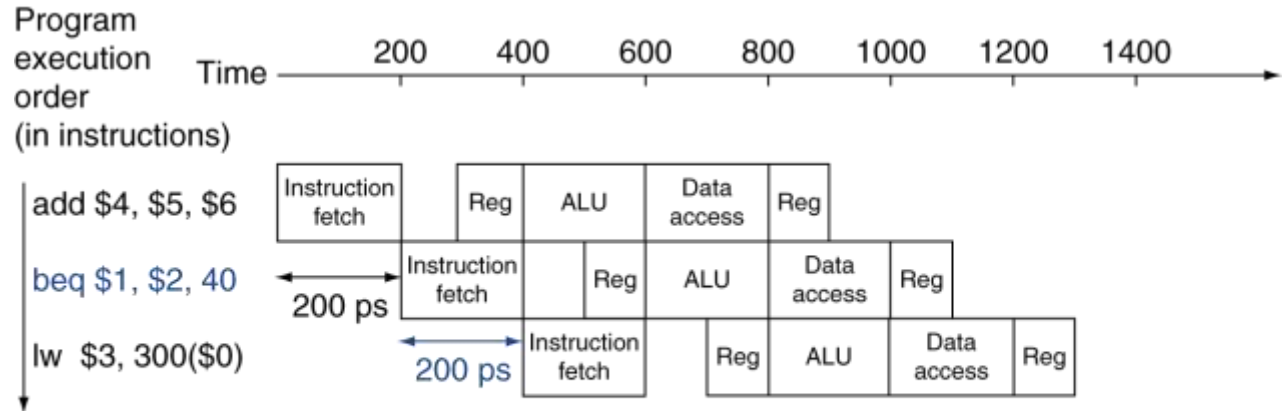


# Branch Prediction

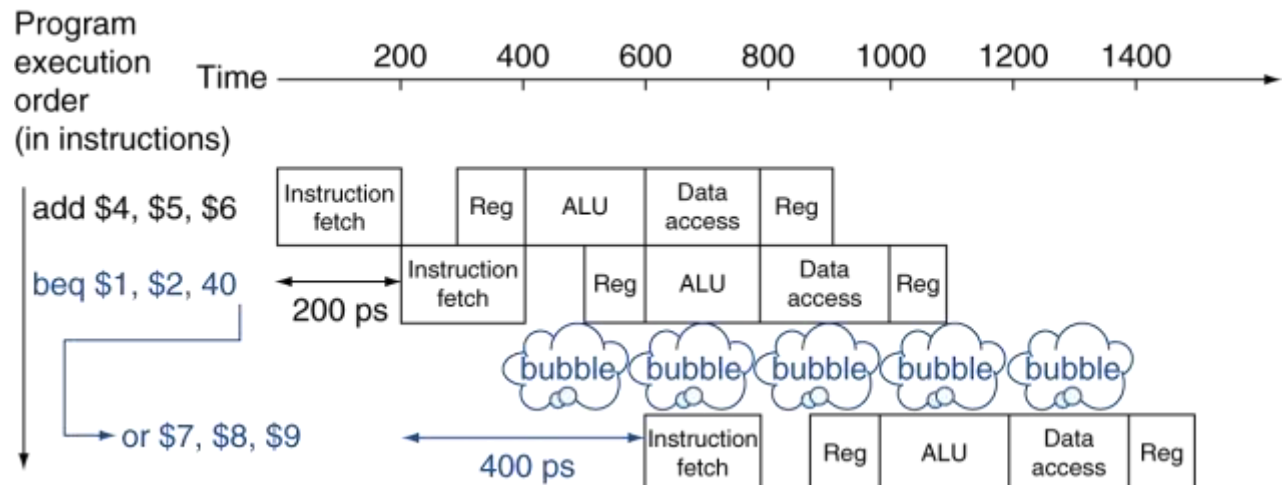
- Longer pipelines can't readily determine branch outcome early
  - Stall penalty becomes unacceptable
- Predict outcome of branch
  - Only stall if prediction is wrong
- In MIPS pipeline
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

# MIPS with Predict Not Taken

Prediction  
correct



Prediction  
incorrect

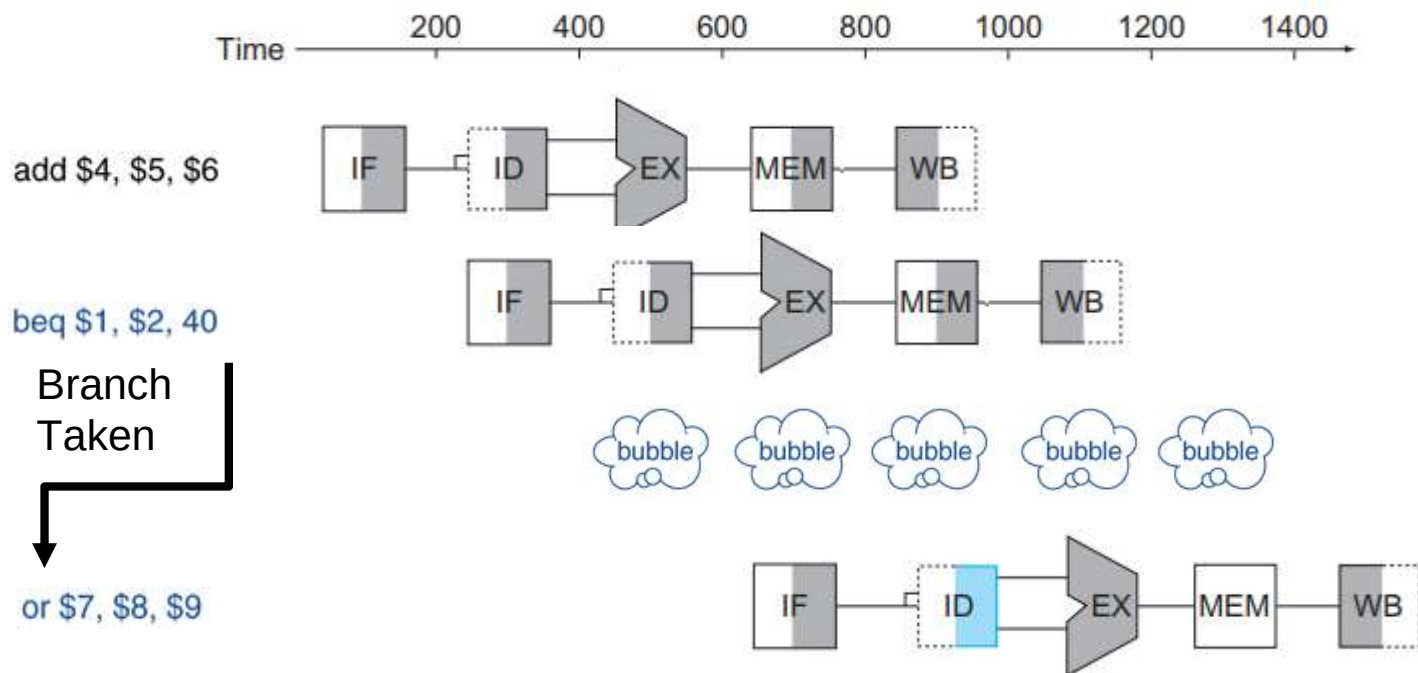


# More-Realistic Branch Prediction

- Static branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not taken
- Dynamic branch prediction
  - Hardware measures actual branch behavior
    - e.g., record recent history of each branch
  - Assume future behavior will continue the trend
    - When wrong, stall while re-fetching, and update history

# Delayed Branch

- Always execute the next sequential instruction
  - i.e., taken branch is always delayed
  - The delay is hidden by rearrangement



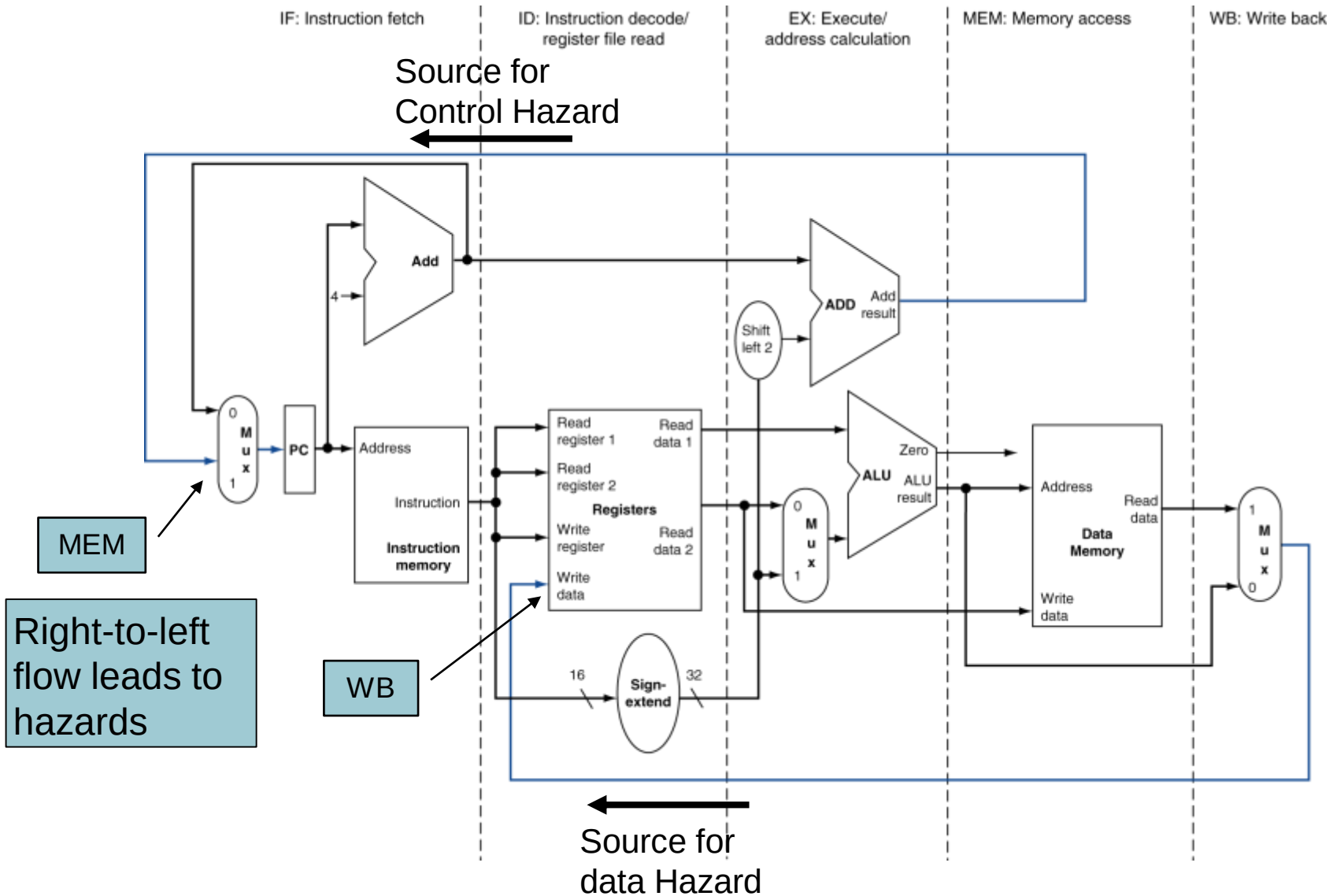
**MIPS Solution to Control Hazard**

# Pipeline Summary

## The BIG Picture

- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards
  - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

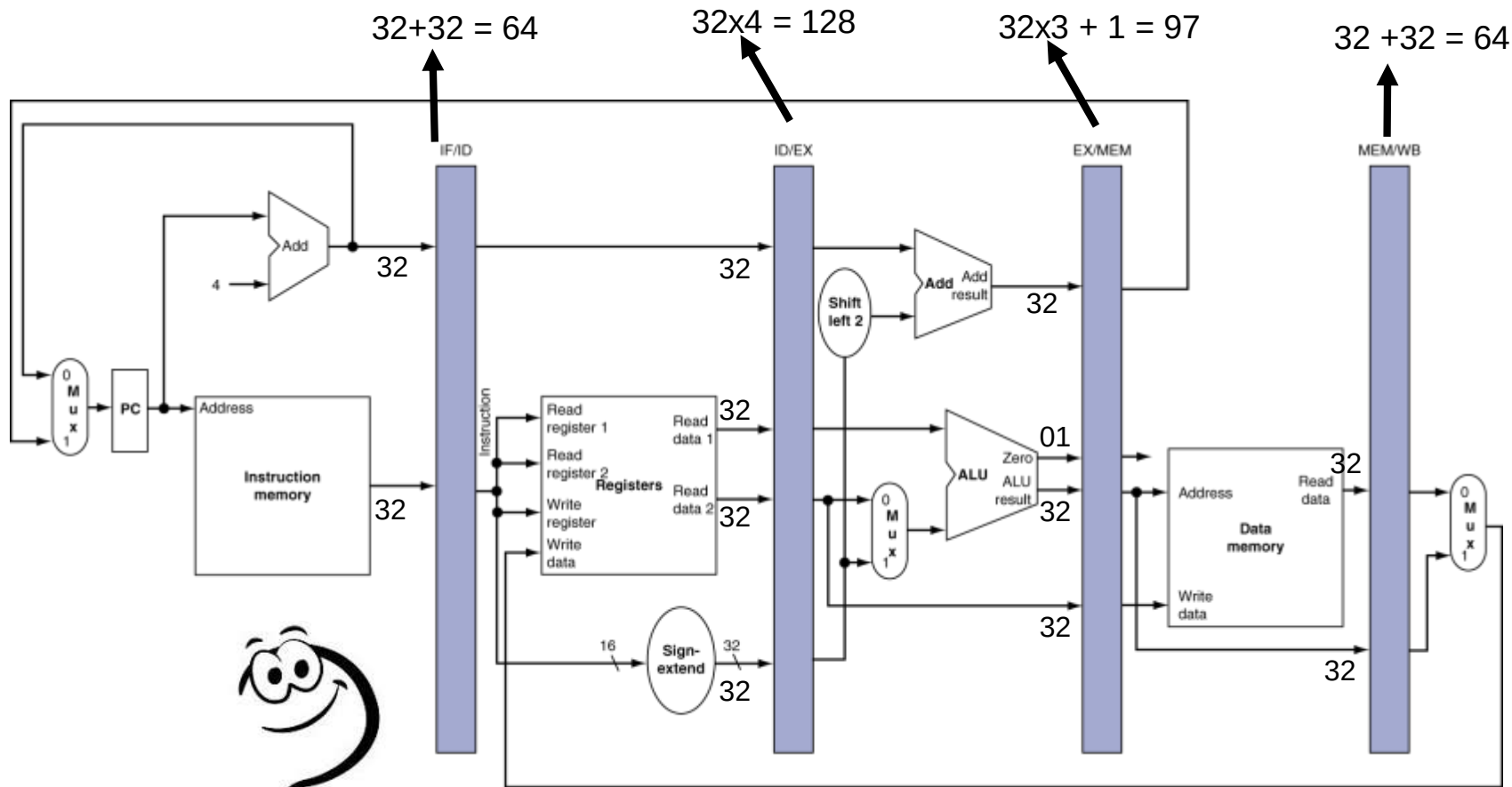
# MIPS Pipelined Datapath



# Pipeline registers

We may need to update the sizes as we proceed!!!

- Need registers between stages
  - To hold information produced in previous cycle

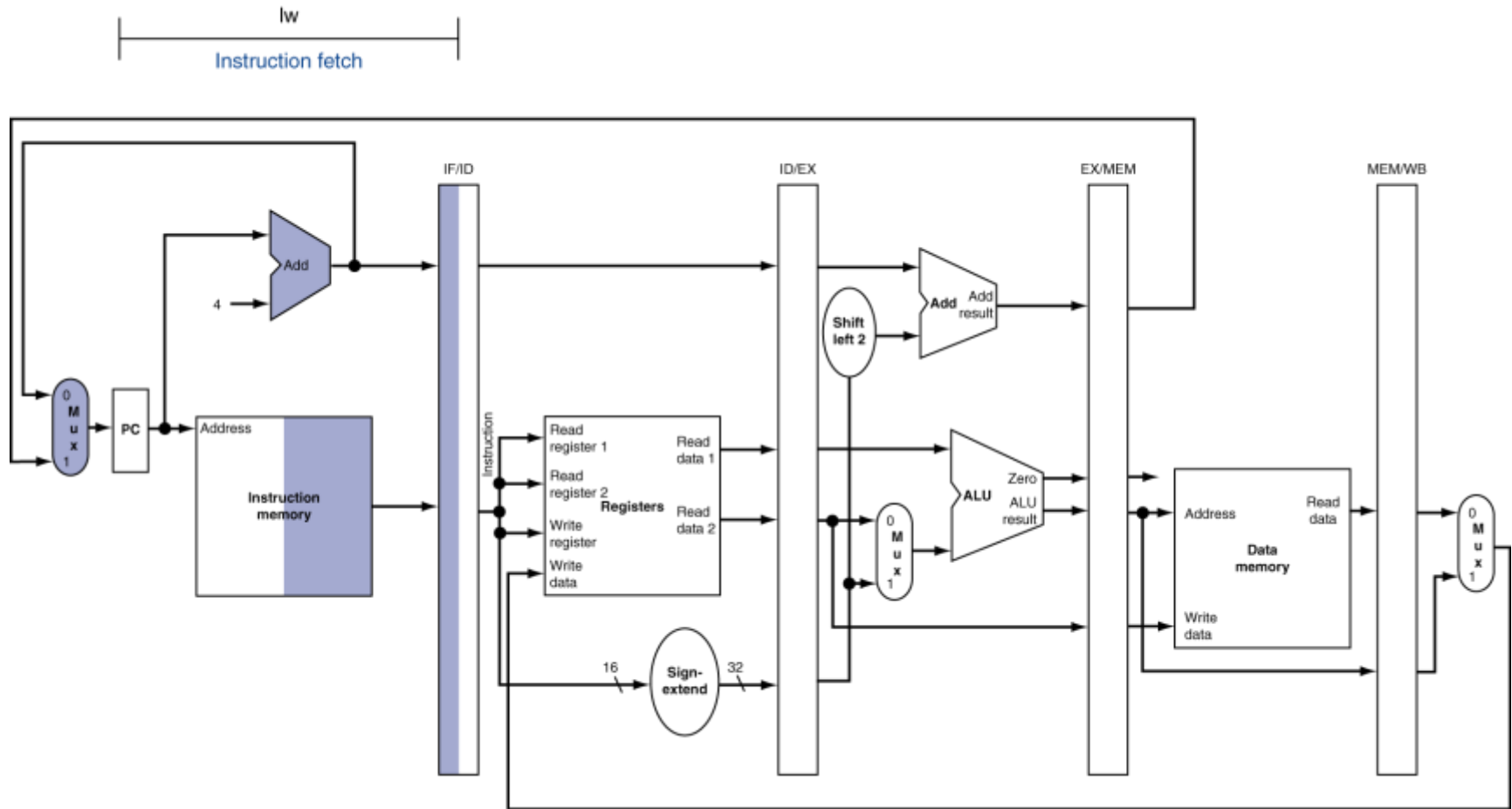


What is the size of the pipeline registers?

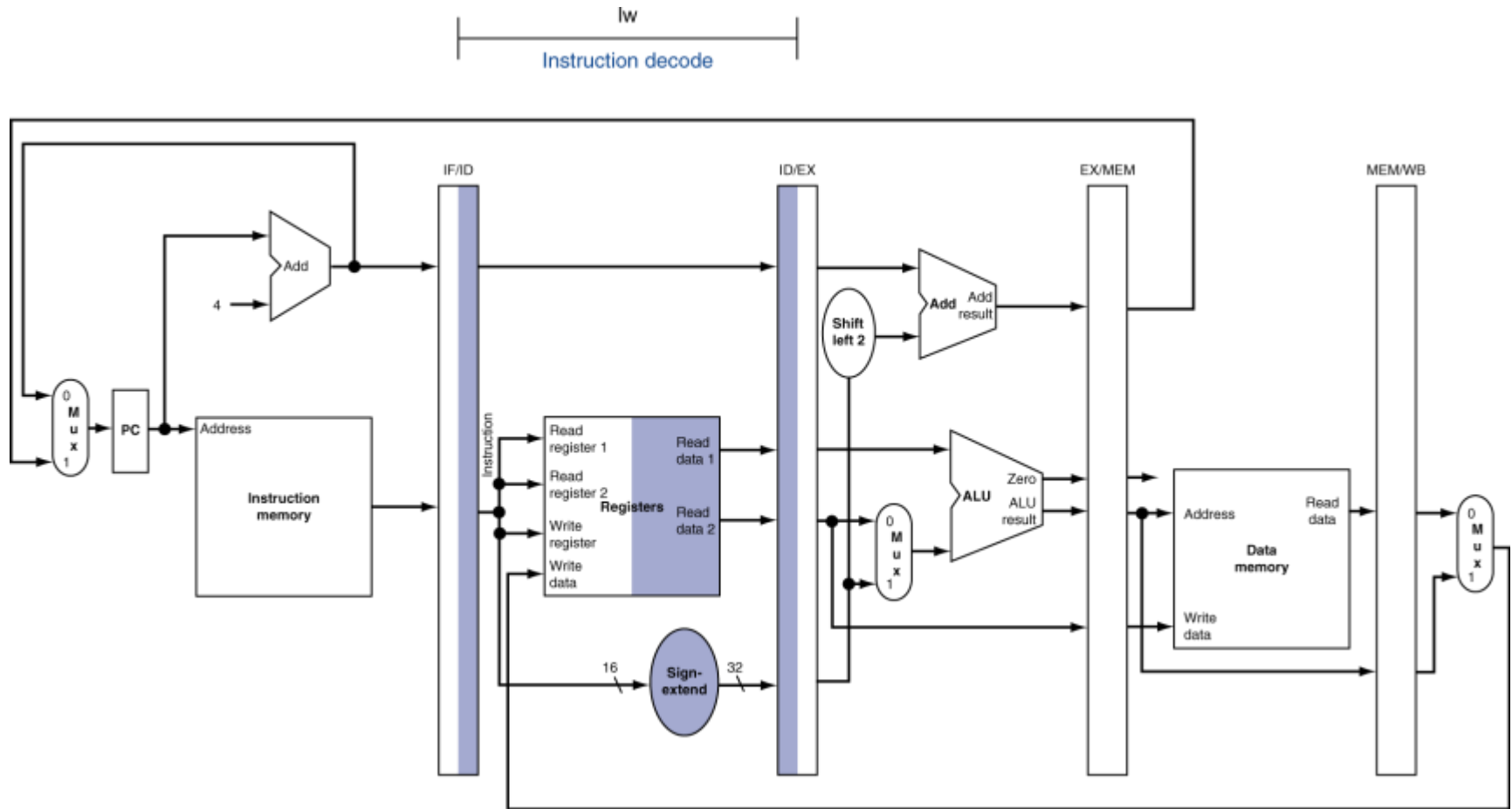
# Pipeline Operation

- Cycle-by-cycle flow of instructions through the pipelined datapath
  - “Single-clock-cycle” pipeline diagram
    - Shows pipeline usage in a single cycle
    - Highlight resources used
  - c.f. “multi-clock-cycle” diagram
    - Graph of operation over time
- We’ll look at “single-clock-cycle” diagrams for load & store

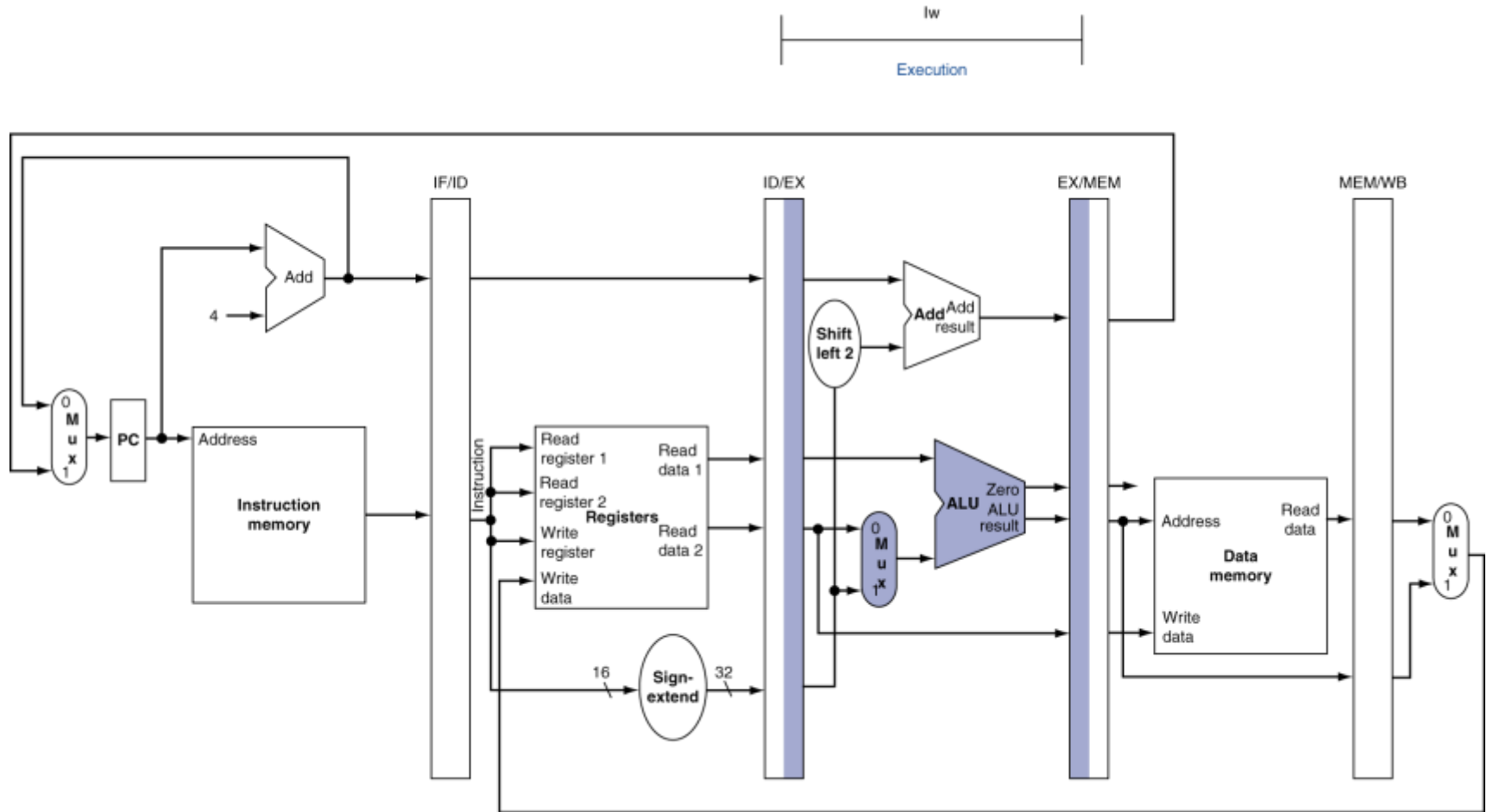
# IF for Load



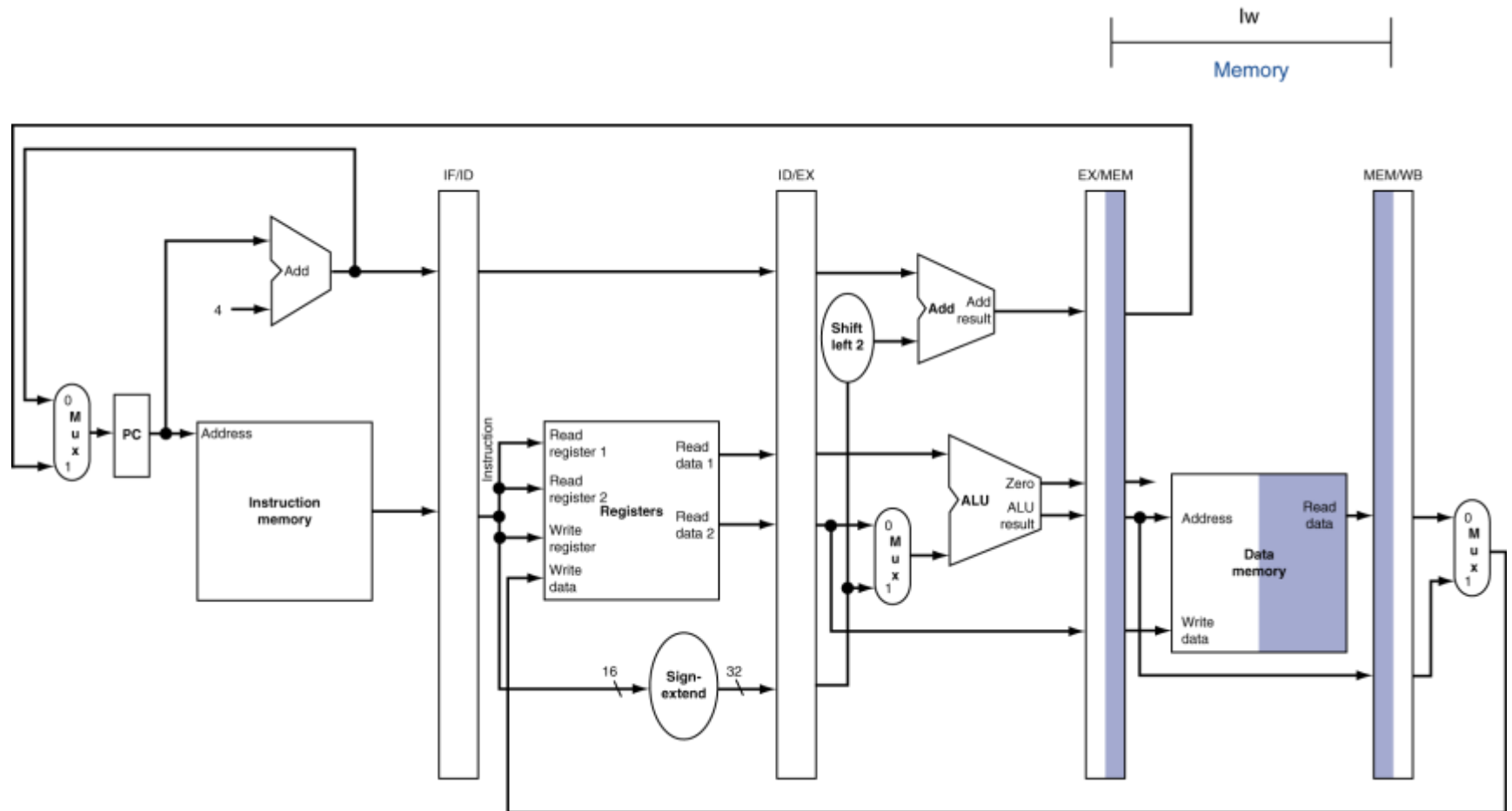
# ID for Load



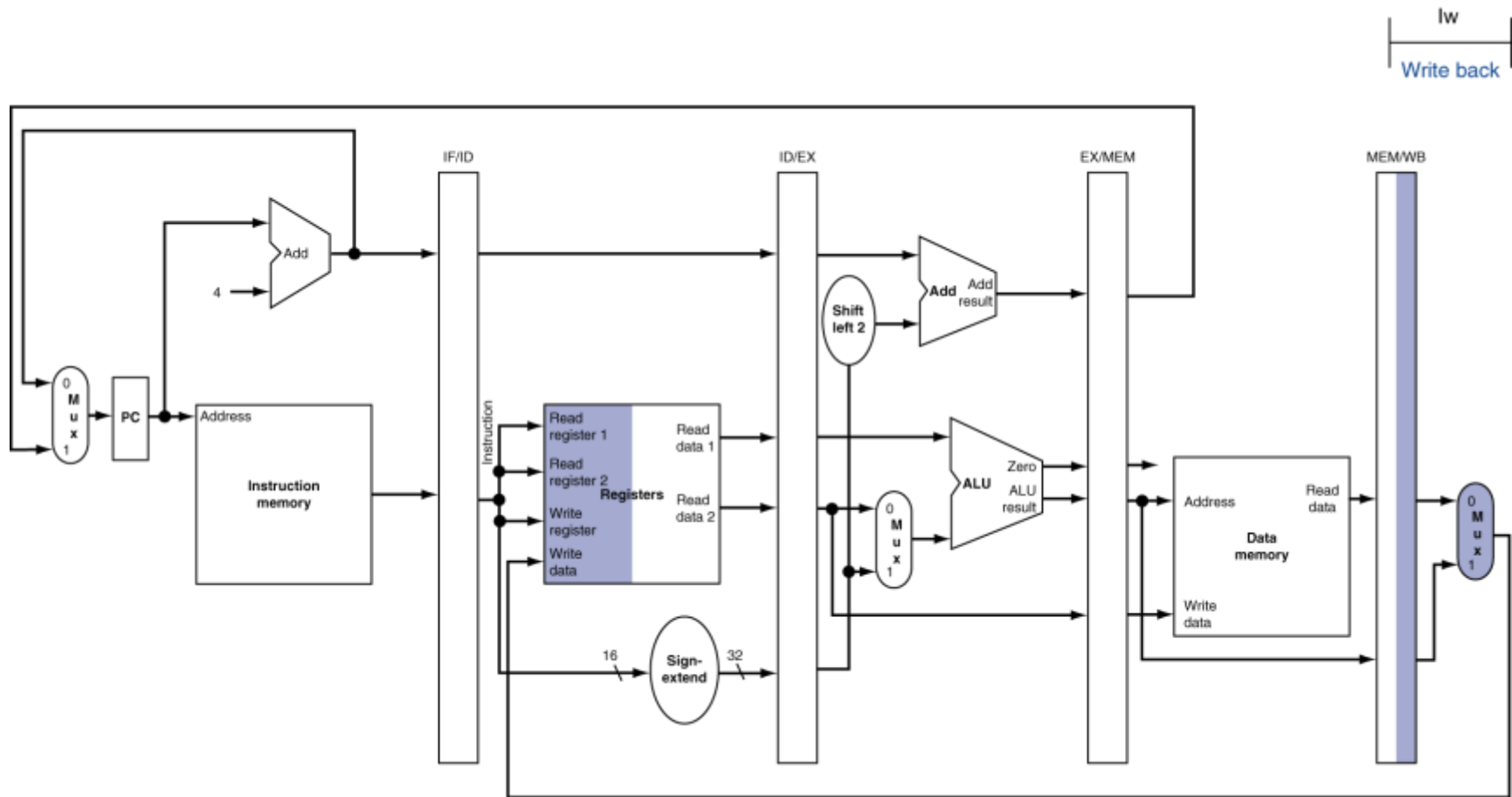
# EX for Load



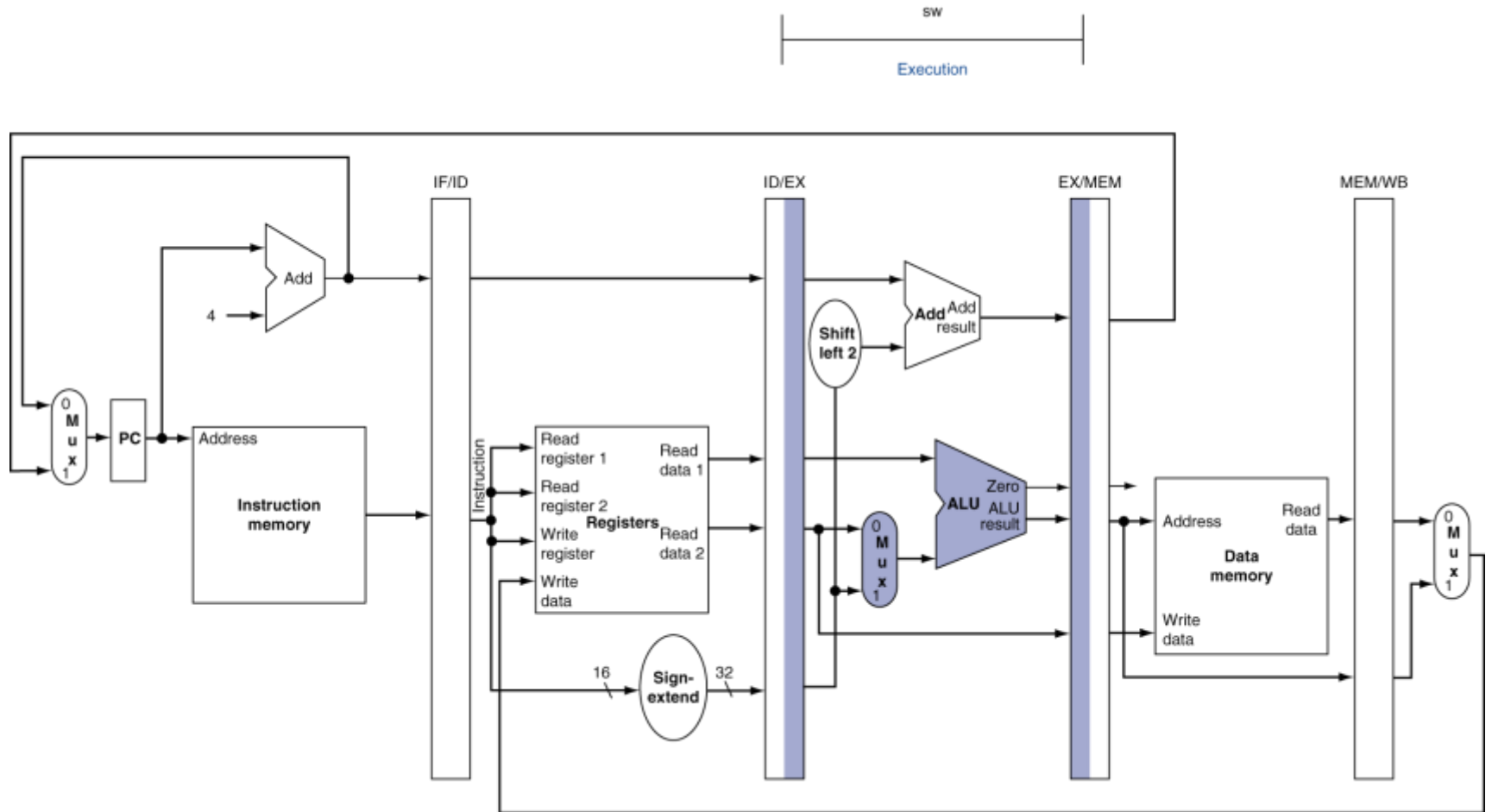
# MEM for Load



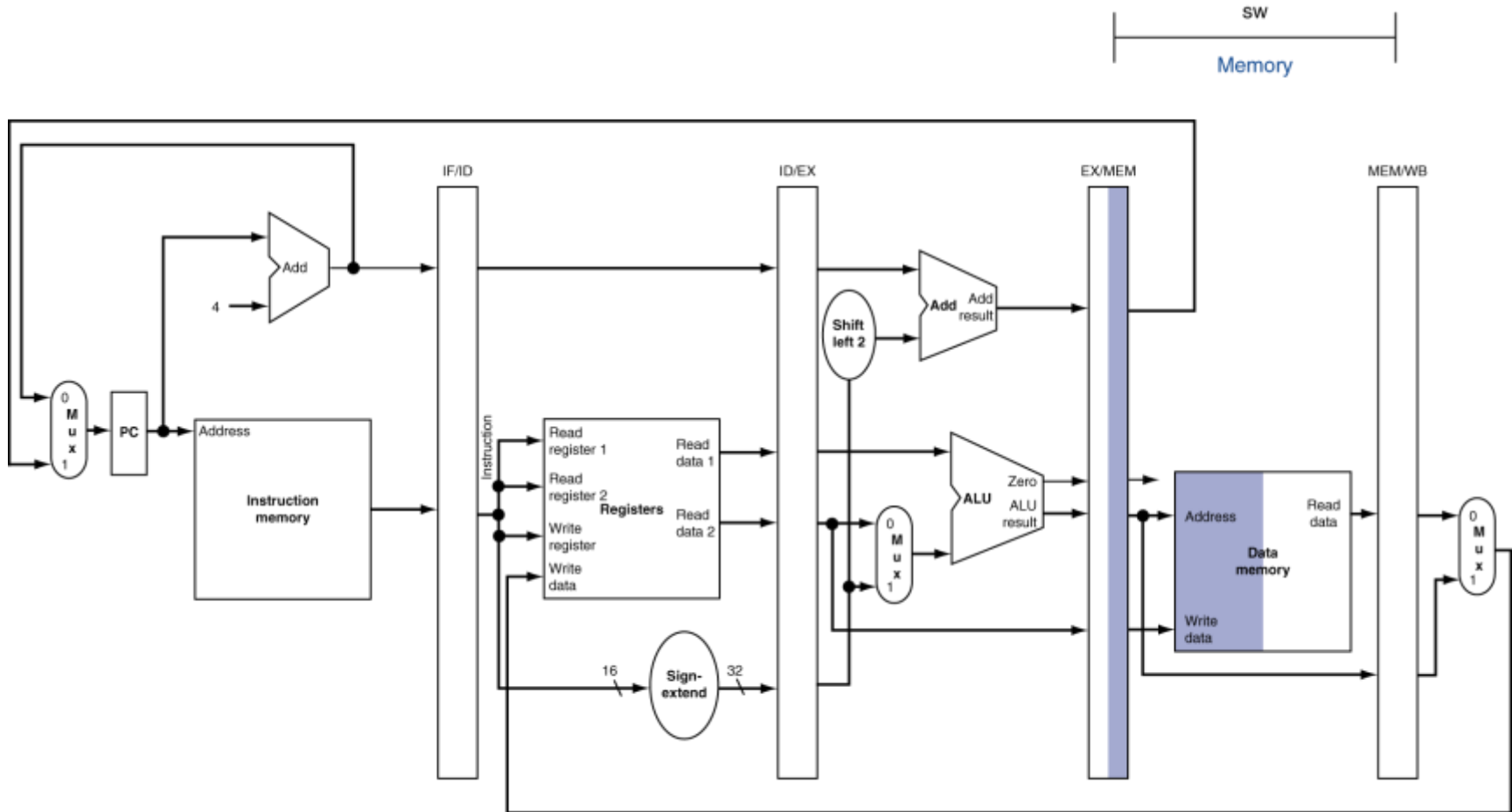
# WB for Load



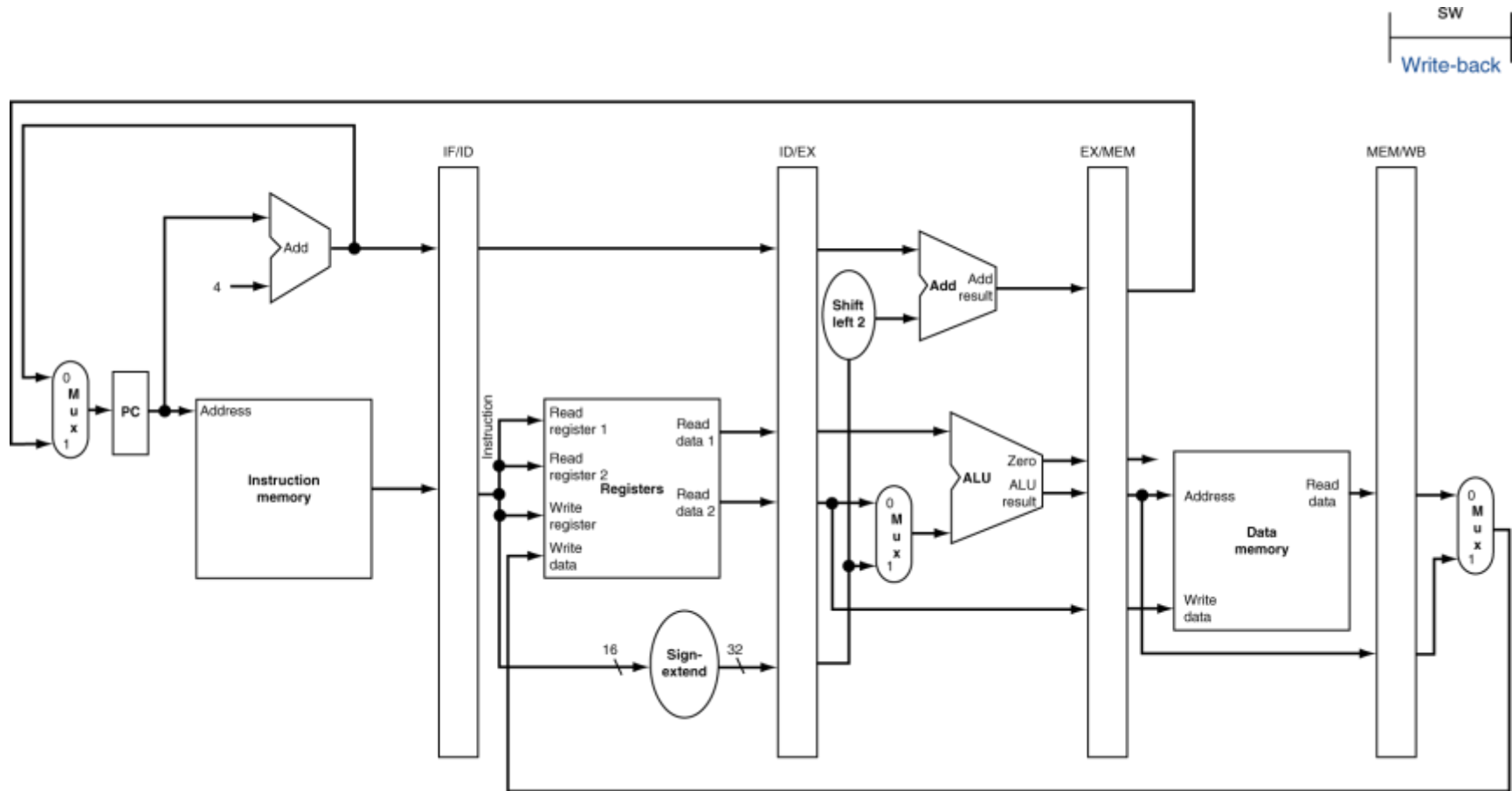
# EX for Store



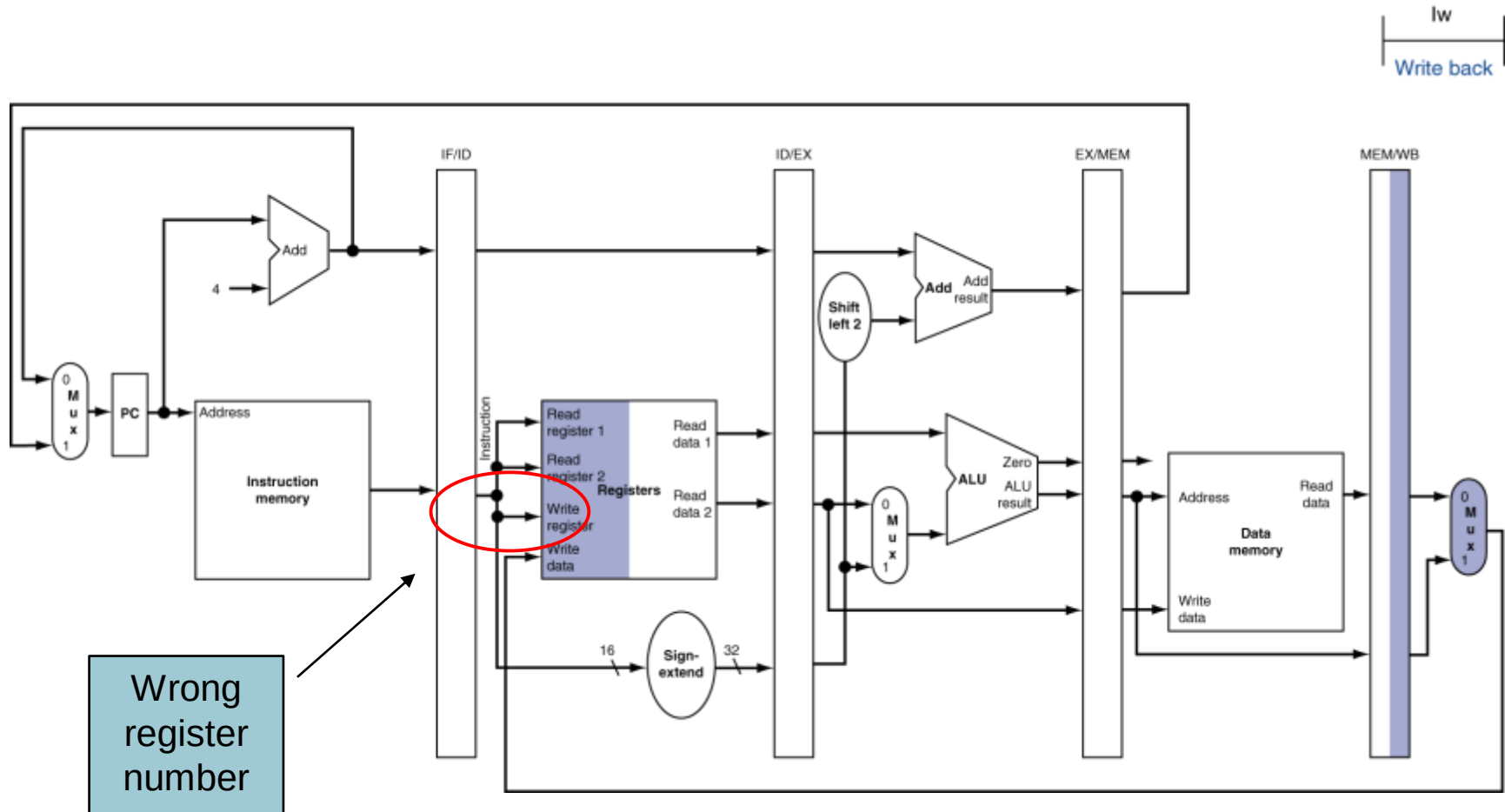
# MEM for Store



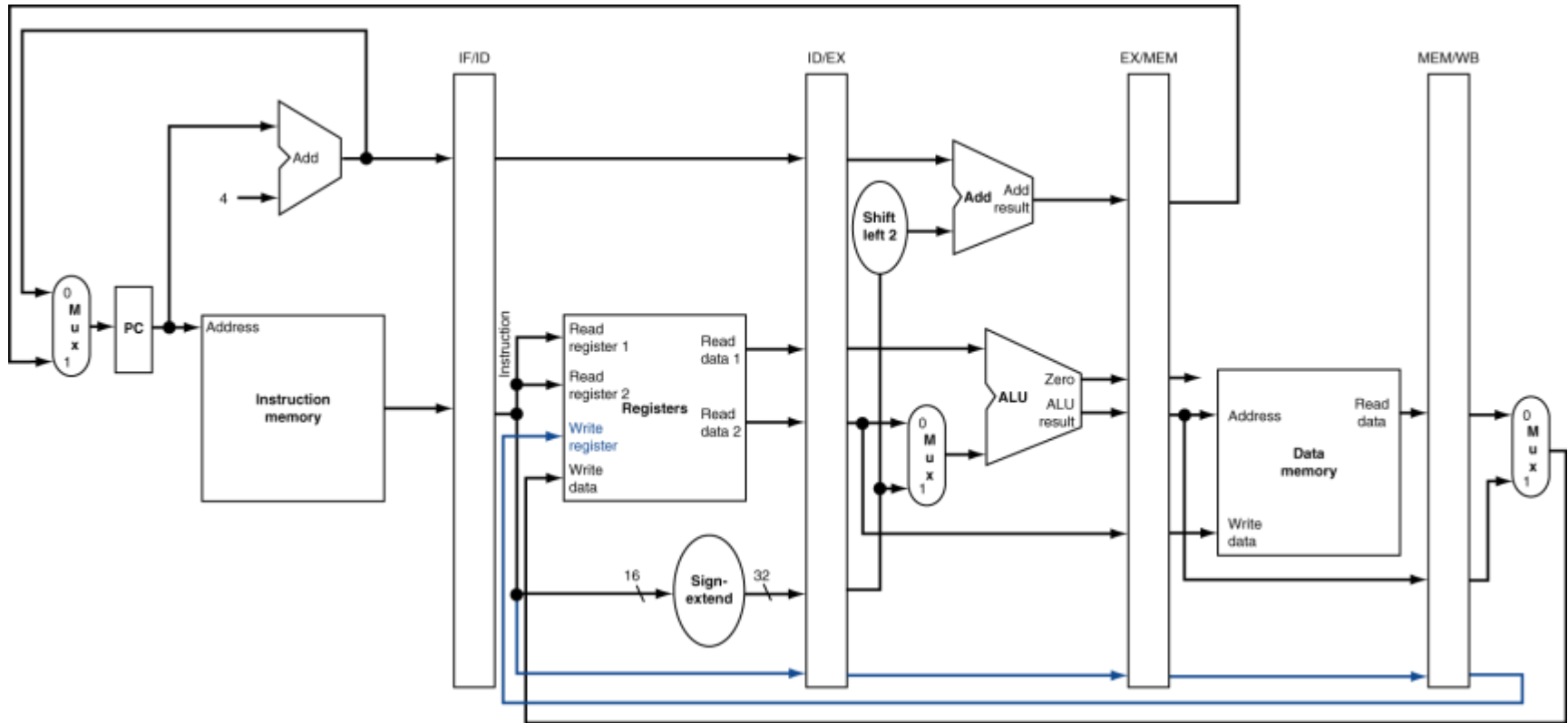
# WB for Store



# WB for Load Revisited

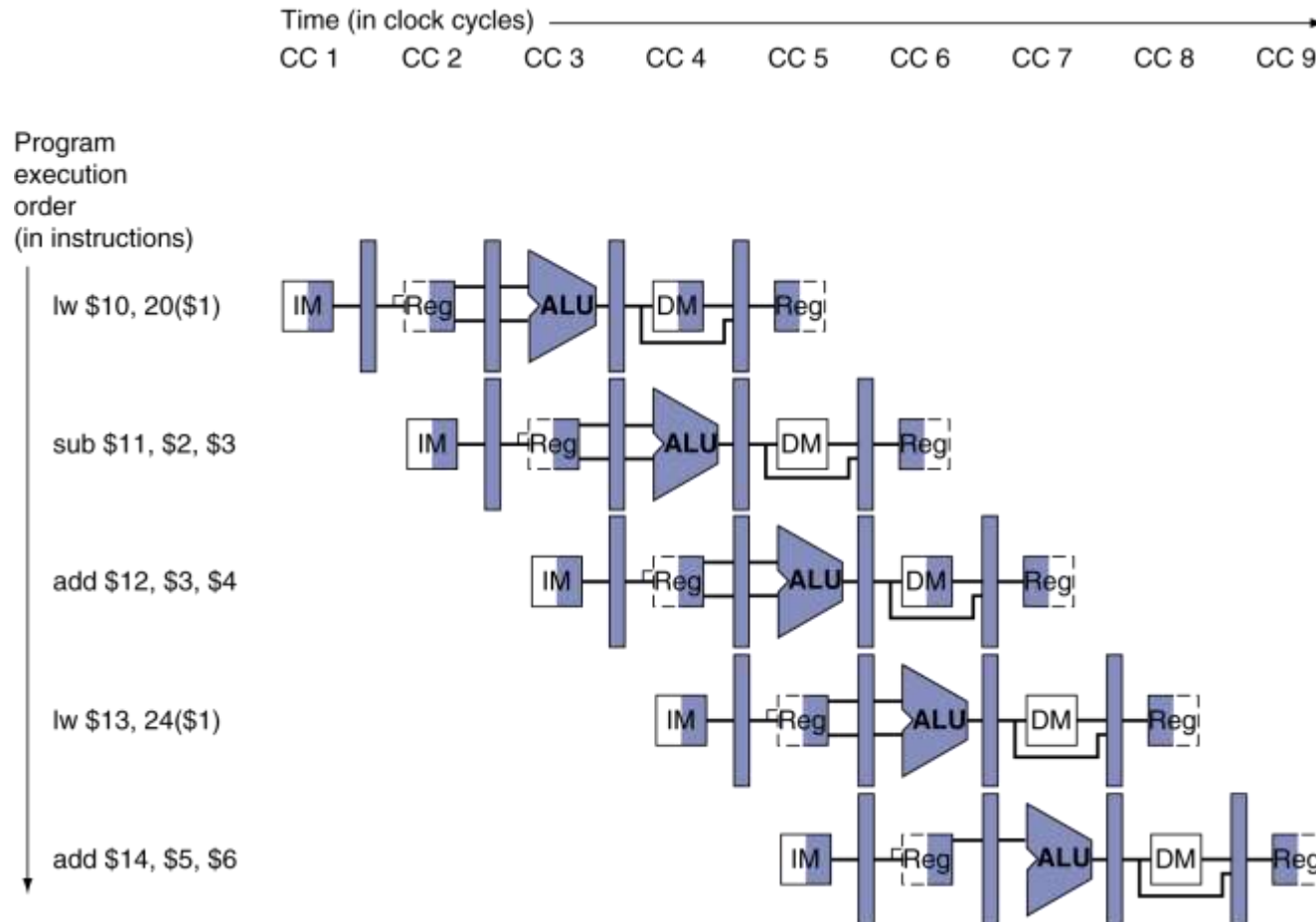


# Corrected Datapath for Load



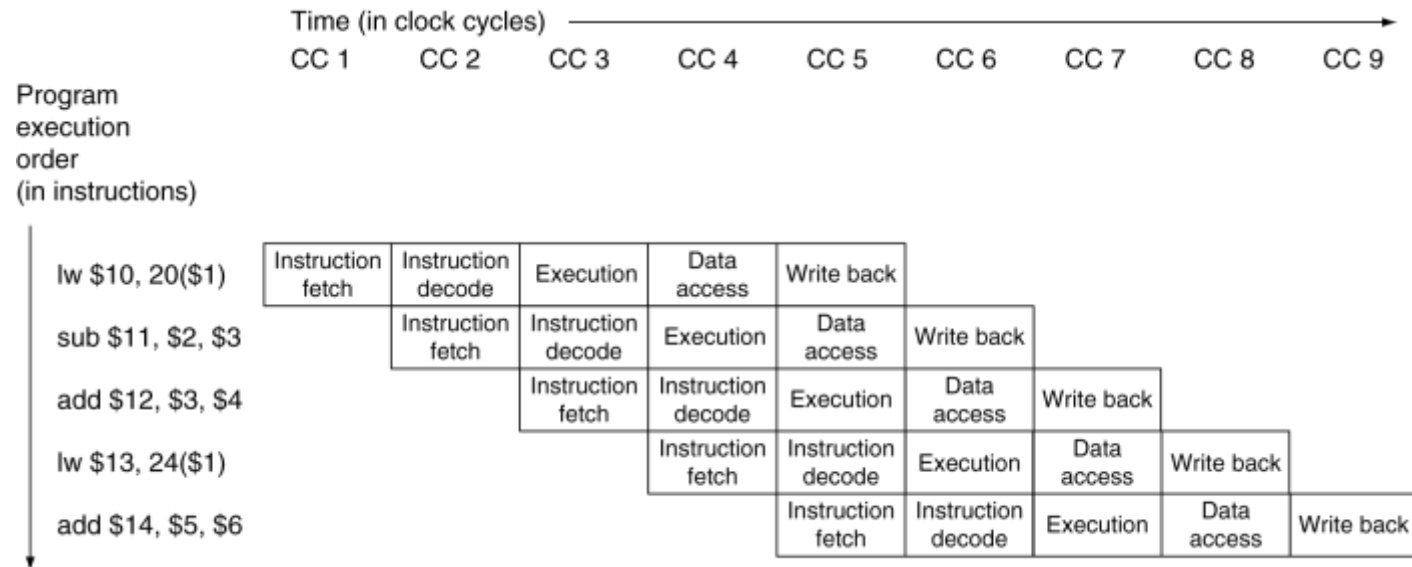
# Multi-Cycle Pipeline Diagram

- Form showing resource usage



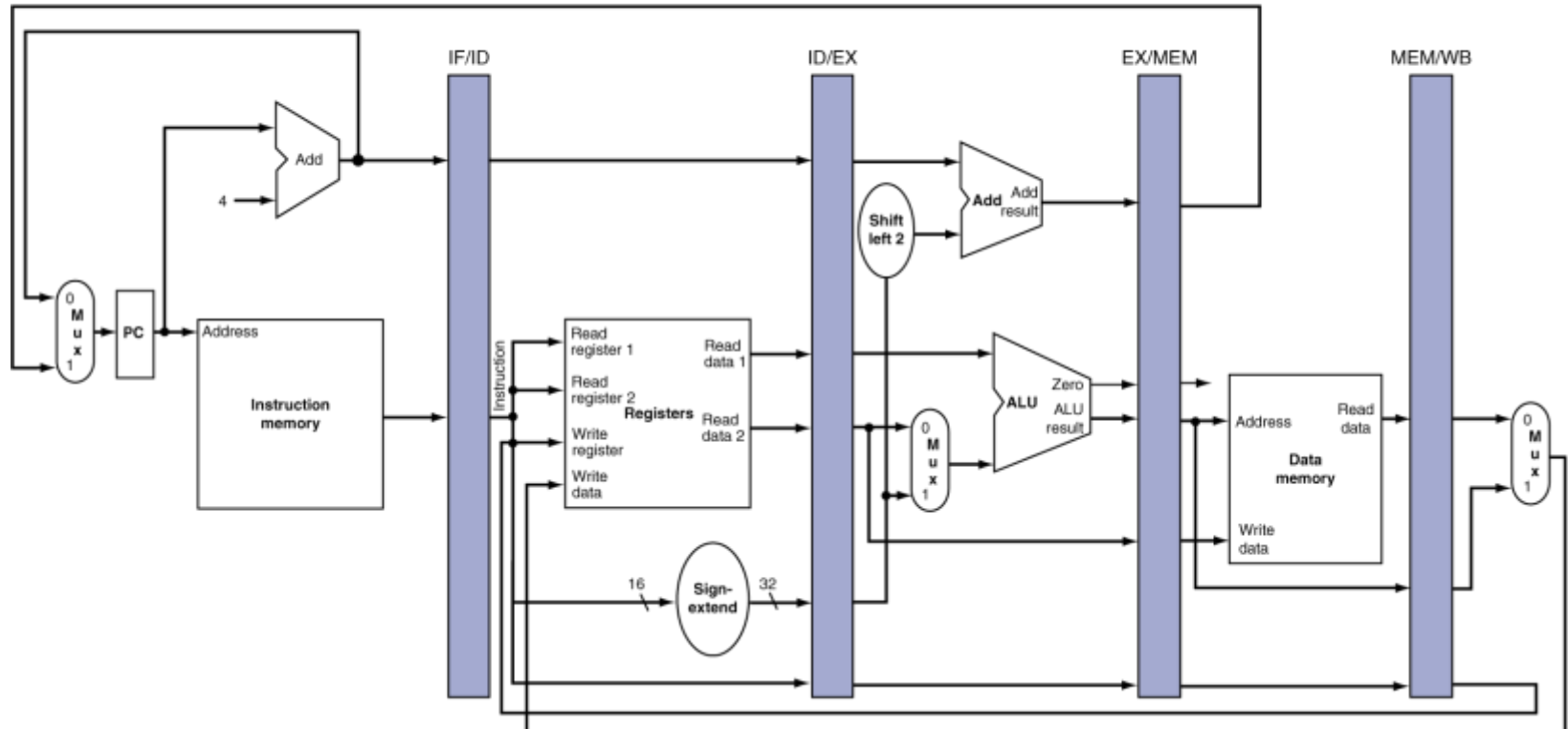
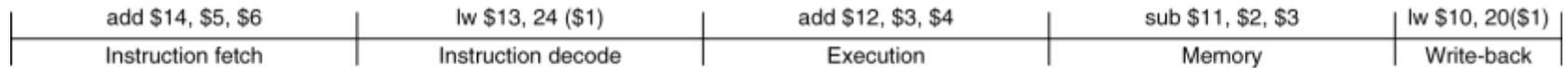
# Multi-Cycle Pipeline Diagram

- Traditional form



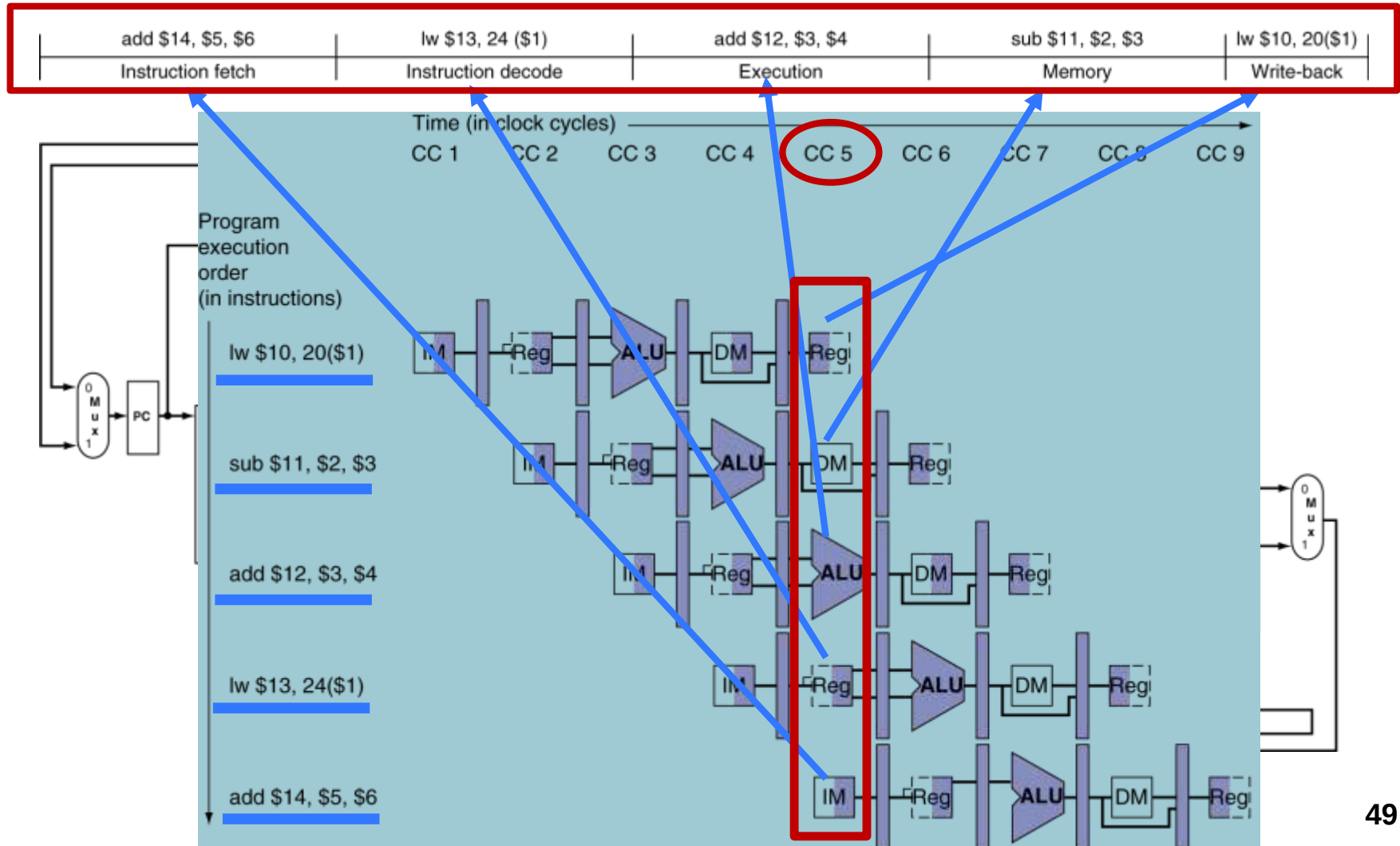
# Single-Cycle Pipeline Diagram

- State of pipeline in a given cycle

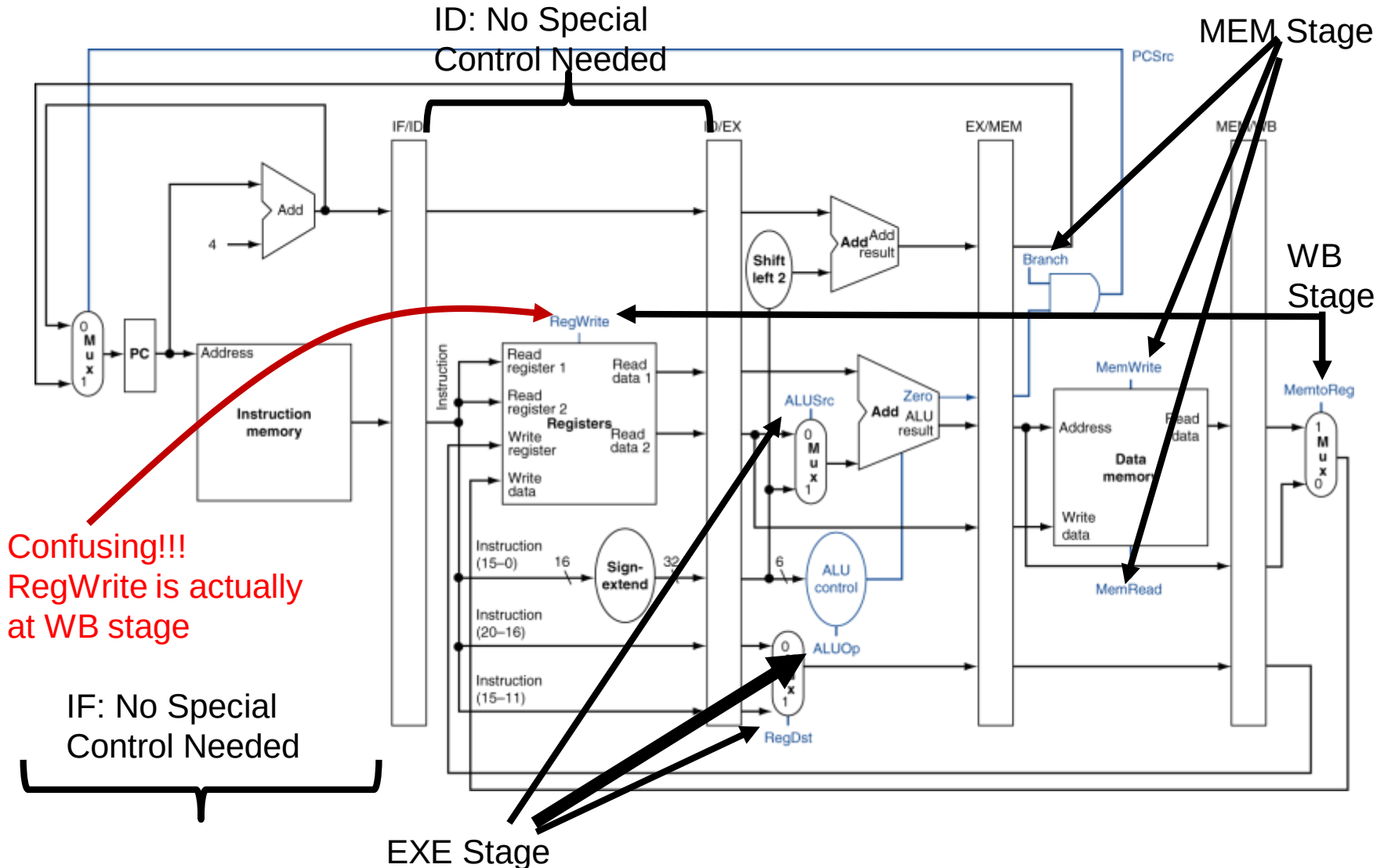


# Single-Cycle Pipeline Diagram

- State of pipeline in a given cycle

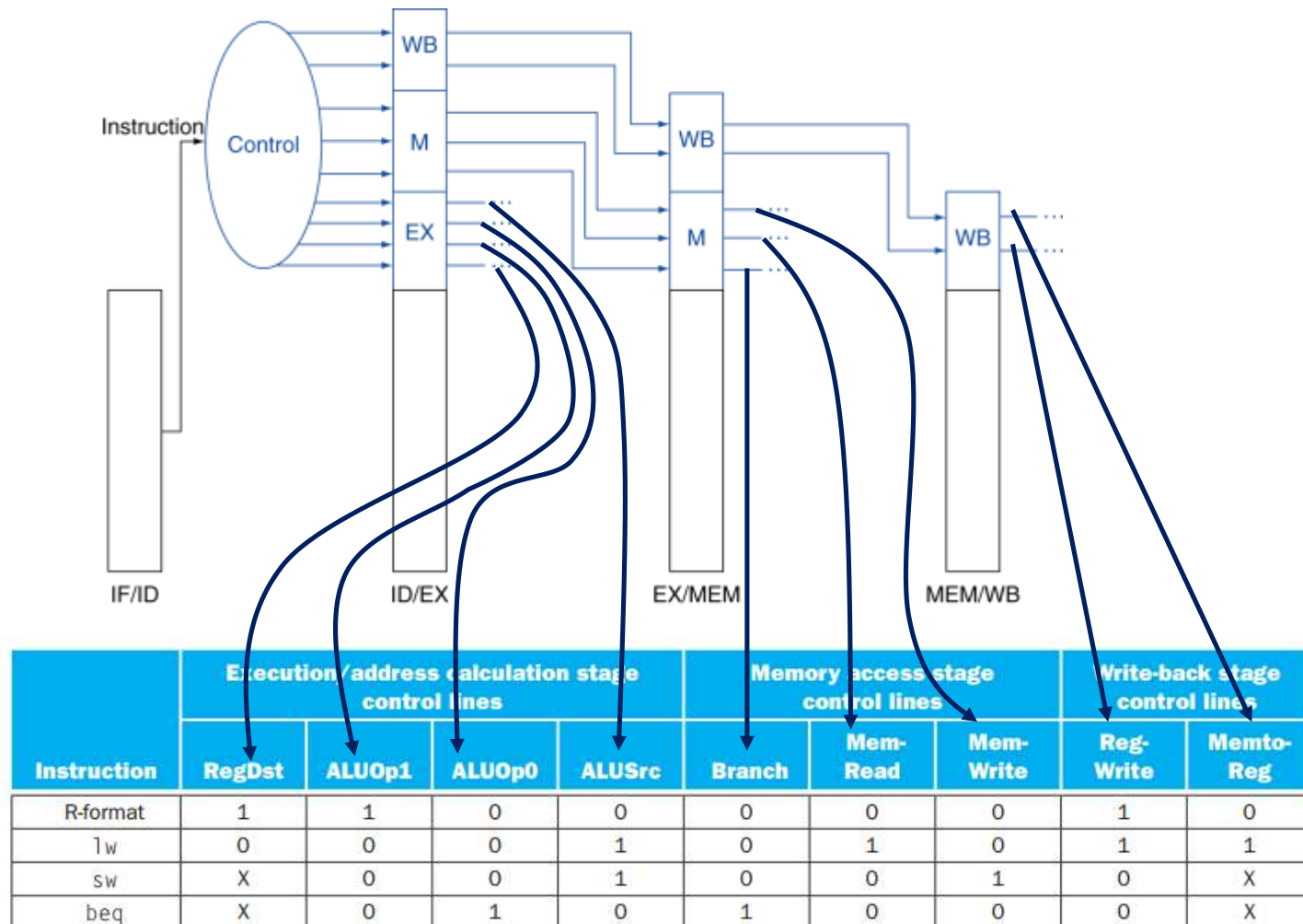


# Pipelined Control (Simplified)

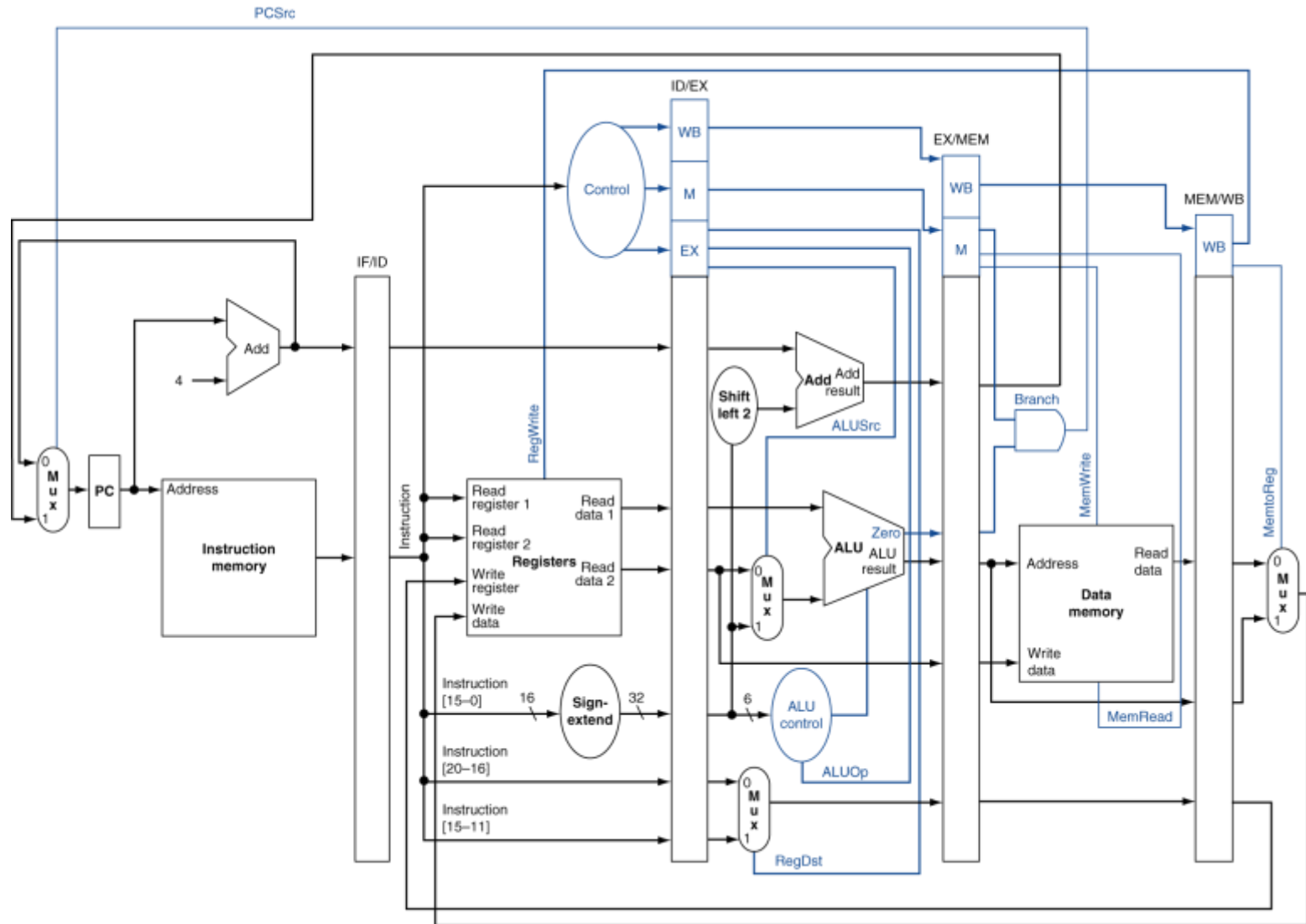


# Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation



# Pipelined Control



# Data Hazards in ALU Instructions

- Consider this sequence:

sub \$2, \$1, \$3

and \$12, \$2, \$5

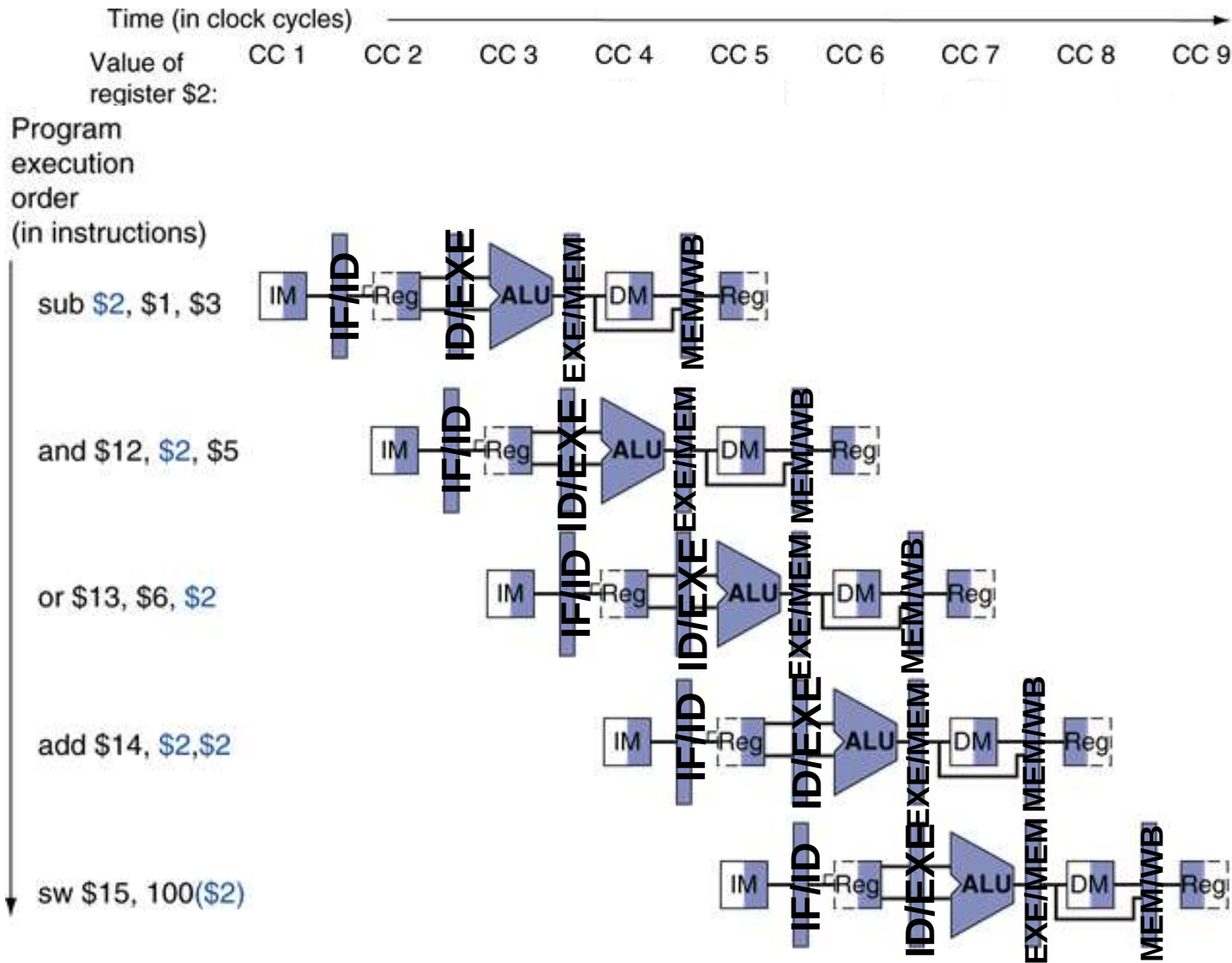
or \$13, \$6, \$2

add \$14, \$2, \$2

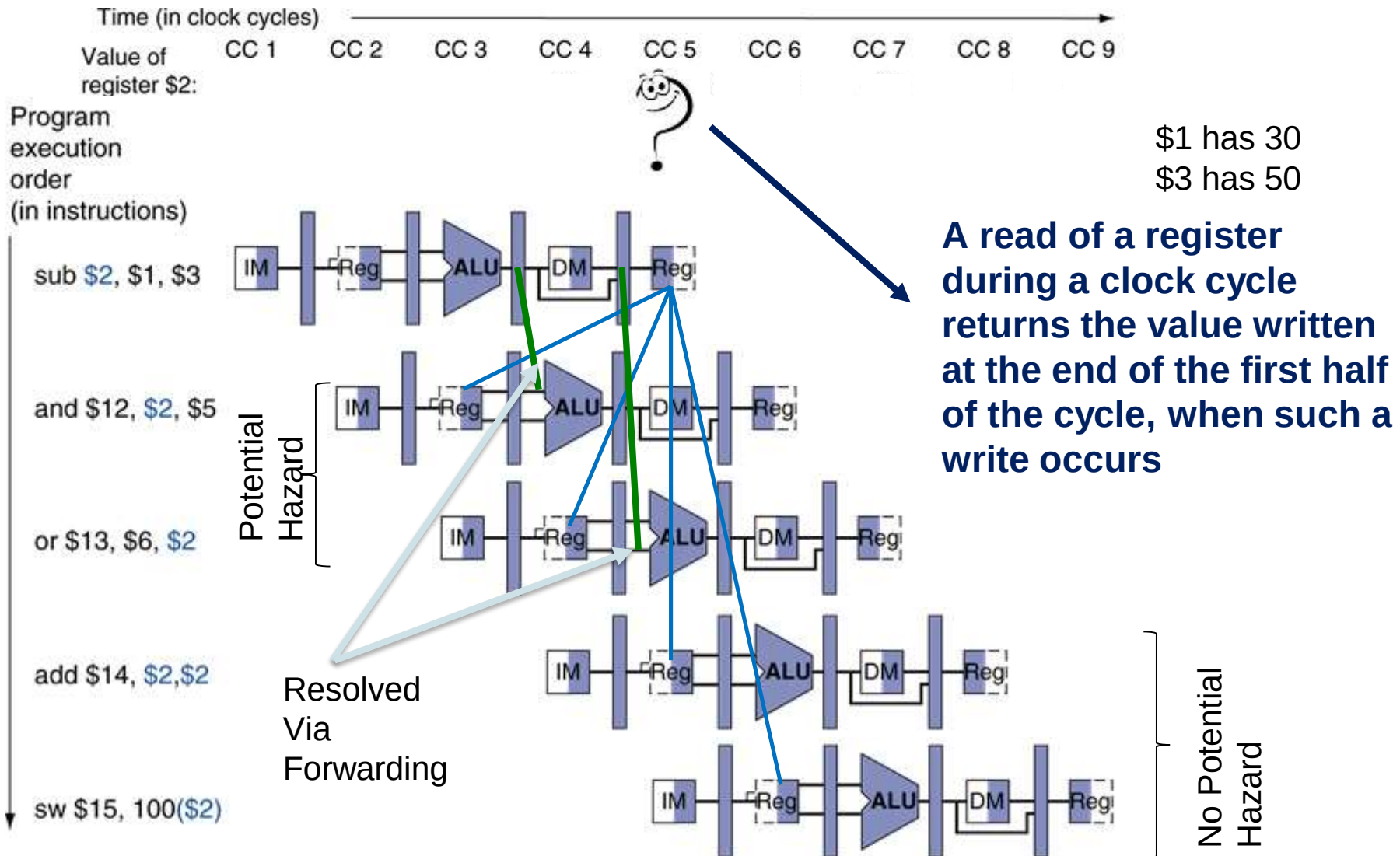
sw \$15, 100(\$2)

- We can resolve hazards with forwarding
  - How do we detect when to forward?

# Dependencies & Forwarding



# Dependencies & Forwarding



# Detecting the Need to Forward

- Pass register numbers along pipeline
  - e.g., ID/EX.RegisterRs = register number for Rs sitting in ID/EX pipeline register
- ALU operand register numbers in EX stage are given by
  - ID/EX.RegisterRs, ID/EX.RegisterRt
- Data hazards when
  - EX/MEM.RegisterRd = ID/EX.RegisterRs
  - EX/MEM.RegisterRd = ID/EX.RegisterRt
  - MEM/WB.RegisterRd = ID/EX.RegisterRs
  - MEM/WB.RegisterRd = ID/EX.RegisterRt

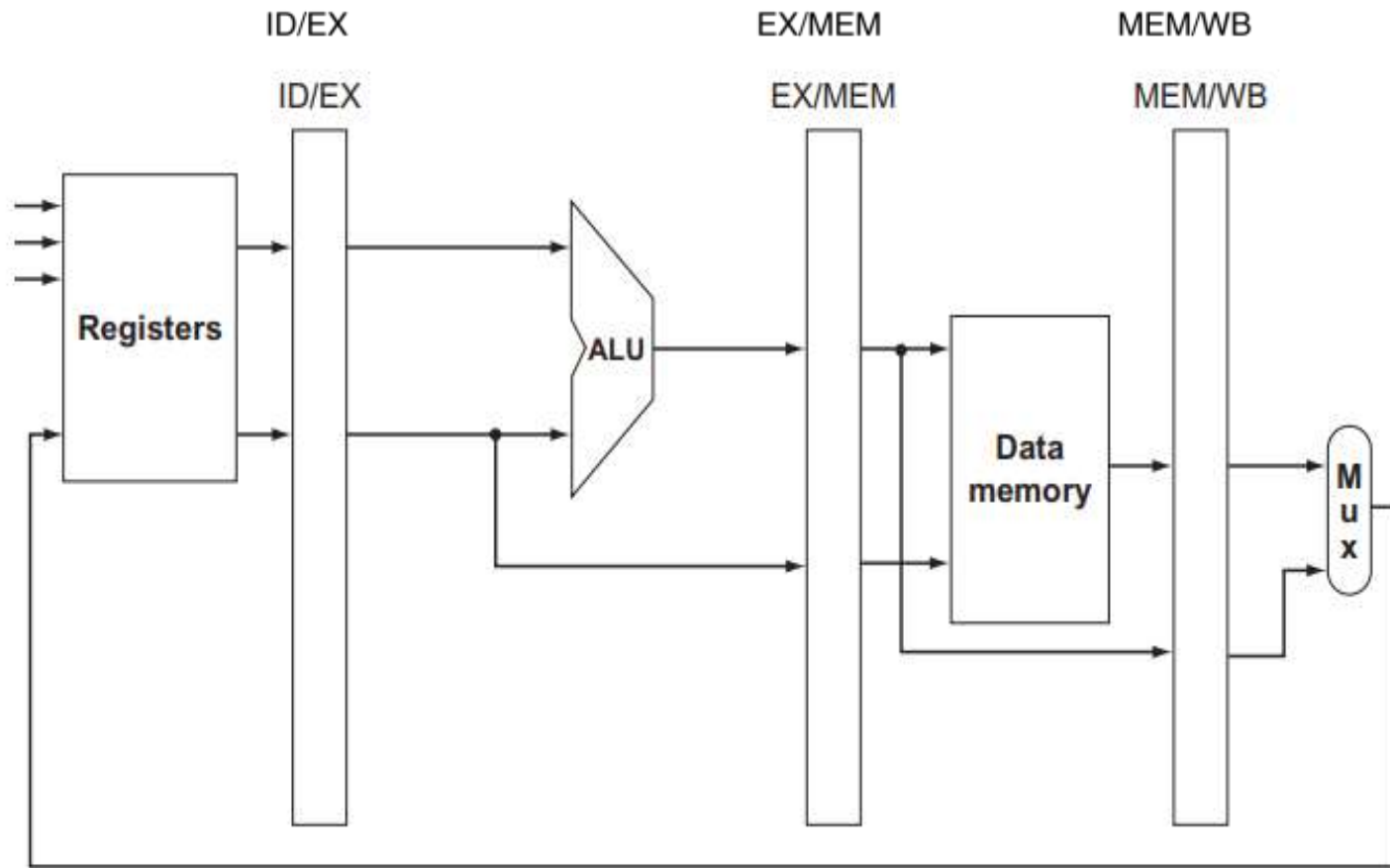
Fwd from  
EX/MEM  
pipeline reg

Fwd from  
MEM/WB  
pipeline reg

# Detecting the Need to Forward

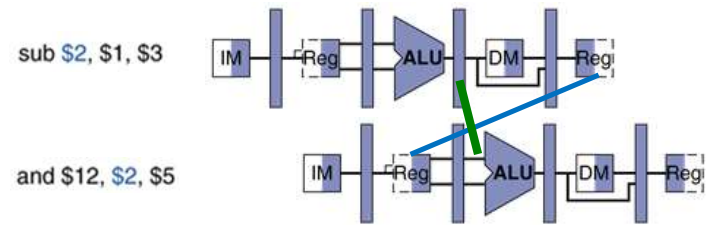
- But only if forwarding instruction will write to a register!
  - EX/MEM.RegWrite, MEM/WB.RegWrite
- And only if Rd for that instruction is not \$zero
  - EX/MEM.RegisterRd  $\neq$  0,  
MEM/WB.RegisterRd  $\neq$  0

# Forwarding Paths

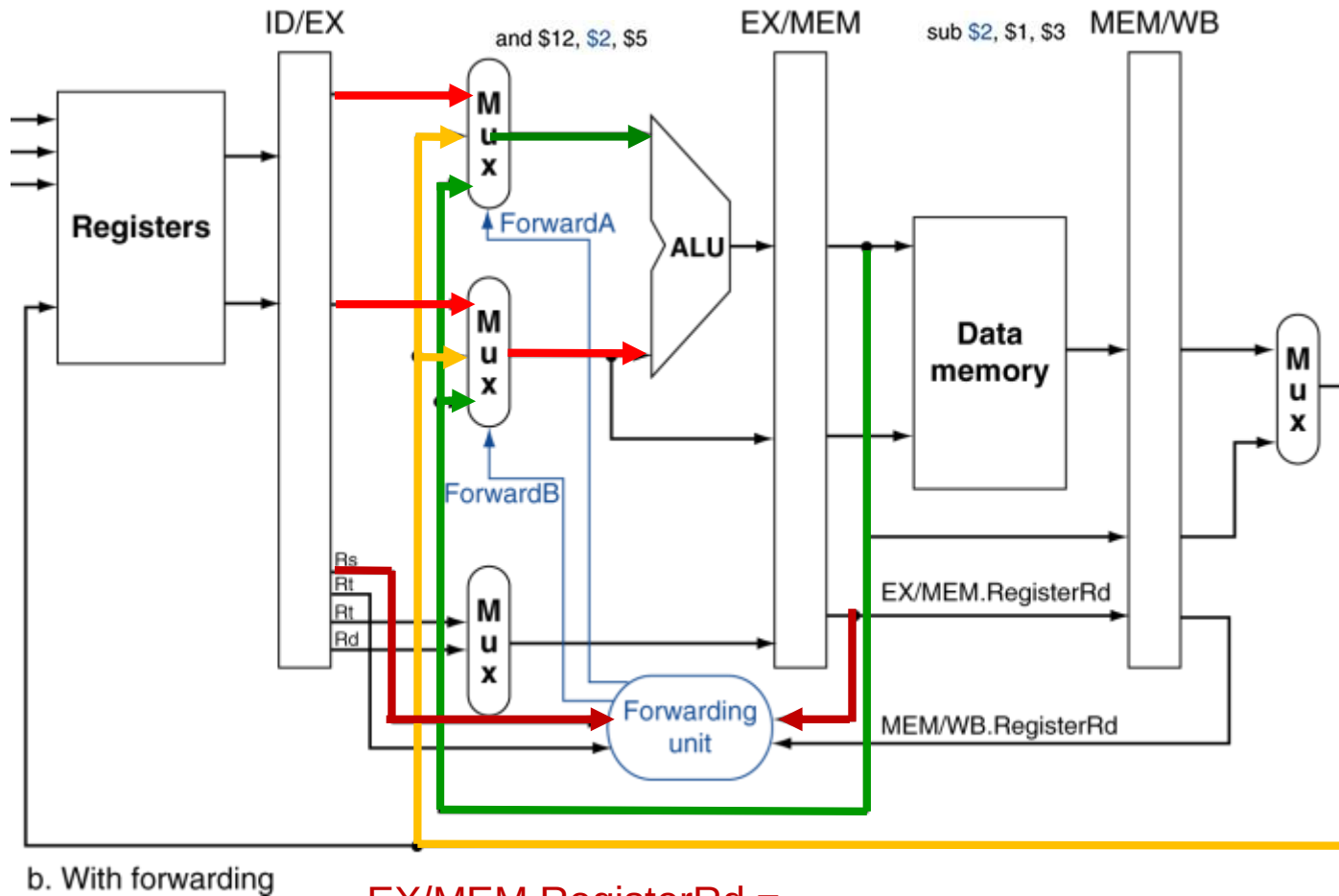


a. No forwarding

# Forwarding Paths

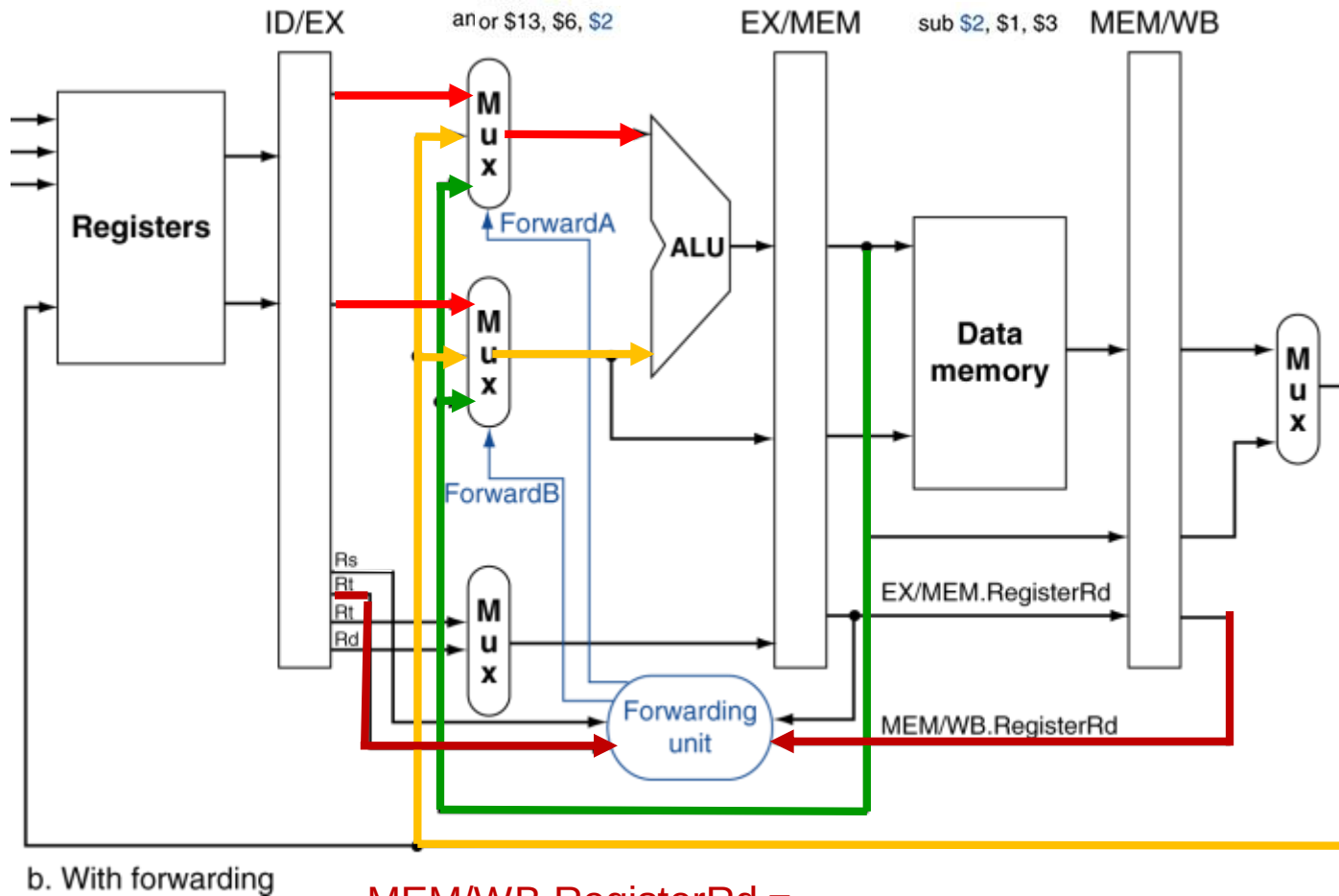


We have not shown regWrite and \$zero checks here.



EX/MEM.RegisterRd =  
ID/EX.RegisterRs

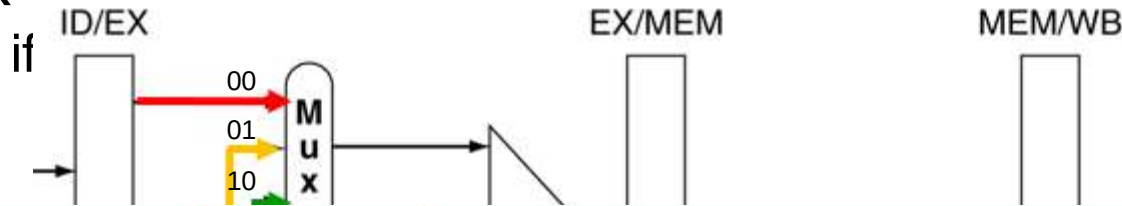
**We have not shown regWrite and \$zero checks here.**



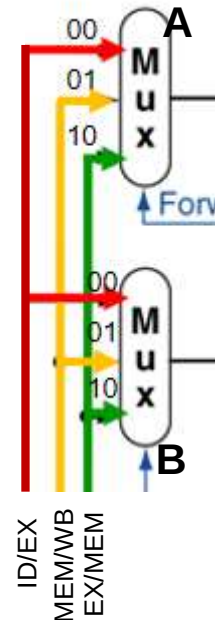
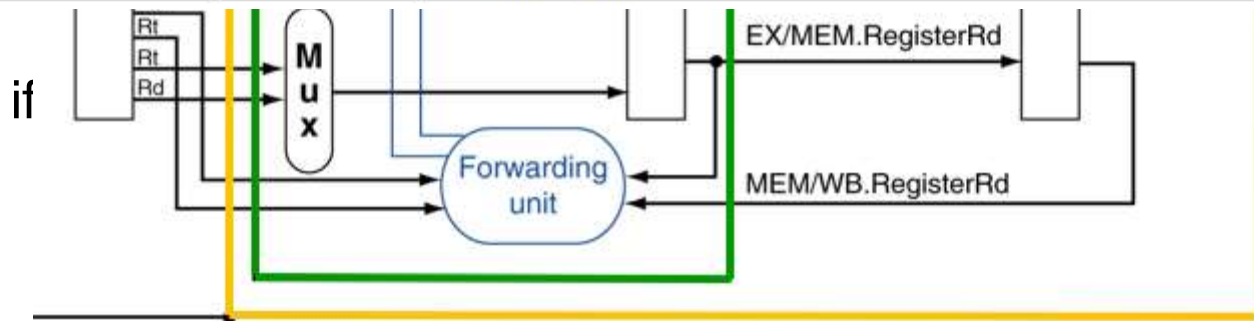
60

# Forwarding Conditions

EX hazard



| Mux control   | Source | Explanation                                                                    |
|---------------|--------|--------------------------------------------------------------------------------|
| ForwardA = 00 | ID/EX  | The first ALU operand comes from the register file.                            |
| ForwardA = 10 | EX/MEM | The first ALU operand is forwarded from the prior ALU result.                  |
| ForwardA = 01 | MEM/WB | The first ALU operand is forwarded from data memory or an earlier ALU result.  |
| ForwardB = 00 | ID/EX  | The second ALU operand comes from the register file.                           |
| ForwardB = 10 | EX/MEM | The second ALU operand is forwarded from the prior ALU result.                 |
| ForwardB = 01 | MEM/WB | The second ALU operand is forwarded from data memory or an earlier ALU result. |



# Double Data Hazard

- Consider the sequence:
  - add \$1, \$1, \$2
  - add \$1, \$1, \$3
  - add \$1, \$1, \$4
- Both hazards occur
  - Want to use the most recent
- Revise MEM hazard condition
  - Only fwd if EX hazard condition isn't true

# Double Data Hazard

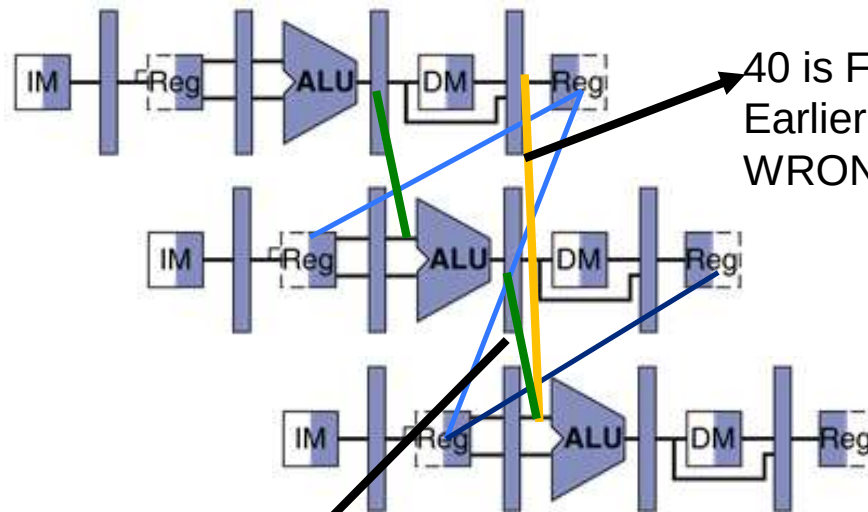
|      |     |     |     |     |       |     |     |
|------|-----|-----|-----|-----|-------|-----|-----|
| CC:  | cc1 | cc2 | cc3 | cc4 | cc5   | cc6 | cc7 |
| \$2: | 30  | 30  | 30  | 30  | 30    | 30  | 30  |
| \$3: | 20  | 20  | 20  | 20  | 20    | 20  | 20  |
| \$4: | 4   | 4   | 4   | 4   | 4     | 4   | 4   |
| \$1: | 10  | 10  | 10  | 10  | 10/40 |     |     |

What is expected?

add \$1, \$1, \$2

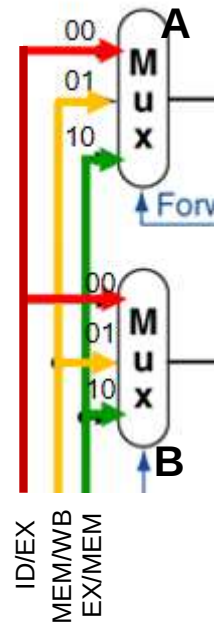
add \$1, \$1, \$3

add \$1, \$1, \$4



40 is Forwarded  
Earlier value;  
WRONG!!!

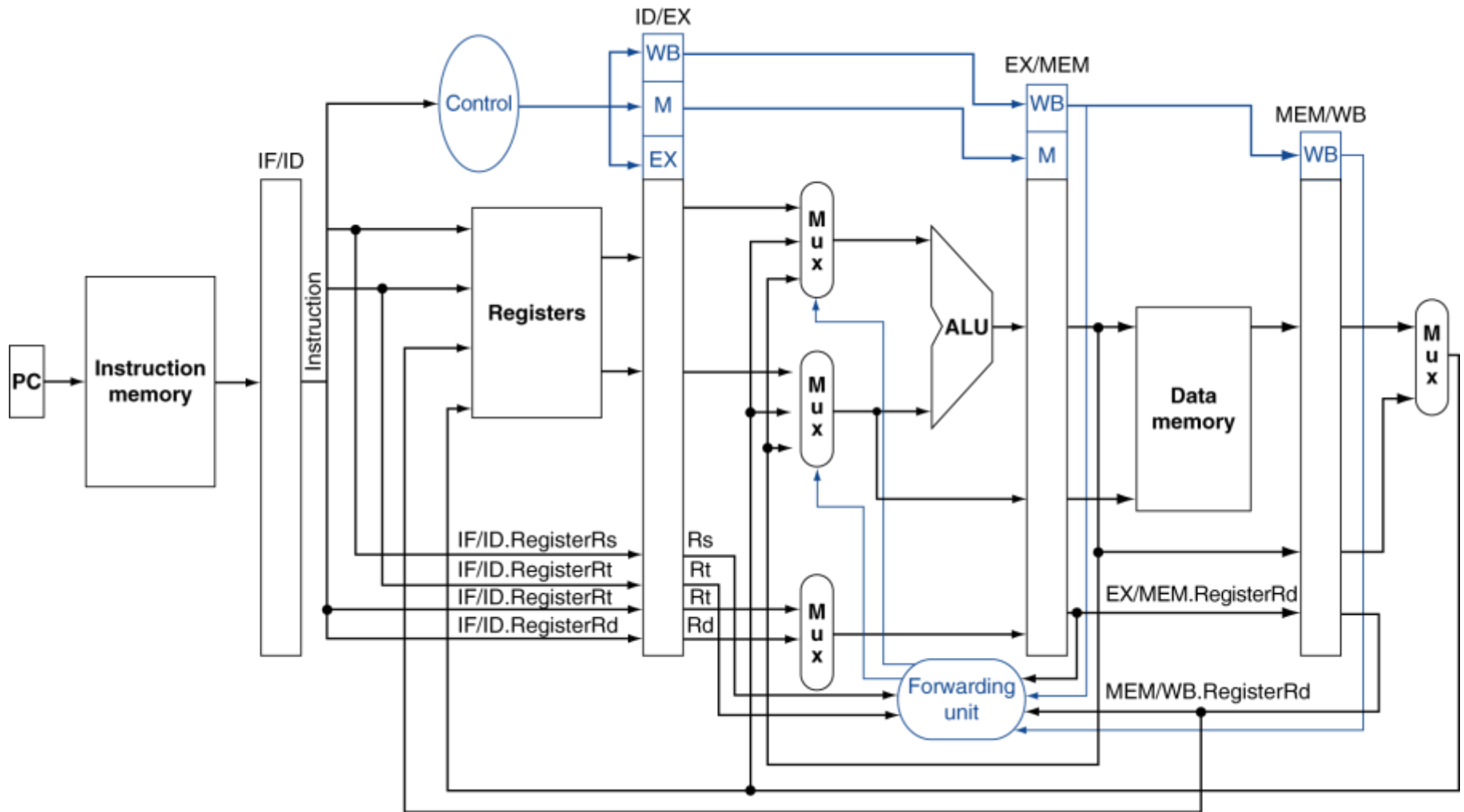
60 is Forwarded  
Correct!!!



# Revised Forwarding Condition

- MEM hazard
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd  $\neq$  0)  
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)  
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))  
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))  
ForwardA = 01
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd  $\neq$  0)  
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)  
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))  
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))  
ForwardB = 01

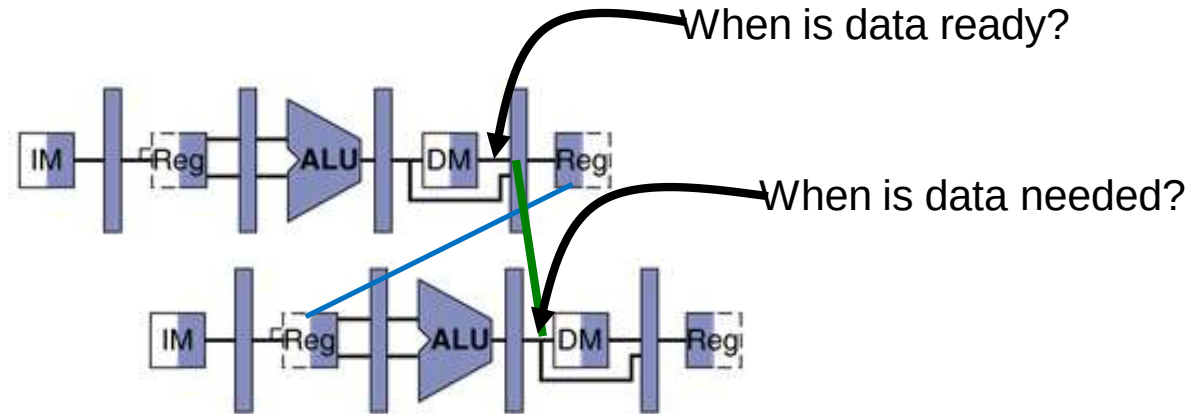
# Datapath with Forwarding



# Load -> Store (Mem to Mem Copy)

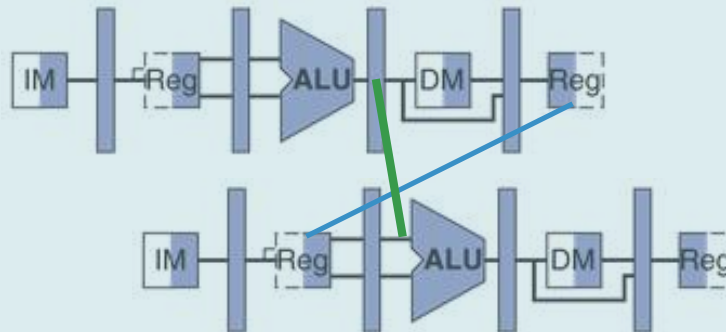
lw \$2, 20(\$1)

sw \$2, 15(\$3)

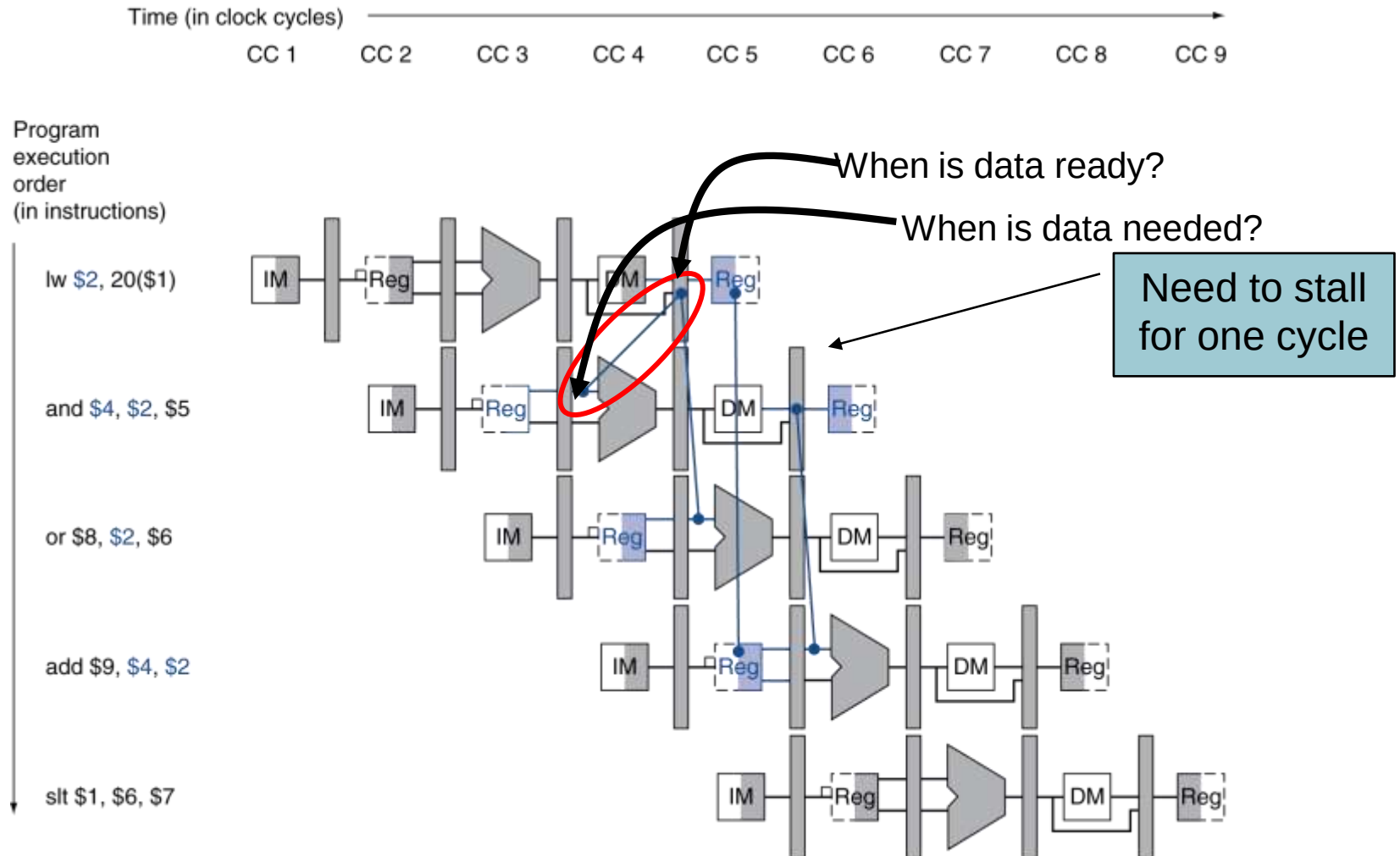


sub \$2, \$1, \$3

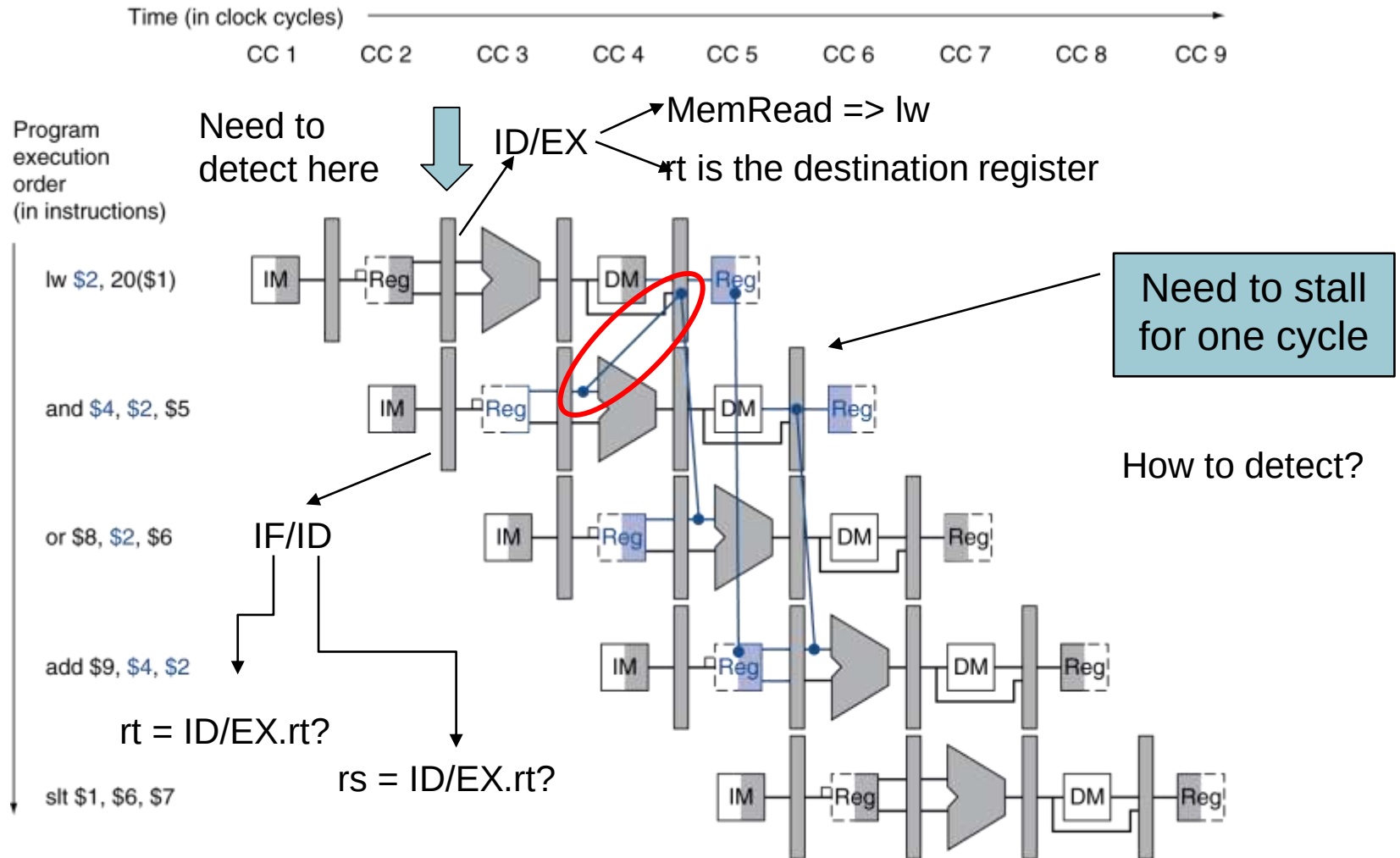
and \$12, \$2, \$5



# Load-Use Data Hazard



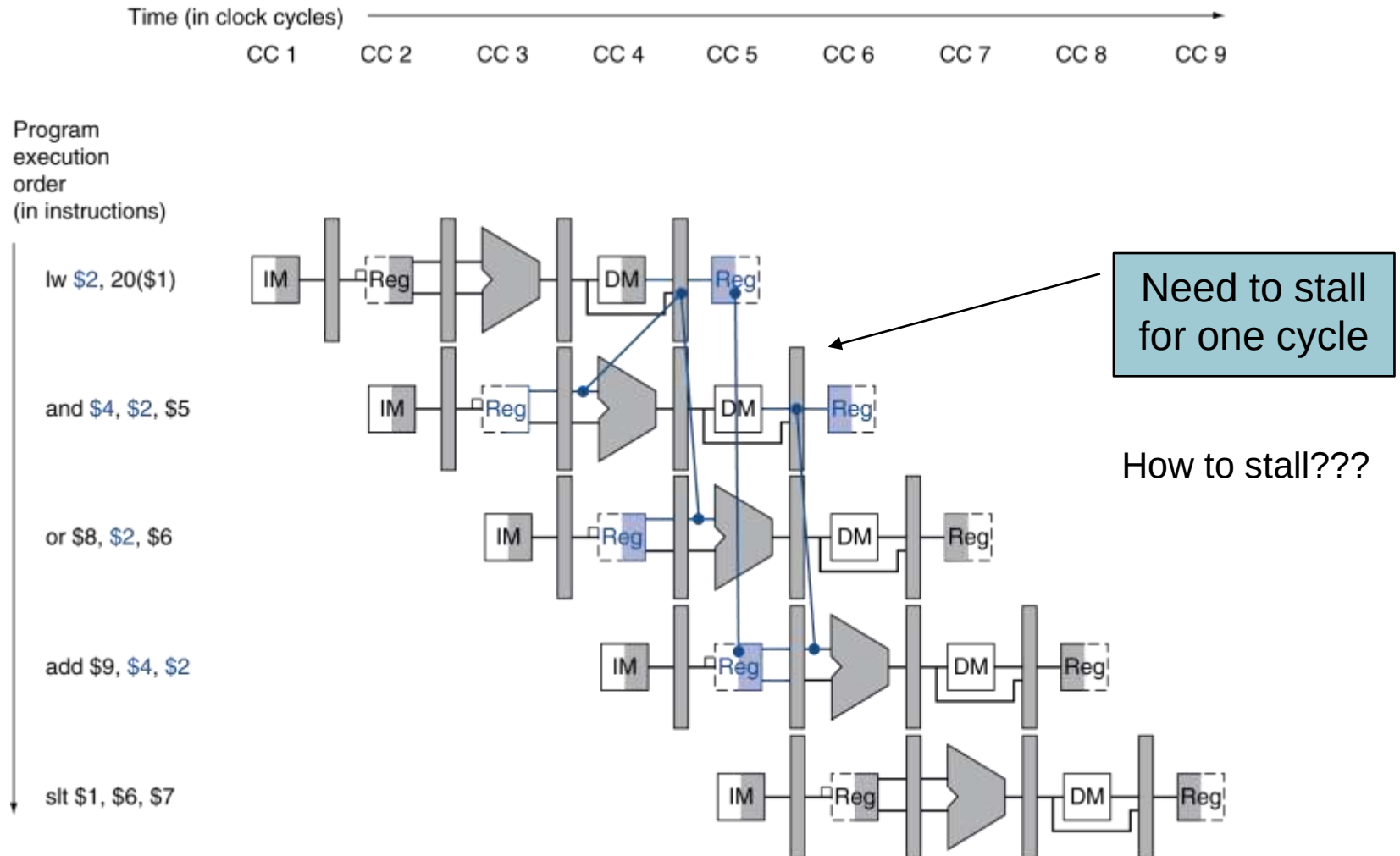
# Load-Use Data Hazard



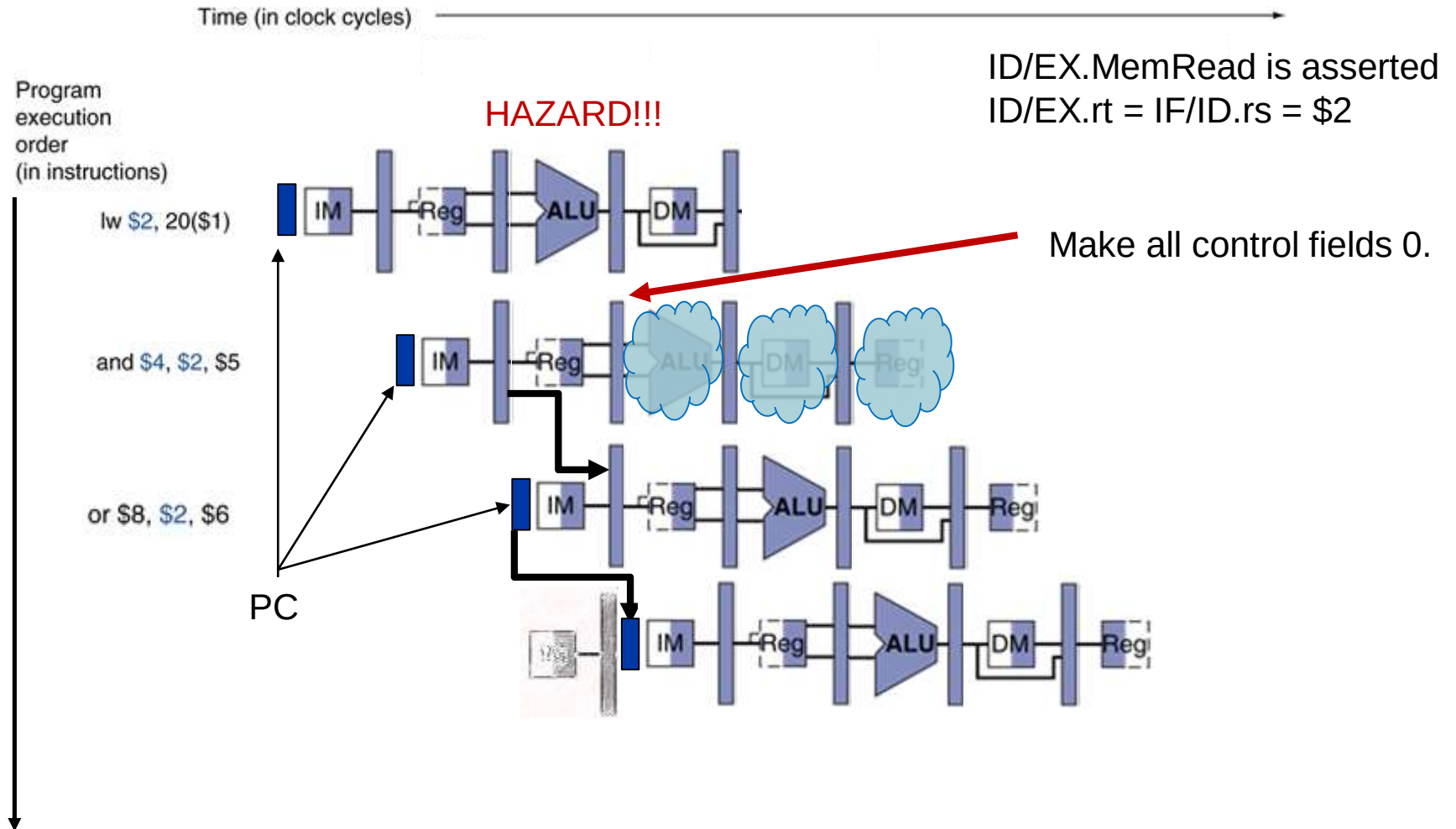
# Load-Use Hazard Detection

- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
  - IF/ID.RegisterRs, IF/ID.RegisterRt
- Load-use hazard when
  - ID/EX.MemRead and  
((ID/EX.RegisterRt = IF/ID.RegisterRs) or  
(ID/EX.RegisterRt = IF/ID.RegisterRt))
- If detected, stall and insert bubble

# Load-Use Data Hazard



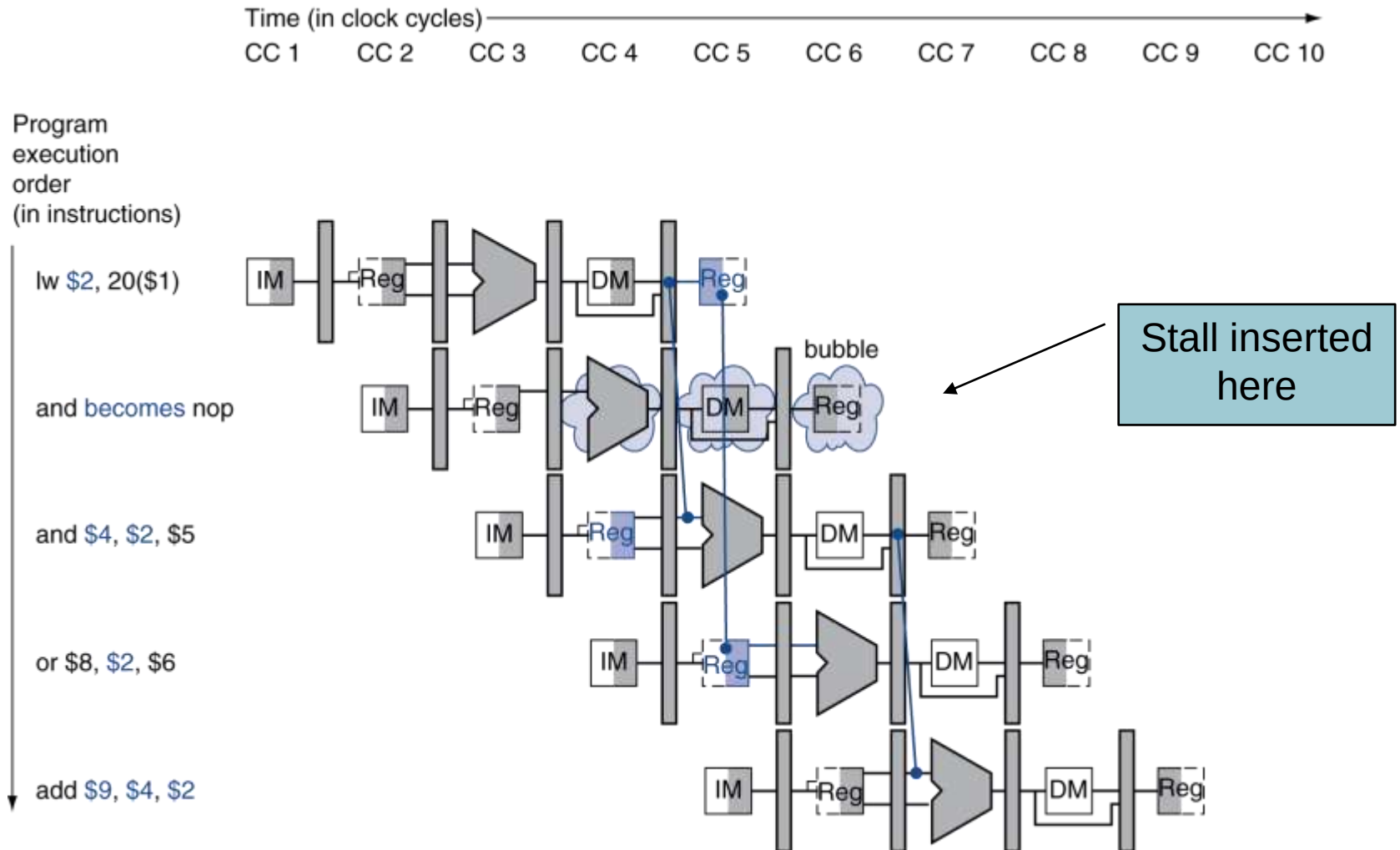
# How to Stall



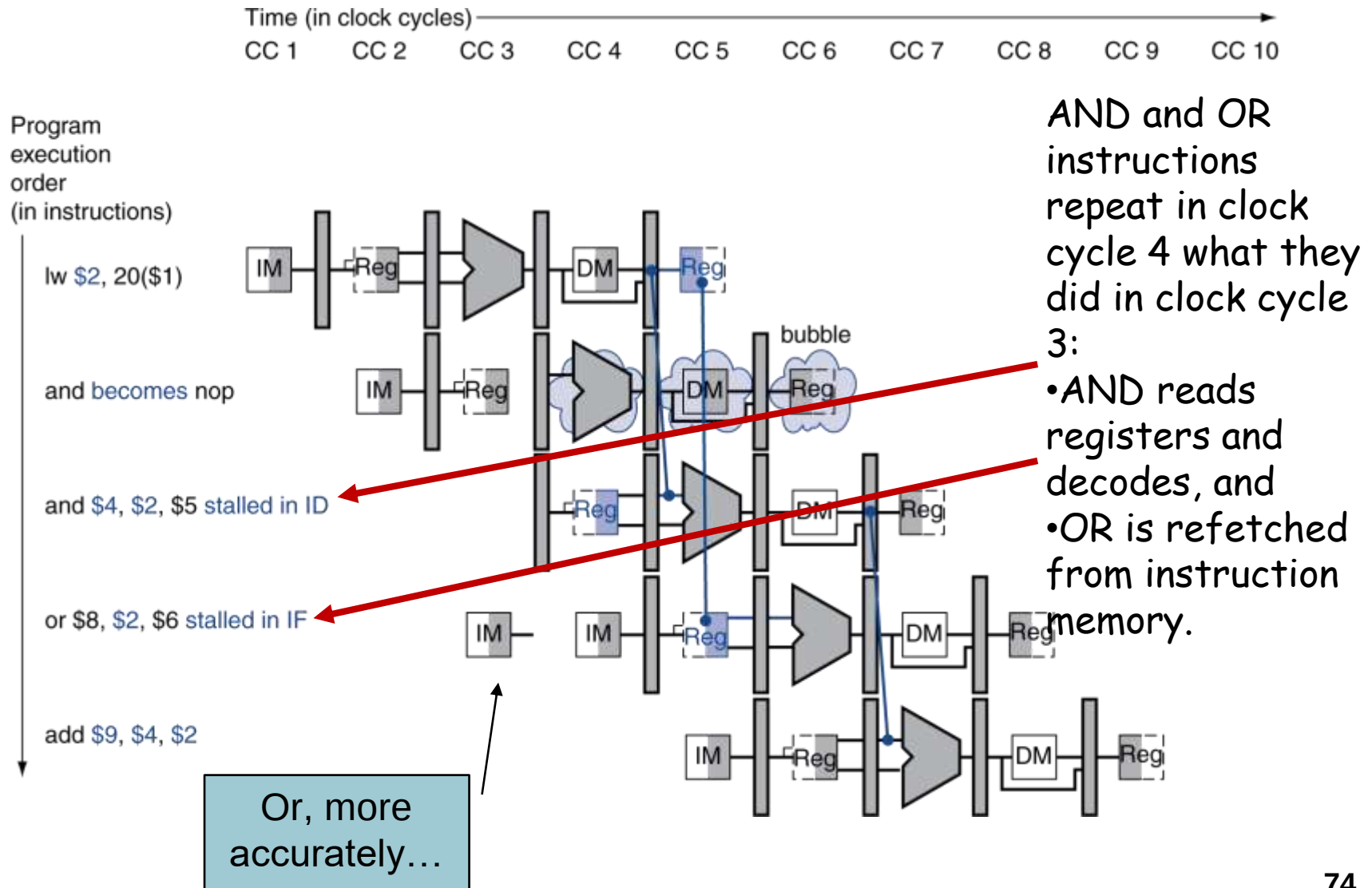
# How to Stall the Pipeline

- Force control values in ID/EX register to 0
  - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for 1w
    - Can subsequently forward to EX stage

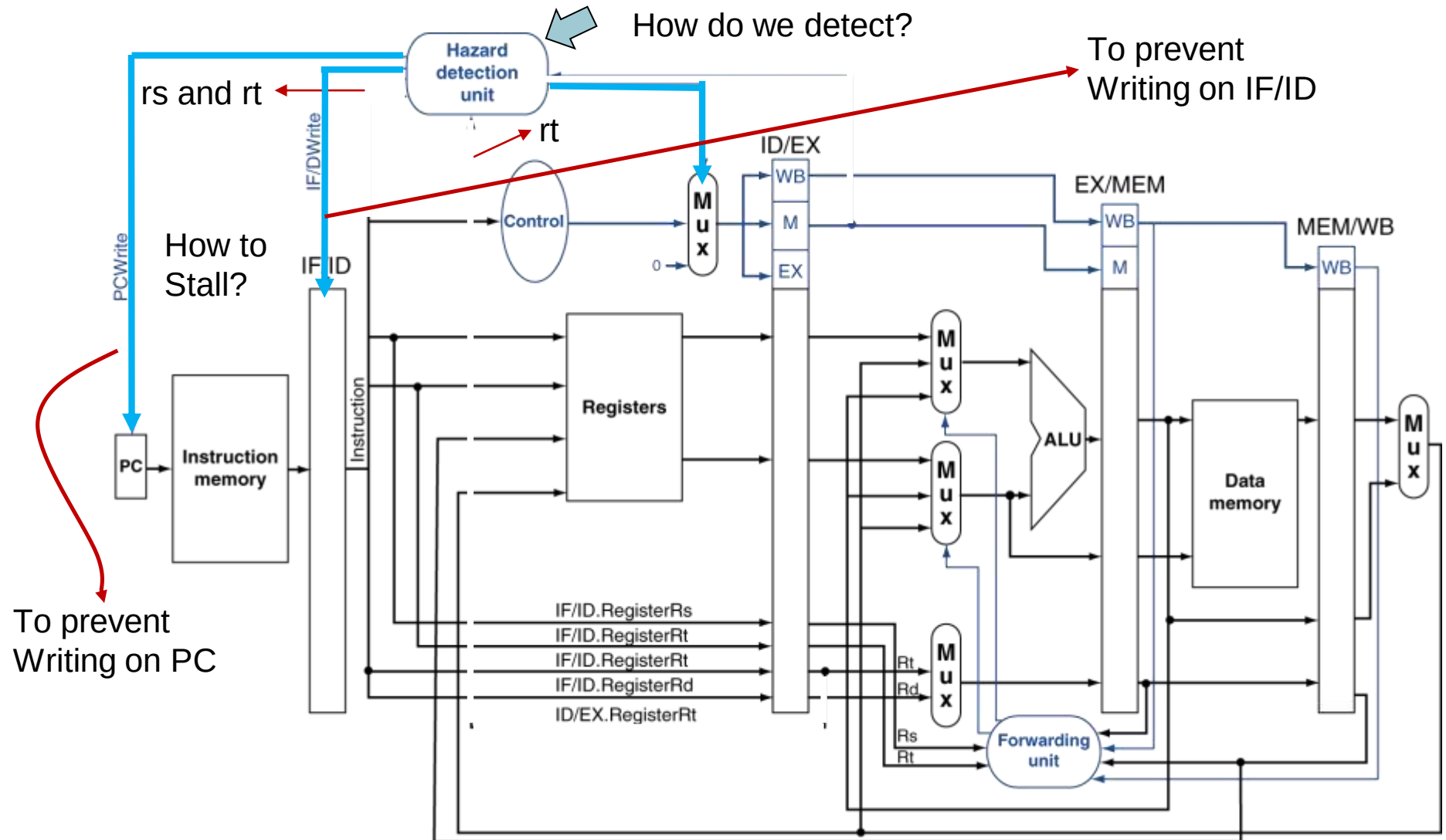
# Stall/Bubble in the Pipeline



# Stall/Bubble in the Pipeline



# Datapath with Hazard Detection



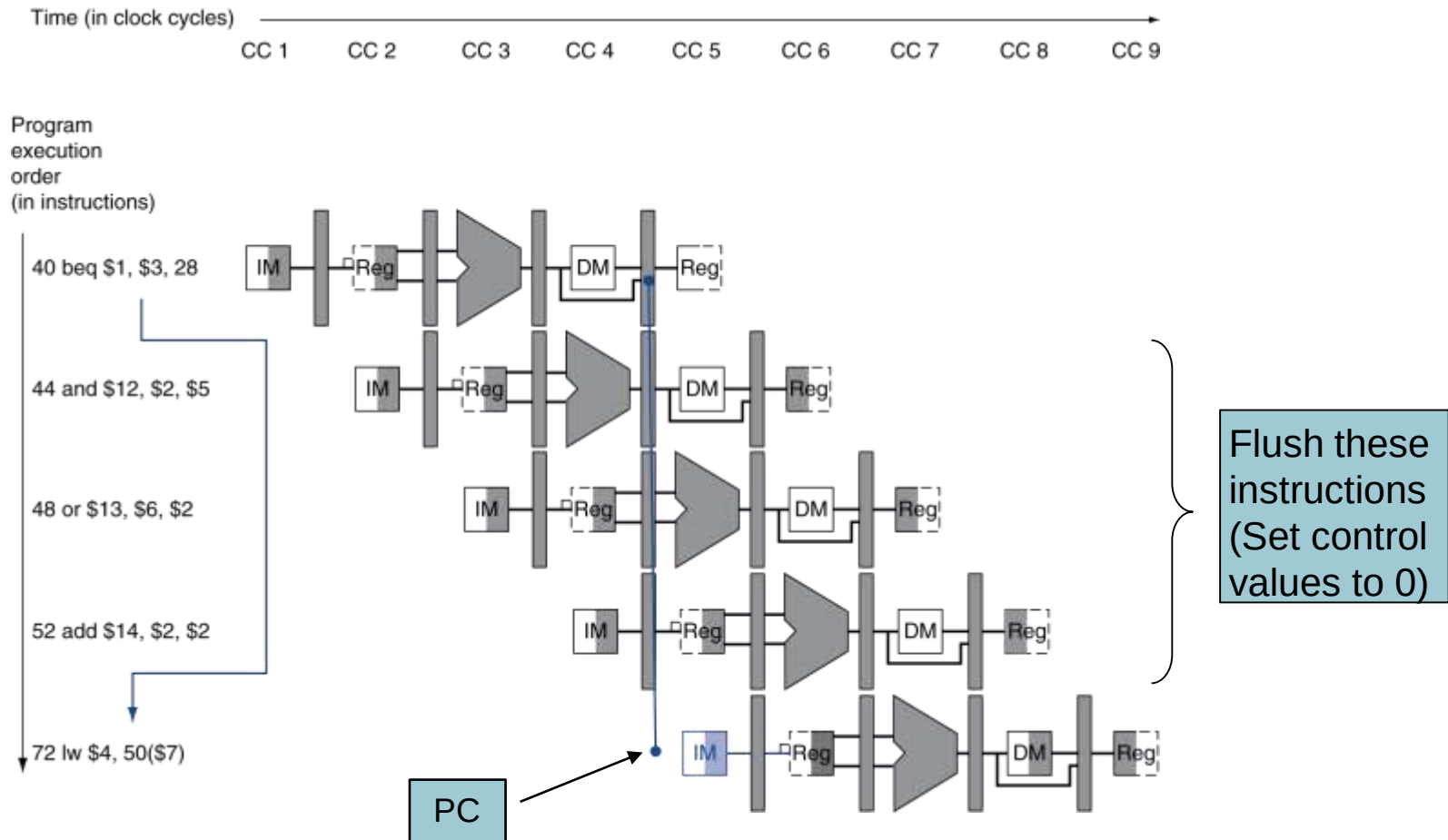
# Stalls and Performance

## The BIG Picture

- Stalls reduce performance
  - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
  - Requires knowledge of the pipeline structure

# Branch Hazards

- If branch outcome determined in MEM



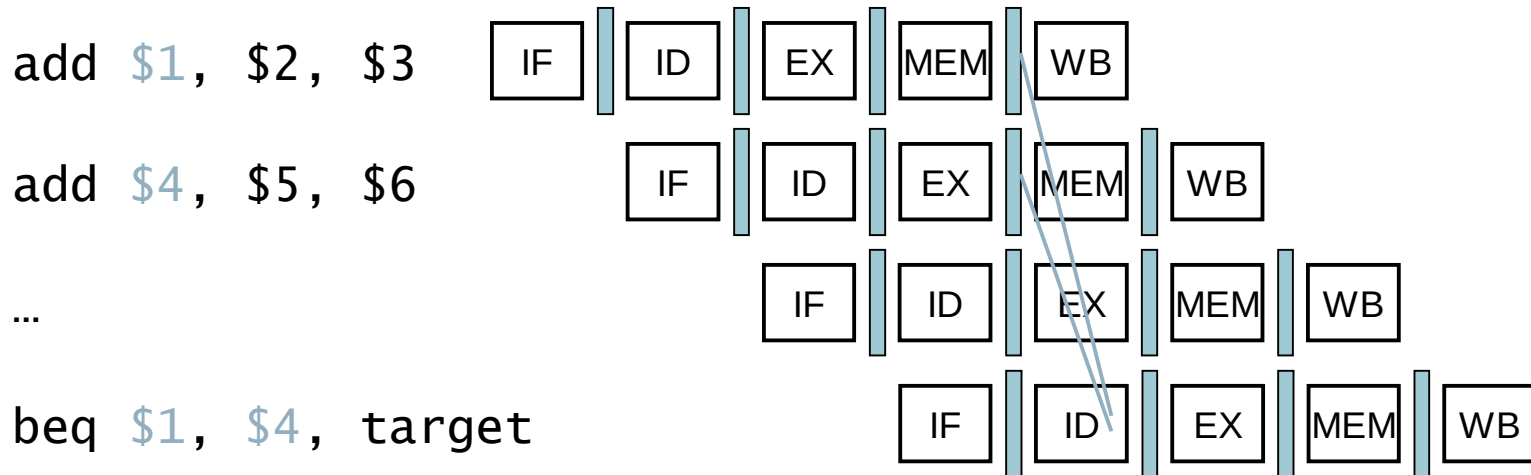
# Reducing Branch Delay

- Move hardware to determine outcome to ID stage
  - Target address adder
  - Register comparator
- Example: branch taken

```
36:  sub    $10, $4, $8
40:  beq    $1,  $3, 7
44:  and    $12, $2, $5
48:  or     $13, $2, $6
52:  add    $14, $4, $2
56:  slt    $15, $6, $7
    ...
72:  lw     $4, 50($7)
```

# Data Hazards for Branches

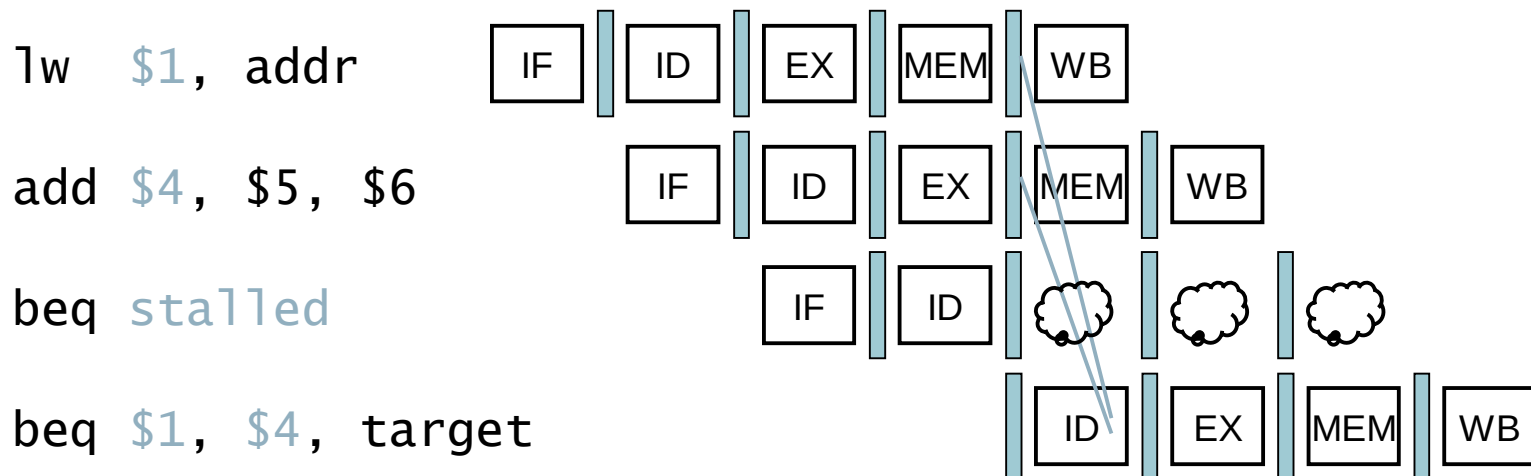
- If a comparison register is a destination of 2<sup>nd</sup> or 3<sup>rd</sup> preceding ALU instruction



- Can resolve using forwarding

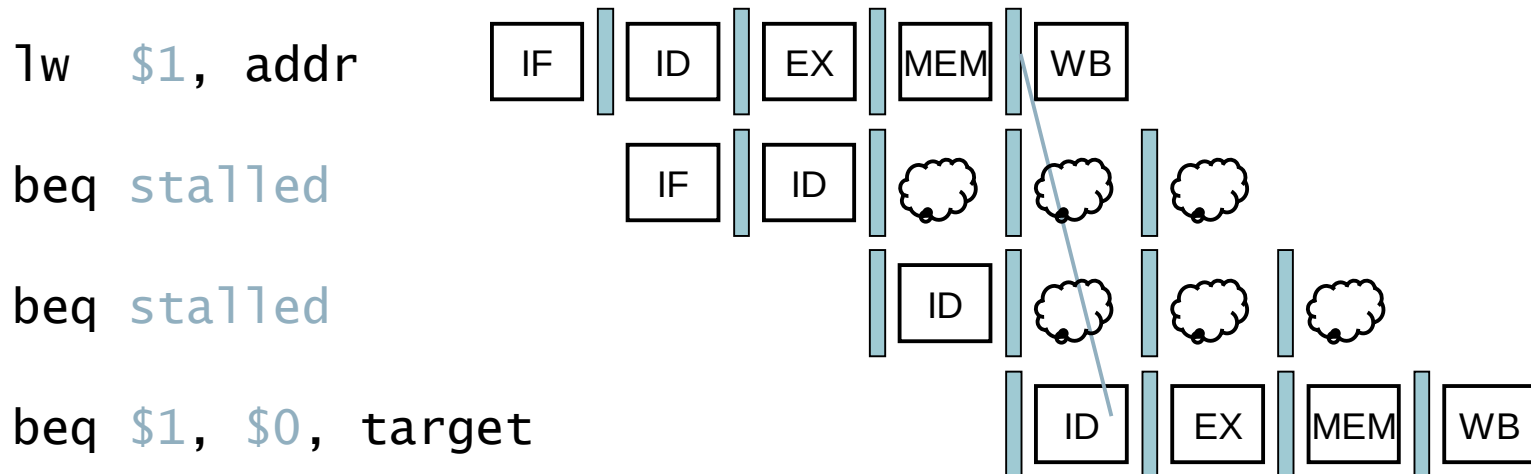
# Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2<sup>nd</sup> preceding load instruction
  - Need 1 stall cycle



# Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction
  - Need 2 stall cycles

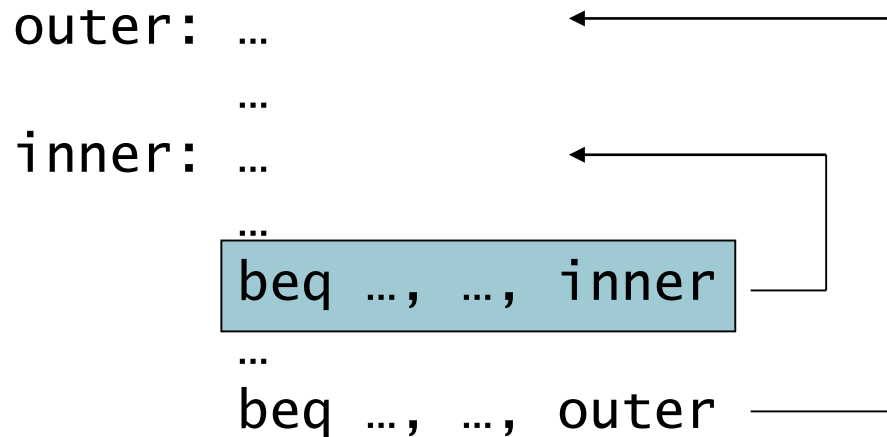


# Dynamic Branch Prediction

- In deeper and superscalar pipelines, branch penalty is more significant
- Use dynamic prediction
  - Branch prediction buffer (aka branch history table)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
  - To execute a branch
    - Check table, expect the same outcome
    - Start fetching from fall-through or target
    - If wrong, flush pipeline and flip prediction

# 1-Bit Predictor: Shortcoming

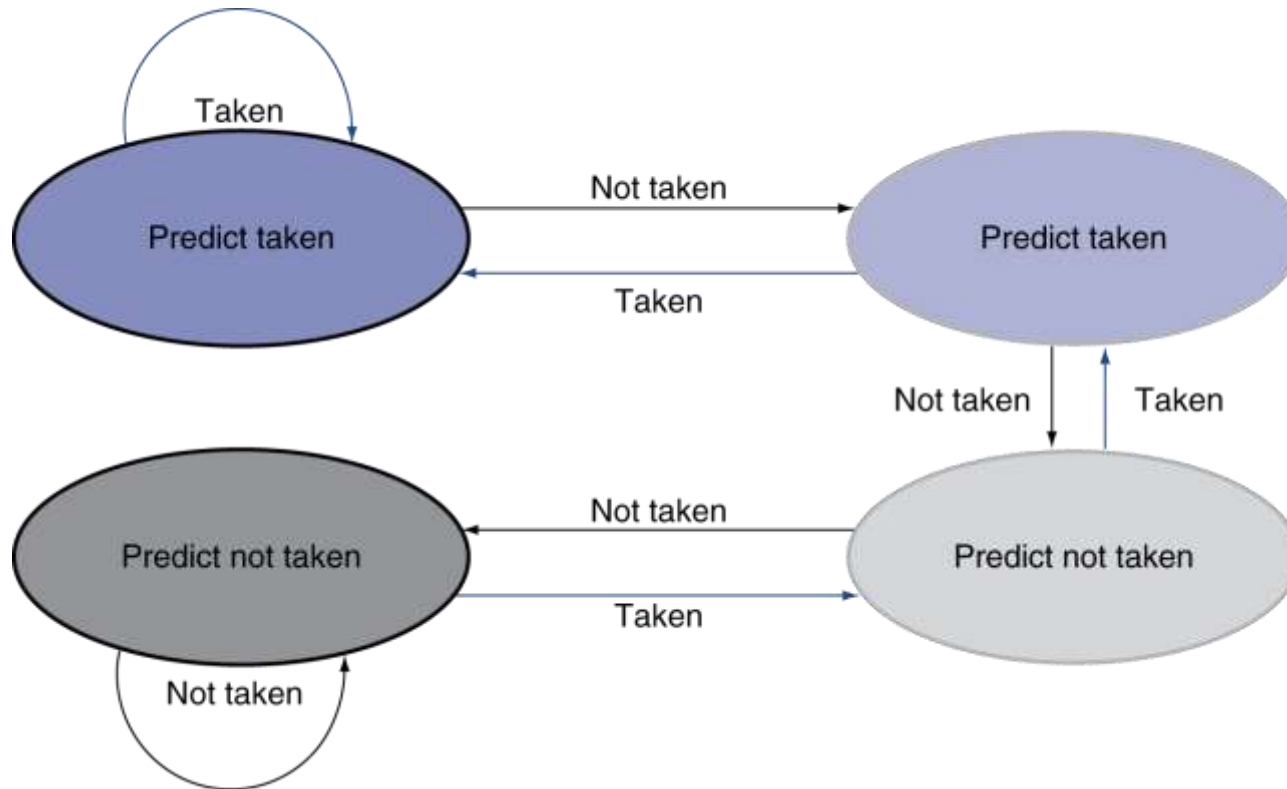
- Inner loop branches mispredicted twice!



- Mispredict as taken on last iteration of inner loop
- Then mispredict as not taken on first iteration of inner loop next time around

# 2-Bit Predictor

- Only change prediction on two successive mispredictions



# Calculating the Branch Target

- Even with predictor, still need to calculate the target address
  - 1-cycle penalty for a taken branch
- Branch target buffer
  - Cache of target addresses
  - Indexed by PC when instruction fetched
    - If hit and instruction is branch predicted taken, can fetch target immediately

# Instruction-Level Parallelism (ILP)

- Pipelining: executing multiple instructions in parallel
- To increase ILP
  - Deeper pipeline
    - Less work per stage  $\Rightarrow$  shorter clock cycle
  - Multiple issue
    - Replicate pipeline stages  $\Rightarrow$  multiple pipelines
    - Start multiple instructions per clock cycle
    - $CPI < 1$ , so use Instructions Per Cycle (IPC)
    - E.g., 4GHz 4-way multiple-issue
      - 16 BIPS, peak  $CPI = 0.25$ , peak  $IPC = 4$
    - But dependencies reduce this in practice

# Multiple Issue

- Static multiple issue
  - Compiler groups instructions to be issued together
  - Packages them into “issue slots”
  - Compiler detects and avoids hazards
- Dynamic multiple issue
  - CPU examines instruction stream and chooses instructions to issue each cycle
  - Compiler can help by reordering instructions
  - CPU resolves hazards using advanced techniques at runtime

# Speculation

- “Guess” what to do with an instruction
  - Start operation as soon as possible
  - Check whether guess was right
    - If so, complete the operation
    - If not, roll-back and do the right thing
- Common to static and dynamic multiple issue
- Examples
  - Speculate on branch outcome
    - Roll back if path taken is different
  - Speculate on load
    - Roll back if location is updated

# Compiler/Hardware Speculation

- Compiler can reorder instructions
  - e.g., move load before branch
  - Can include “fix-up” instructions to recover from incorrect guess
- Hardware can look ahead for instructions to execute
  - Buffer results until it determines they are actually needed
  - Flush buffers on incorrect speculation

# Static Multiple Issue

- Compiler groups instructions into “issue packets”
  - Group of instructions that can be issued on a single cycle
  - Determined by pipeline resources required
- Think of an issue packet as a very long instruction
  - Specifies multiple concurrent operations
  - $\Rightarrow$  Very Long Instruction Word (VLIW)

# Scheduling Static Multiple Issue

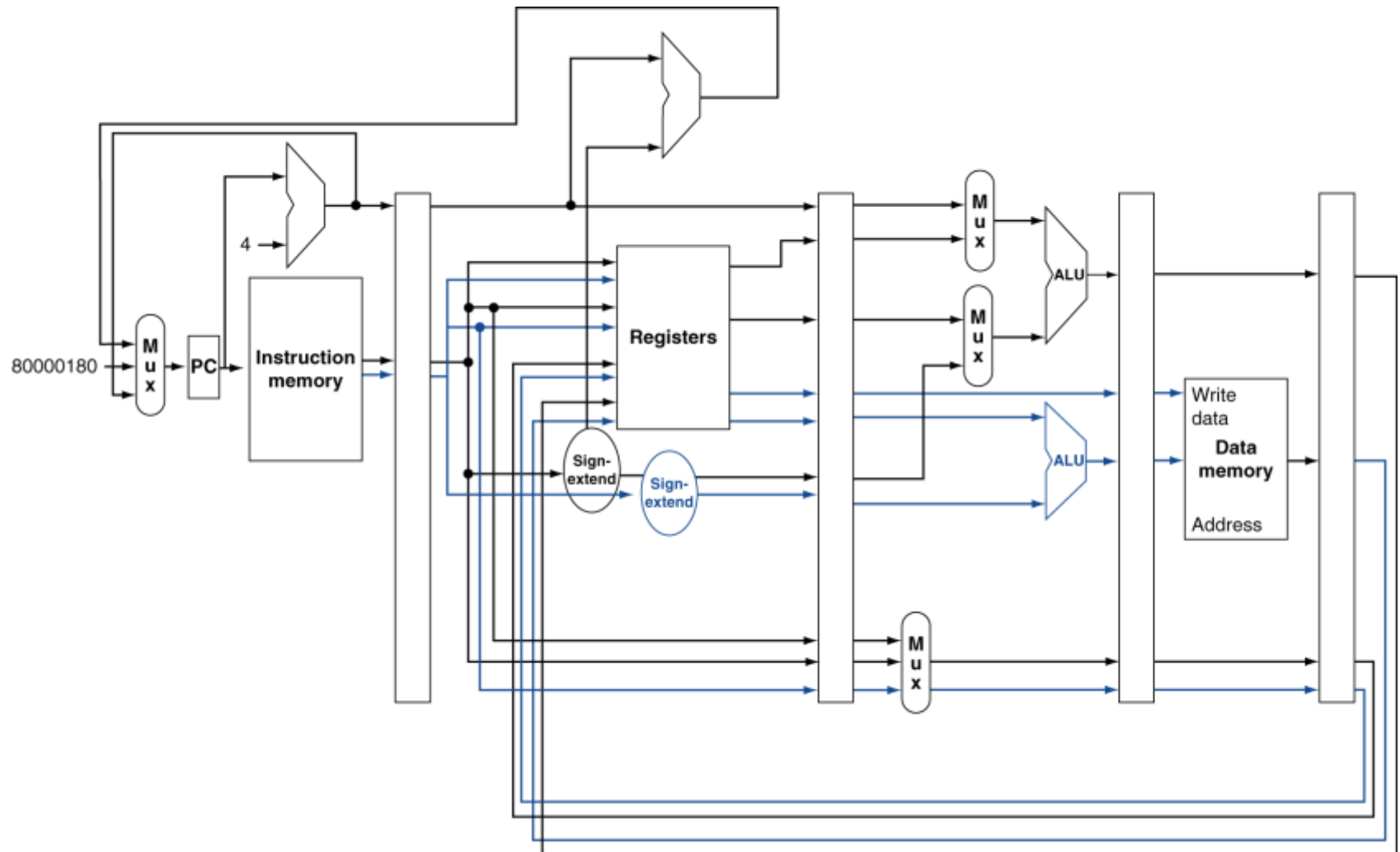
- Compiler must remove some/all hazards
  - Reorder instructions into issue packets
  - No dependencies within a packet
  - Possibly some dependencies between packets
    - Varies between ISAs; compiler must know!
  - Pad with nop if necessary

# MIPS with Static Dual Issue

- Two-issue packets
  - One ALU/branch instruction
  - One load/store instruction
  - 64-bit aligned
    - ALU/branch, then load/store
    - Pad an unused instruction with nop

| Address | Instruction type | Pipeline Stages |    |    |     |     |     |    |
|---------|------------------|-----------------|----|----|-----|-----|-----|----|
| n       | ALU/branch       | IF              | ID | EX | MEM | WB  |     |    |
| n + 4   | Load/store       | IF              | ID | EX | MEM | WB  |     |    |
| n + 8   | ALU/branch       |                 | IF | ID | EX  | MEM | WB  |    |
| n + 12  | Load/store       |                 | IF | ID | EX  | MEM | WB  |    |
| n + 16  | ALU/branch       |                 |    | IF | ID  | EX  | MEM | WB |
| n + 20  | Load/store       |                 |    | IF | ID  | EX  | MEM | WB |

# MIPS with Static Dual Issue



# Hazards in the Dual-Issue MIPS

- More instructions executing in parallel
- EX data hazard
  - Forwarding avoided stalls with single-issue
  - Now can't use ALU result in load/store in same packet
    - add `$t0, $s0, $s1`  
load `$s2, 0($t0)`
    - Split into two packets, effectively a stall
- Load-use hazard
  - Still one cycle use latency, but now two instructions
- More aggressive scheduling required

# Scheduling Example

- Schedule this for dual-issue MIPS

```
Loop: lw    $t0, 0($s1)      # $t0=array element
      addu  $t0, $t0, $s2    # add scalar in $s2
      sw    $t0, 0($s1)      # store result
      addi  $s1, $s1, -4     # decrement pointer
      bne   $s1, $zero, Loop # branch $s1!=0
```

|       | ALU/branch               | Load/store          | cycle |
|-------|--------------------------|---------------------|-------|
| Loop: | nop                      | lw    \$t0, 0(\$s1) | 1     |
|       | addi  \$s1, \$s1, -4     | nop                 | 2     |
|       | addu  \$t0, \$t0, \$s2   | nop                 | 3     |
|       | bne   \$s1, \$zero, Loop | sw    \$t0, 4(\$s1) | 4     |

- $IPC = 5/4 = 1.25$  (c.f. peak  $IPC = 2$ )

# Loop Unrolling

- Replicate loop body to expose more parallelism
  - Reduces loop-control overhead
- Use different registers per replication
  - Called “register renaming”
  - Avoid loop-carried “anti-dependencies”
    - Store followed by a load of the same register
    - Aka “name dependence”
      - Reuse of a register name

# Loop Unrolling Example

|       | ALU/branch                            | Load/store                | cycle |
|-------|---------------------------------------|---------------------------|-------|
| Loop: | addi <b>\$s1</b> , \$s1, -16          | lw <b>\$t0</b> , 0(\$s1)  | 1     |
|       | nop                                   | lw <b>\$t1</b> , 12(\$s1) | 2     |
|       | addu <b>\$t0</b> , <b>\$t0</b> , \$s2 | lw <b>\$t2</b> , 8(\$s1)  | 3     |
|       | addu <b>\$t1</b> , <b>\$t1</b> , \$s2 | lw <b>\$t3</b> , 4(\$s1)  | 4     |
|       | addu <b>\$t2</b> , <b>\$t2</b> , \$s2 | sw <b>\$t0</b> , 16(\$s1) | 5     |
|       | addu <b>\$t3</b> , <b>\$t4</b> , \$s2 | sw <b>\$t1</b> , 12(\$s1) | 6     |
|       | nop                                   | sw <b>\$t2</b> , 8(\$s1)  | 7     |
|       | bne <b>\$s1</b> , \$zero, Loop        | sw <b>\$t3</b> , 4(\$s1)  | 8     |

- $IPC = 14/8 = 1.75$ 
  - Closer to 2, but at cost of registers and code size

# Dynamic Multiple Issue

- “Superscalar” processors
- CPU decides whether to issue 0, 1, 2, ... each cycle
  - Avoiding structural and data hazards
- Avoids the need for compiler scheduling
  - Though it may still help
  - Code semantics ensured by the CPU

# Dynamic Pipeline Scheduling

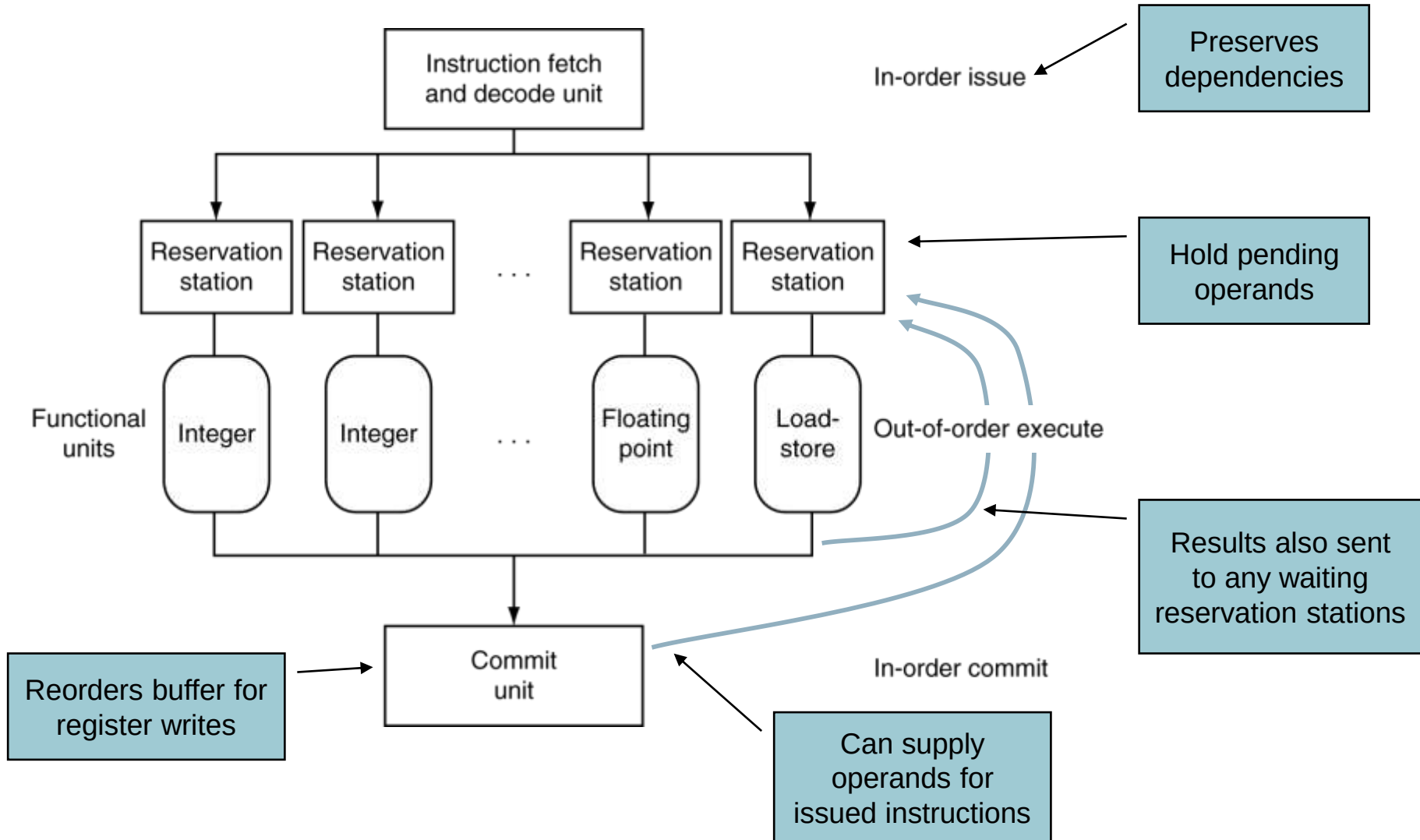
- Allow the CPU to execute instructions out of order to avoid stalls
  - But commit result to registers in order

- Example

```
lw      $t0, 20($s2)
addu    $t1, $t0, $t2
sub      $s4, $s4, $t3
slli    $t5, $s4, 20
```

- Can start sub while addu is waiting for lw

# Dynamically Scheduled CPU



# Why Do Dynamic Scheduling?

- Why not just let the compiler schedule code?
- Not all stalls are predicable
  - e.g., cache misses
- Can't always schedule around branches
  - Branch outcome is dynamically determined
- Different implementations of an ISA have different latencies and hazards

# Does Multiple Issue Work?

## The BIG Picture

- Yes, but not as much as we'd like
- Programs have real dependencies that limit ILP
- Some dependencies are hard to eliminate
  - e.g., pointer aliasing
- Some parallelism is hard to expose
  - Limited window size during instruction issue
- Memory delays and limited bandwidth
  - Hard to keep pipelines full
- Speculation can help if done well

# Power Efficiency

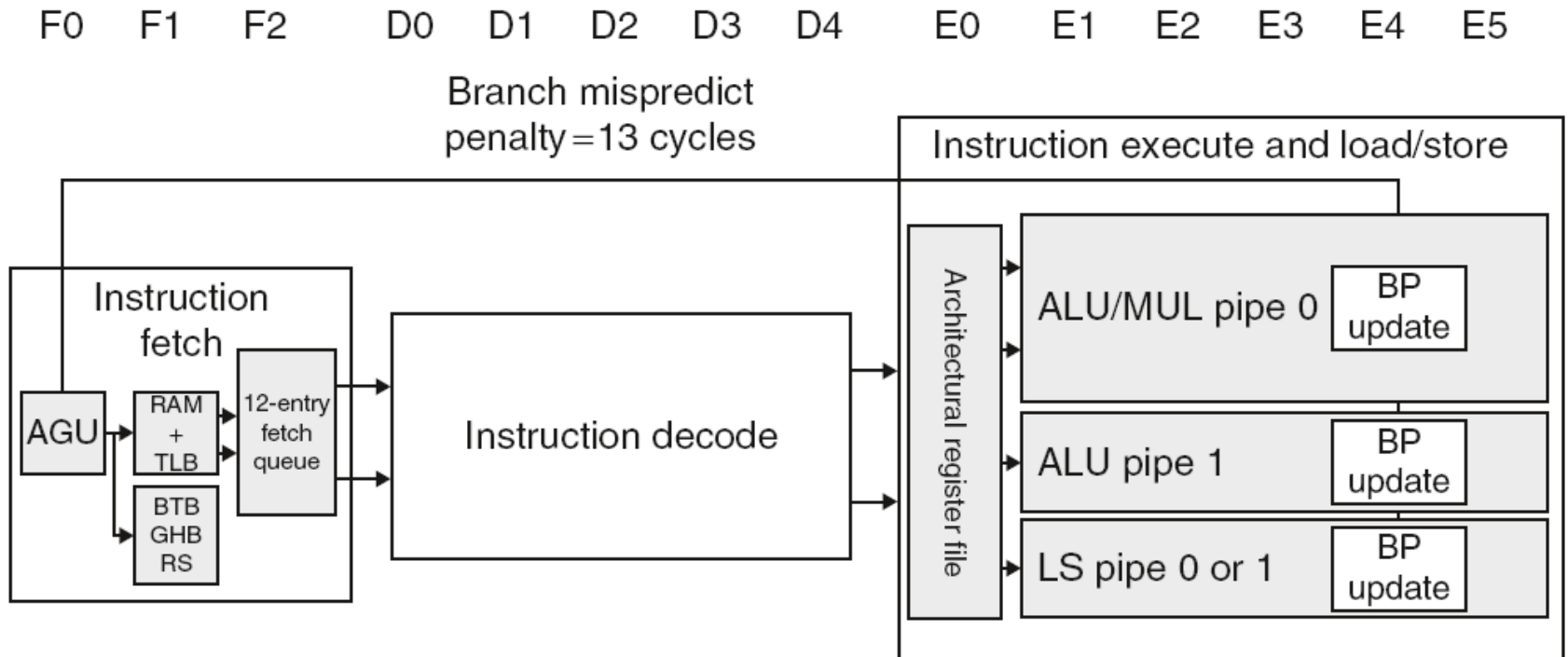
- Complexity of dynamic scheduling and speculations requires power
- Multiple simpler cores may be better

| Microprocessor | Year | Clock Rate | Pipeline Stages | Issue width | Out-of-order/ Speculation | Cores | Power |
|----------------|------|------------|-----------------|-------------|---------------------------|-------|-------|
| i486           | 1989 | 25MHz      | 5               | 1           | No                        | 1     | 5W    |
| Pentium        | 1993 | 66MHz      | 5               | 2           | No                        | 1     | 10W   |
| Pentium Pro    | 1997 | 200MHz     | 10              | 3           | Yes                       | 1     | 29W   |
| P4 Willamette  | 2001 | 2000MHz    | 22              | 3           | Yes                       | 1     | 75W   |
| P4 Prescott    | 2004 | 3600MHz    | 31              | 3           | Yes                       | 1     | 103W  |
| Core           | 2006 | 2930MHz    | 14              | 4           | Yes                       | 2     | 75W   |
| UltraSparc III | 2003 | 1950MHz    | 14              | 4           | No                        | 1     | 90W   |
| UltraSparc T1  | 2005 | 1200MHz    | 6               | 1           | No                        | 8     | 70W   |

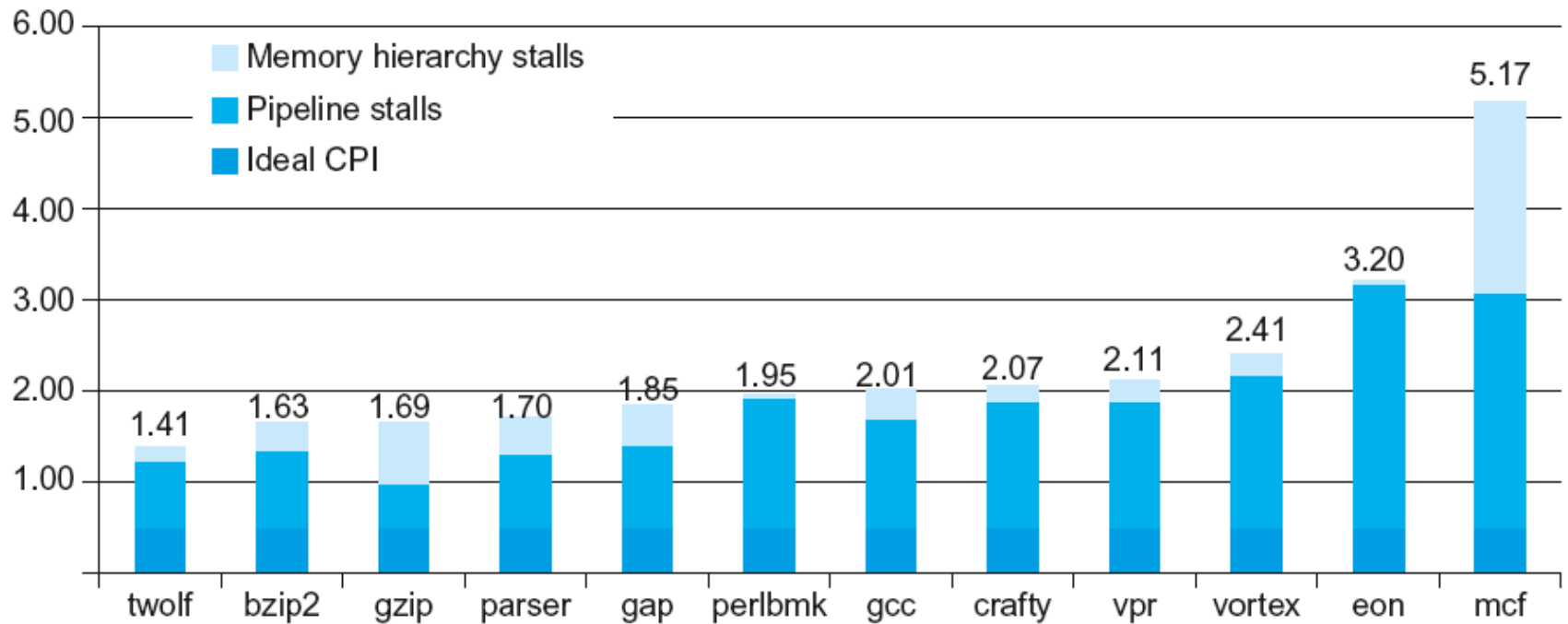
# Cortex A8 and Intel i7

| Processor                             | ARM A8                 | Intel Core i7 920                     |
|---------------------------------------|------------------------|---------------------------------------|
| Market                                | Personal Mobile Device | Server, cloud                         |
| Thermal design power                  | 2 Watts                | 130 Watts                             |
| Clock rate                            | 1 GHz                  | 2.66 GHz                              |
| Cores/Chip                            | 1                      | 4                                     |
| Floating point?                       | No                     | Yes                                   |
| Multiple issue?                       | Dynamic                | Dynamic                               |
| Peak instructions/clock cycle         | 2                      | 4                                     |
| Pipeline stages                       | 14                     | 14                                    |
| Pipeline schedule                     | Static in-order        | Dynamic out-of-order with speculation |
| Branch prediction                     | 2-level                | 2-level                               |
| 1 <sup>st</sup> level caches/core     | 32 KiB I, 32 KiB D     | 32 KiB I, 32 KiB D                    |
| 2 <sup>nd</sup> level caches/core     | 128-1024 KiB           | 256 KiB                               |
| 3 <sup>rd</sup> level caches (shared) | -                      | 2- 8 MB                               |

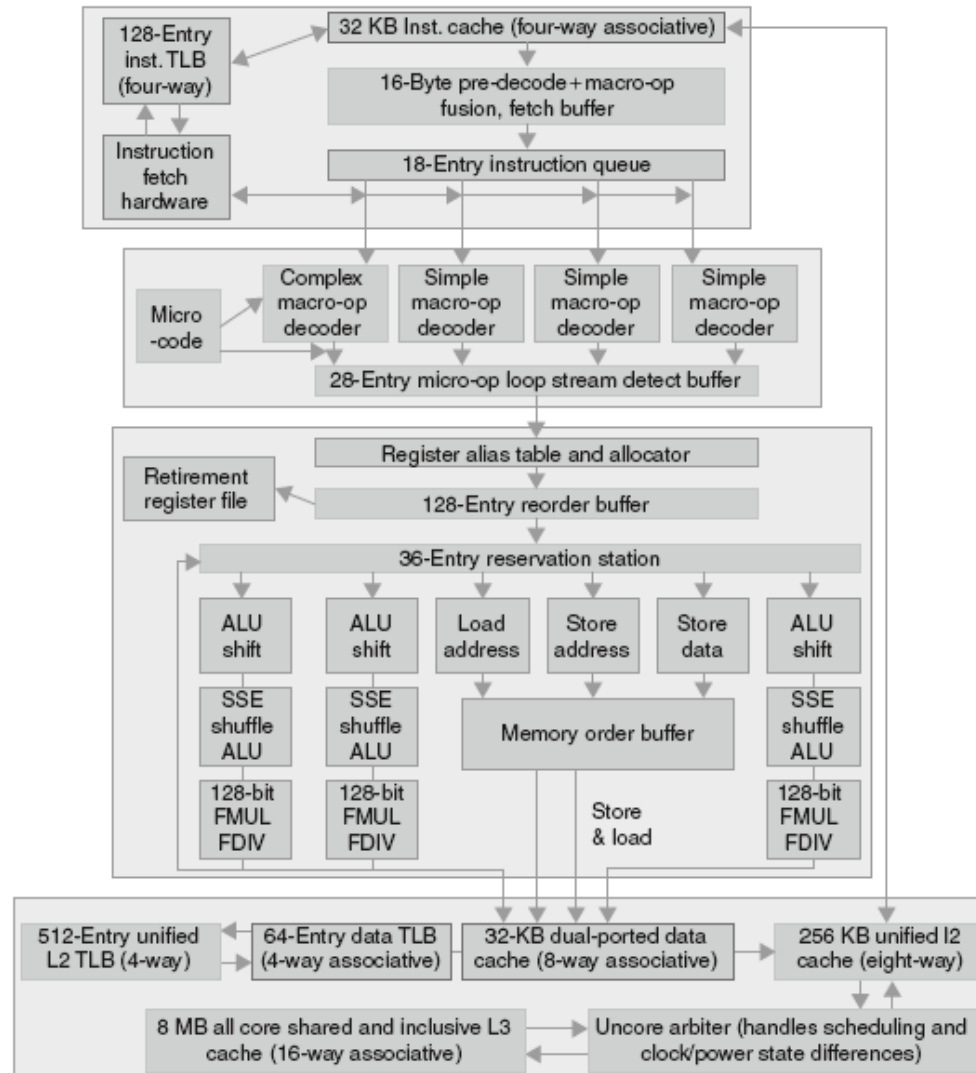
# ARM Cortex-A8 Pipeline



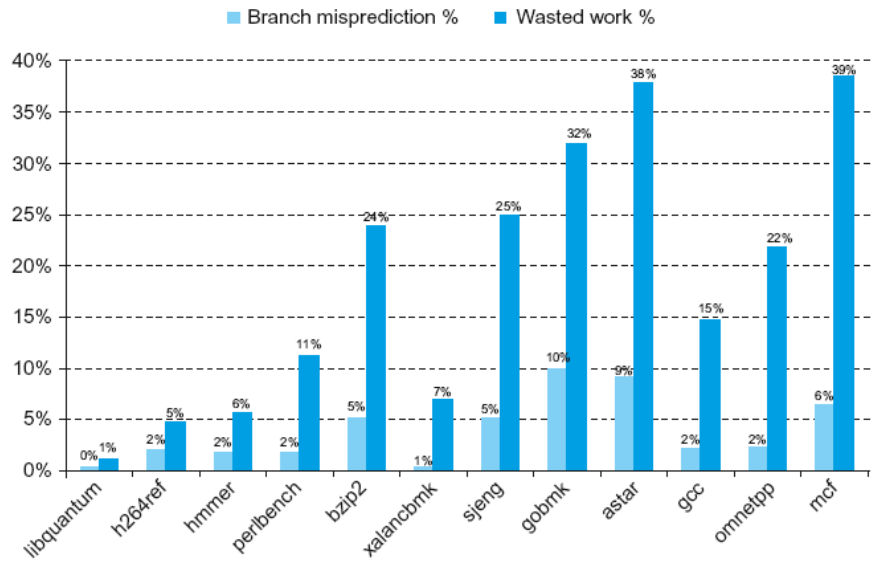
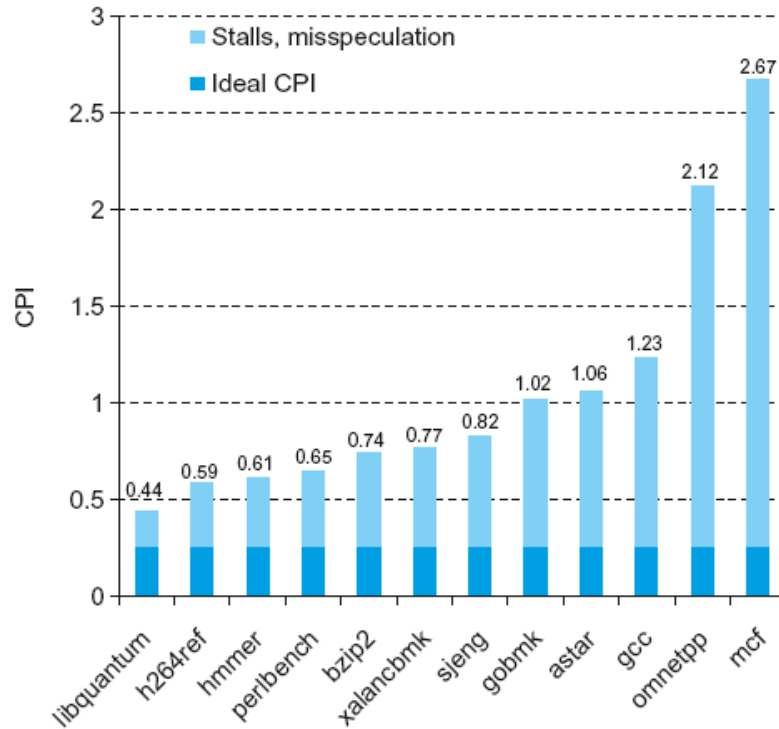
# ARM Cortex-A8 Performance



# Core i7 Pipeline



# Core i7 Performance



# Fallacies

- Pipelining is easy (!)
  - The basic idea is easy
  - The devil is in the details
    - e.g., detecting data hazards
- Pipelining is independent of technology
  - So why haven't we always done pipelining?
  - More transistors make more advanced techniques feasible
  - Pipeline-related ISA design needs to take account of technology trends
    - e.g., predicated instructions

# Pitfalls

- Poor ISA design can make pipelining harder
  - e.g., complex instruction sets (VAX, IA-32)
    - Significant overhead to make pipelining work
    - IA-32 micro-op approach
  - e.g., complex addressing modes
    - Register update side effects, memory indirection
  - e.g., delayed branches
    - Advanced pipelines have long delay slots

# Concluding Remarks

- ISA influences design of datapath and control
- Datapath and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
  - More instructions completed per second
  - Latency for each instruction not reduced
- Hazards: structural, data, control
- Multiple issue and dynamic scheduling (ILP)
  - Dependencies limit achievable parallelism
  - Complexity leads to the power wall